توثيق مشروع MediCard

الدليل الفني الكامل لتطبيق Flutter

الإصدار 1.0.0 — Build 2

تاريخ التوليد: 14 أكتوبر 2026

محتويات الملف

1 نظرة عامة

2 المعمارية

3 الميزات

4 واجهات API والتنقل

5 التطوير والصيانة

6 مرجع الكود الكامل

يتضمن الملف شرح المعمارية والميزات وواجهات API ومسارات التنقل ودليل التطوير ومرجع جميع تصريحات Dart المكتوبة يدويًا.

MediCard Mobile

تطبيق Flutter لإدارة البطاقة الطبية والوصول إلى مقدمي الخدمات والباقات، البحث والحجز، السلة والدفع، الفواتير، أفراد الأسرة، الملف الشخصي، والإشعارات. يدعم التطبيق العربية والإنجليزية ويعمل على Android وiOS.

ابدأ من هنا

المتطلبات المستخدمة حاليًا:

- Flutter `3.44.1` أو إصدار متوافق.
- Dart `3.10.7+` وفق `pubspec.yaml`.
- Xcode وCocoaPods لتشغيل iOS، وAndroid Studio/SDK لتشغيل Android.
- إعدادات Firebase الموجودة في `lib/firebase_options.dart` وملفات المنصات.

```
flutter pub get
```

```
dart run build_runner build --delete-conflicting-outputs
flutter run
```

للتحقق من المشروع:

```
flutter analyze
flutter test
```

خريطة التوثيق

- نسخة PDF الشاملة: كل التوثيق ومرجع الكود في ملف واحد قابل للبحث والطباعة.
- دليل التوثيق: نقطة الدخول لكل مستندات المشروع.
- المعمارية وتدفّق البيانات: طبقات التطبيق، بدء التشغيل، المصادقة، الشبكة، الحالة، والتخزين.
- دليل الـ Features: مسؤولية كل ميزة وأهم ملفاتها وتدفّقها.
- REST API والـ Routes: جميع الـ endpoints ومسارات التنقل والمدخلات المطلوبة.
- دليل التطوير والصيانة: إضافة Feature أو endpoint، التوليد، الاختبارات، وقائمة التحقق.
- مرجع الكود الكامل: فهرس مولّد لكل class/constructor/function/method في الملفات المكتوبة يدويًا، مع روابط للأسطر.

المعمارية باختصار

المسار المعتاد للبيانات هو:

```
Screen / Widget
      ↓ event
Cubit → Use Case → Repository contract → Repository implementation
                                                ↓
                                  Retrofit data source / local storage
                                                ↓
                                          REST API / device
```

تستخدم أغلب الميزات تقسيم `data/domain/presentation`. ما زالت بعض الأجزاء الأقدم موجودة في `lib/data` و`lib/domain` و`lib/presentation`، بينما صفحة مقدم الخدمة موجودة في `lib/medicard_network`. راجع شرح المعمارية قبل نقل أو إضافة ملفات.

أهم التقنيات

- `flutter_bloc` لإدارة الحالة.
- `get_it` لحقن الاعتماديات من `lib/core/di/service_locator.dart`.
- `retrofit` + `dio` للاتصال بالـ API.
- `freezed` لحالات Cubit و`json_serializable` للـ DTOs.
- `go_router` للتنقل.
- `easy_localization` للغتين العربية والإنجليزية.
- `flutter_secure_storage` لحفظ الـ tokens، و`shared_preferences` للإعدادات غير الحساسة.
- Firebase Messaging مع local notifications للإشعارات.

بنية المشروع

```
lib/
```

```
├── core/                # الشبكة، المصادقة، DI، routing، theme، وخدمات مشتركة
├── features/            # الميزات الحديثة مقسمة إلى data/domain/presentation
├── medicard_network/    # صفحة مقدم الخدمة والباقات والخدمات والتقييمات
├── data/ domain/        # نماذج وعقود/use cases أقدم مشتركة
├── presentation/        # شاشات login/register القديمة وحالتها
├── network/             # نماذج صفحة مقدم الخدمة
├── repository/          # تنفيذ المستودع الرئيسي
├── service/             # MedicardApiService الرئيسي
├── firebase_options.dart
└── main.dart

assets/                  # صور، خطوط، وترجمات ar/en
test/                    # unit وwidget tests
tool/                    # أدوات التوثيق والتوليد المحلية
docs/                    # توثيق المشروع
```

توليد الملفات والمرجع

بعد تعديل model أو Retrofit service أو Freezed state:

```
dart run build_runner build --delete-conflicting-outputs
```

بعد إضافة أو إعادة تسمية class/function/method:

```
dart run tool/generate_code_reference.dart
```

ولتحديث نسخة الـ PDF الشاملة:

```
dart run tool/generate_documentation_pdf.dart
```

لا تعدّل يدويًا ملفات `*.g.dart` أو `*.freezed.dart` أو `docs/CODE_REFERENCE.md`.

ملاحظات إعداد مهمة

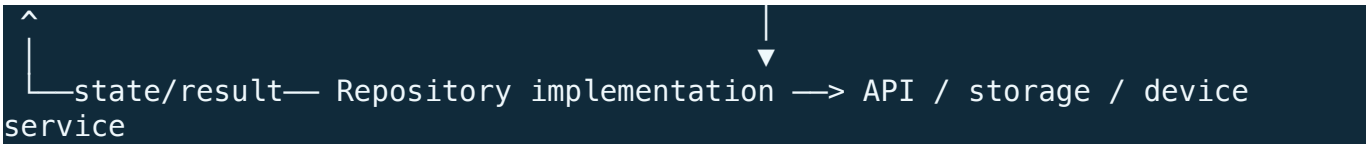
- عنوان الـ API الأساسي الحالي هو `https://medicard-api-v2.medicardeg.com/api/` وموجود في أكثر من Retrofit service وفي `DioFactory`.
- التطبيق يستخدم صلاحيات الموقع، الإشعارات، والصور بحسب الـ feature والمنصة.
- مفاتيح الترجمة يجب إضافتها في `assets/translations/ar.json` و`assets/translations/en.json` معًا.
- بيانات الجلسة الحساسة تُحفظ في secure storage؛ لا تُطبع الـ tokens في logs.
- إصدار التطبيق الحالي في `pubspec.yaml` هو `1.0.0+2`.

معمارية التطبيق

الصورة العامة

MediCard تطبيق Flutter يعتمد على Feature-first Clean Architecture بصورة عملية وليست صارمة بالكامل. الميزات الحديثة داخل `lib/features` تفصل بين `data` و`domain` و`presentation`، بينما أجزاء المصادقة والبطاقة الأقدم ما زالت موزعة على مجلدات الجذر. إدارة الحالة تتم غالبًا بواسطة Cubit، وGetIt ينشئ الاعتماديات، وDio/Retrofit ينفّذان طلبات الشبكة.

```
UI ──event──> Cubit ──> Use Case ──> Repository interface
```



مسؤولية الطبقات

- `presentation`: الشاشات والـ widgets والـ Cubits والـ states. لا يفترض أن تعرف تفاصيل HTTP أو شكل التخزين.
- `domain`: نماذج العمل، عقود repositories، والـ use cases. هذه الطبقة تصف ما يستطيع التطبيق فعله.
- `data`: Retrofit data sources وDTOs وتنفيذ repositories وتحويل DTO إلى domain model.
- `core`: بنية مشتركة مثل الشبكة، session، الأخطاء، DI، routing، اللغة، theme، والتخزين.

بدء التشغيل

ينفّذ `main()` الخطوات التالية بالترتيب:

1. تهيئة Flutter bindings و`easy_localization` وبيانات تنسيق التاريخ للعربية والإنجليزية.
2. تهيئة Firebase باستخدام `DefaultFirebaseOptions`.
3. قراءة لغة التطبيق بواسطة `AppLanguageManager`.
4. تشغيل `setupServiceLocator()` لتسجيل الشبكة والمستودعات والـ use cases والـ Cubits.
5. تحديد أول route بواسطة `AppRouter.getInitialLocation()` مع timeout مدته 10 ثوانٍ.
6. تشغيل `MyApp` داخل `EasyLocalization`.
7. بعد أول frame، تشغيل مراقبة الاتصال وPush Notifications دون حجب ظهور الواجهة.
8. عرض فيديو splash فوق التطبيق إلى أن ينتهي أو يعمل fallback timer.

اختيار أول شاشة

`AppRouter.getInitialLocation()` يطبّق القرار التالي:

- لا توجد session: يفتح onboarding لأول مرة، وإلا guest home.
- access token صالح: يفتح authenticated home.
- access token منتهي وrefresh token صالح: يحاول silent refresh ثم يفتح home عند النجاح.
- refresh token منتهي أو refresh فشل بشكل نهائي: يمسح session ويفتح login.
- خطأ غير متوقع: يعود إلى guest home بدل تعطيل بدء التطبيق.

حقن الاعتماديات

الـ service locator العام هو `sl = GetIt.instance`. التسجيل مرتب لأن بعض الخدمات تعتمد على سابقتها:

1. `AppLanguageManager` و`AppRefreshCubit`.
2. `AuthSessionStorage` وDio منفصل للـ refresh و`AuthRefreshService`.
3. Dio الرئيسي ثم `AuthInterceptor`.
4. Retrofit API clients.
5. Repository implementations.

6. Use cases.
7. Cubits.

يستخدم المشروع ثلاثة lifetimes:

- `registerLazySingleton`: للخدمات المشتركة والعملاء والمستودعات التي يجب إعادة استخدامها.
- `registerFactory`: لـ Cubit/UseCase جديد عند كل طلب.
- `registerFactoryParam`: عندما يحتاج الإنشاء قيمة runtime؛ مثل `ProviderPageCubit(providerId)` و`CartCubit(providerId?)`.

الشبكة والأخطاء

`DioFactory` ينشئ Dio واحدًا بالـ base URL الأساسي وtimeouts، ثم يضيف:

- `Accept-Language` من اللغة الحالية.
- logger في debug مع منع طباعة Authorization واستثناء payloads الحساسة.
- retry logic للطلبات المؤهلة.
- error mapping لتحويل أخطاء Dio إلى `AppFailure` مفهوم.
- `AuthInterceptor` بعد إنشاء خدمات session لمنع dependency cycle.

تُعاد نتائج معظم repositories كـ `ApiResult<T>`، إما `success(data)` أو `failure(message)`. واجهة المستخدم تتعامل مع حالات loading/success/failure من Cubit، بينما `parseApiError` وطبقة error mapping توحّدان الرسائل قدر الإمكان.

دورة المصادقة والـ token refresh

- `AuthSession` يحمل access/refresh tokens وتواريخ انتهائهما بصيغة UTC.
- `AuthSessionStorage` يحفظ القيم الحساسة في `FlutterSecureStorage` ويحافظ على مفتاح `userToken` القديم للتوافق.
- يعتبر access token بحاجة للتجديد قبل انتهائه بدقيقتين.
- `AuthRefreshService` يستخدم Dio منفصلًا بلا `AuthInterceptor` لمنع loop، ويطبّق single-flight حتى تشترك الطلبات المتزامنة في refresh واحد.
- `AuthInterceptor` يضيف Bearer token، وينفّذ refresh عند الحاجة، ثم يعيد الطلب المناسب.
- عند انتهاء session، يطلق `AppRefreshCubit.sessionExpired`؛ يستمع `MyApp` للحدث، يمسح header القديم وينتقل إلى `/medicard-login`.
- لا ينبغي تسجيل token أو password أو refresh payload في console.

إدارة الحالة والتزامن

كل شاشة كبيرة تقريبًا لها Cubit وحالة immutable أو Freezed. النمط المتوقع:

1. الواجهة ترسل intent مثل `loadProfile()` أو `addToCart()`.
2. Cubit يمنع التكرار إن كان هناك طلب مماثل قيد التنفيذ عند الحاجة.
3. Cubit يصدر loading/optimistic state.
4. Use case أو repository ينفّذ العملية.

5 Cubit يصدر success أو يعيد الحالة السابقة مع error.

حالات خاصة مهمة:

- `CartSyncService` يبث تغييرات السلة بين search/provider/cart دون ربط الـ Cubits ببعضها مباشرة.
- `CartCubit` يحتوي guards لإنشاء invoice وتعيين provider، ويعالج حالة timeout ذات النتيجة غير المعروفة بالمصالحة مع الخادم.
- `HomeCubit` و`ProfileCubit` و`ECardCubit` يمنعون تكرار الطلبات المتزامنة ويحتفظون ببيانات سابقة أثناء refresh.
- `ProviderPageCubit` يقسم الحالة لتقليل rebuilds بين header/content/search/cart bar/ package sheet.
- `AppRefreshCubit` قناة أحداث عامة لانتهاء session وتغيير اللغة وتحديث ملخص الملف والفواتير.

التخزين والكاش

- secure storage: access token، refresh token، وتواريخ الانتهاء.
- shared preferences: اللغة، onboarding، ورقم البطاقة أو مفاتيح legacy غير الحساسة.
- `NetworkCacheService`: يحفظ بيانات الدعم في shared preferences بصيغة JSON لمدة 48 ساعة؛ فشل الكاش لا يجب أن يمنع طلب الشبكة أو يعطل التطبيق.
- `cached_network_image`: كاش صور الشبكة على مستوى العرض.

اللغة والاتجاه

- اللغات المدعومة: `ar` و`en`.
- ملفات النصوص: `assets/translations/ar.json` و`assets/translations/en.json`.
- `AppLanguageManager` يقرأ اللغة المحفوظة أو لغة الجهاز، ثم يضيف `Accept-Language` لكل طلب.
- `LocalizationCubit` يحدّث `EasyLocalization` ويطلق refresh للـ router حتى تعاد ترجمة الصفحات.
- الخط الافتراضي يتحدد عبر `FontManager`: Cairo للعربية وRoboto للإنجليزية.

التنقل

`AppRouter` ينشئ `GoRouter` واحدًا. كل route مسؤول عن إنشاء Cubit الخاص به أو إعادة استخدام Cubit مرّرته الشاشة السابقة في `normalizeLocation`. `state.extra` يحافظ على روابط legacy مثل `/home` و`currentLocation`. `/login` يسمح لعناصر overlay مثل floating cart button بالاختفاء في شاشات محددة.

راجع API_AND_ROUTES.md لقائمة المسارات ومدخلاتها.

الإشعارات

`PushNotificationService` يهيّئ FCM وlocal notifications وقنوات Android، ويتعامل مع foreground/background/tap/token refresh. الرسالة التي تحتوي notification في الخلفية يترك عرضها للنظام لتجنب التكرار، بينما data-only message يمكن تحويلها إلى local notification إذا احتوت title/body.

الضغط على الإشعار يُحوّل payload إلى `NotificationAction` ثم إلى route مناسب.

الملفات المولّدة

- Retrofit وJSON serialization ينتجان `*.g.dart`.
- Freezed ينتج `*.freezed.dart`.
- `firebase_options.dart` ناتج FlutterFire ويُحدّث بواسطة أدوات Firebase.
- لا تعدّل هذه الملفات يدويًا. عدّل المصدر ثم شغّل:

```
dart run build_runner build --delete-conflicting-outputs
```

مناطق legacy والانتباه عند التعديل

- يوجد Profile API قديم يعتمد `{lang}` و`CardNo` بجانب Profile v2 المعتمد على session.
- login/register الأساسيان تحت `lib/presentation` وrepository عام تحت `lib/repository`، وليس داخل `features/auth` بالكامل.
- بعض endpoints تستخدم URL كاملًا بدل الـ base URL؛ راجع جدول API قبل تغيير البيئة.
- توجد نماذج API وdomain منفصلة في cart/activity/search؛ حافظ على mapping داخل data layer ولا تمرر DTO مباشرة للواجهة.

دليل الـ Features

خريطة سريعة

Feature	المسؤولية	إدارة الحالة / المدخل الرئيسي	مصدر البيانات
`activate_card`	تفعيل بطاقة المستخدم	`ActivateCardCubit`	`ActivateCardRepository`
`activity`	قائمة الفواتير، التفاصيل، الإلغاء، والتقييم	`ActivityCubit`, `InvoiceDetailsCubit`	`InvoiceRepository`
`auth`	نسيت كلمة المرور وOTP وإعادة التعيين	`ForgotPasswordCubit`	`MedicardRepository`
`cart`	السلة، اختيار مقدم الخدمة والفرع، checkout	`CartCubit`	`CartRepository`, `InvoiceRepository`
`content`	السياسات والشروط من HTML	`ContentCubit`	`ContentRepository`
`ecard`	البطاقة الطبية الرقمية	`ECardCubit`	`MedicardRepository`

`family_members`	إضافة/تعديل/حذف أفراد الأسرة	`FamilyMembersCubit`	`FamilyMembersRepository`
`guest_home`	الصفحة الرئيسية بدون تسجيل	`GuestHomeCubit`	`MedicardRepository`
`help_center`	واجهة قنوات المساعدة	`HelpCenterScreen`	URL/WhatsApp launcher
`home`	الصفحة الرئيسية للمستخدم والموقع	`HomeCubit`, `LocationCubit`	`MedicardRepository`, location services
`notifications`	قائمة الإشعارات وFCM وحالة القراءة	`NotificationCubit`, `PushNotificationService`	`NotificationRepository`
`packages`	عرض كل الباقات والبحث التدريجي	`AllPackagesScreen`	`MedicardNetworkRepository`
`profile`	الملف، الصورة، اللغة، كلمة المرور، تفعيل بطاقة عضو	`ProfileCubit`, `ChangePasswordCubit`	`MedicardRepository`
`search`	بحث مقدمي الخدمة والخدمات والفلاتر	`SearchCubit`, `FilterCubit`	`SearchRepository`, `CartRepository`
`medicard_network`	صفحة مقدم الخدمة وفروعه وخدماته وباقاته وتقييماته	`ProviderPageCubit`	`MedicardNetworkRepository`

المصادقة والتسجيل

Register وLogin

الجزء الأساسي موجود في `lib/presentation` مع `MedicardLoginCubit` و`RegisterCubit`. كلاهما يستدعي use cases في `lib/domain/usecases`، والتي تمر عبر `MedicardRepository`. نجاح الدخول يحفظ `AuthSession` وبيانات التوافق القديمة ثم يسمح بالانتقال إلى home.

Forgot password

التدفّق ثلاثي المراحل:

1. `sendOtp(phoneNumber)` ثم الانتقال إلى `/forgot-password/verify` مع `phoneNumber` و`otp`.
2. `verifyOtp(...)` ثم الانتقال إلى `/forgot-password/reset` مع token التحقق.
3. `resetPassword(...)` ثم العودة إلى login عند النجاح.

الـ OTP الظاهر في الاستجابة/الشاشة مرتبط بسلوك الخادم الحالي ويجب عدم اعتباره آلية production آمنة إن تغيّر عقد الـ API.

Guest Homeو Home

`HomeCubit` يجمع home payload المحمي مع الموقع، ويحوّل DTOs إلى نماذج عرض عبر `home_mappers.dart`. الشاشة تعرض البطاقة، الخدمات المميزة، أقرب مقدمي الخدمة، المواعيد المعلقة، البنرات، والتنقل السفلي. `LocationCubit` يدير الإذن والموقع والعنوان.

`GuestHomeCubit` يحمّل نسخة عامة من الصفحة ويعيد استخدام نماذج العرض عبر `guest_home_mappers.dart`. البحث من وضع guest يمرر `enableCartFeatures: false` ويستخدم endpoint الخدمات العام عند الحاجة.

Search

`SearchCubit` يدير مسارين داخل الشاشة نفسها:

- Providers: keyword، category، governorate، city، sort، الموقع، pagination.
- Services: service categories ثم خدمات الفئة، مع كاش لكل فئة وpagination.

`FilterCubit` يحمّل lookups ويحتفظ بالاختيارات المؤقتة. عند sort بالأقرب يمكن طلب الموقع. عمليات إضافة/حذف خدمة من السلة optimistic وتمنع تكرار نفس العملية؛ `CartSyncService` يبث النتيجة لباقي الشاشات.

صفحة مقدم الخدمة والباقات

`ProviderPageCubit` يبدأ بـ `providerId` ويحمّل معلومات مقدم الخدمة، الفروع، والخدمات. يستطيع المستخدم التبديل بين الخدمات والباقات والتقييمات، البحث، التحميل التدريجي، تصفح فئات الخدمات، وإضافة/حذف خدمات من السلة. تفاصيل الباقة تُحمّل عند الطلب، ويمكن إنشاء invoice لباقة مباشرة بعد اختيار الفرع وطريقة الدفع.

`AllPackagesScreen` يعرض كل الباقات بصفحات مع search. عند فتح مقدم خدمة من notification يمكن تمرير `autoOpenPackageId` لفتح تفاصيل باقة تلقائيًا.

Checkoutو Cart

السلة تستخدم domain models منفصلة عن DTOs. أهم العمليات في `CartCubit`:

- تحميل السلة وخدمات الفئة وإضافة/حذف خدمة ومسح السلة.
- تحميل مقدمي الخدمة القادرين على تغطية عناصر السلة.
- تعيين provider ومصالحة العناصر غير المغطاة.
- تحميل الفروع وأفراد الأسرة واختيار طريقة الدفع.
- إنشاء invoice من cart أو package.
- منع POST مكرر عند الضغط المتزامن.
- عند timeout بعد إرسال الإنشاء، اعتبار النتيجة “unknown” ثم فحص قائمة الفواتير بدل إعادة POST عشوائيًا.

`CartSyncService` يرسل snapshot أو تغيير عنصر بين `SearchCubit` و`ProviderPageCubit` و`CartCubit`. يجب بث التغيير بعد أي mutation ناجح حتى تظل badges والـ floating button متطابقة.

Activity والفواتير

`ActivityCubit` يحمّل invoices حسب tab مع pagination ويمنع تداخل load/loadMore.
`InvoiceDetailsCubit` يحمّل التفاصيل، يلغي invoice، ويرسل review مع الاحتفاظ بالتفاصيل المعروضة عند فشل mutation. بعد إنشاء أو إلغاء invoice يمكن استخدام `AppRefreshCubit.triggerInvoicesRefresh()` لتحديث القائمة.

E-Card وProfile

`ProfileCubit` يحمل profile كاملًا ثم يحمّل summary بصورة مستقلة؛ فشل summary لا يخفي بيانات الملف. العمليات: تعديل البيانات والصورة، حذف الصورة، تغيير اللغة، تفعيل بطاقة عضو، refresh للملخص، وlogout. `ChangePasswordCubit` منفصل لأن له شاشة وحالة عملية مستقلة.

`ECardCubit` يحمّل `Profile/card-data` ويحافظ على طلب واحد قيد التنفيذ. البطاقة تظهر داخل home preview ويمكن تحديثها بقوة عند تغير بيانات المستخدم.

أفراد الأسرة

`FamilyMembersCubit` ينفّذ list/create/update/delete. الحد المحلي الحالي `kMaxFamilyMembers = 3`. بعد create/delete يطلق تحديث ملخص profile حتى يتغير العداد والنقاط في الواجهة. شاشة add/edit مشتركة، ويُعاد استخدام نفس Cubit عند الانتقال من القائمة للحفاظ على البيانات.

الإشعارات

هناك جزآن:

- `NotificationCubit`: pagination، optimistic mark-as-read/mark-all مع rollback عند الفشل، والاستماع للرسائل الجديدة.
- `PushNotificationService`: صلاحيات النظام، FCM token، تسجيل token لدى الخادم، local display، والتعامل مع الضغط/deep links.

الـ repository لا يحاول تسجيل device token إذا لم توجد session. payload الإشعار يجب أن يُحلل دفاعيًا لأن القيم قد تصل كنصوص أو أرقام.

Help Center وContent

`ContentType` يربط privacy/refund/terms بـ slug ومفتاح عنوان. `ContentCubit` يمنع تكرار نفس الطلب ويدعم force retry. `ContentScreen` يعرض HTML، و`ContentLinkHandler` يسمح بالروابط الداخلية المعروفة ويفتح http/https/tel/mailto خارجيًا ويرفض schemes غير المدعومة.

`HelpCenterScreen` يعرض قنوات الدعم ويستخدم `UrlLauncherService` لفتح الهاتف والبريد وWhatsApp مع رسائل خطأ مفهومة إذا تعذر الفتح.

أين أضيف كودًا جديدًا؟

- API DTO: `features/<name>/data/dto` أو `data/models`.

- Retrofit interface: `data/data_source` أو `data/api`.
- business model/contract/use case: `domain`.
- state and user actions: `presentation/cubit`.
- full page: `presentation/pages` أو `screens` بحسب النمط الموجود في feature.
- reusable UI: `presentation/widgets`.
- سلوك مشترك لكل الميزات: `core` فقط إذا كان مستقلًا فعلًا عن feature بعينها.

REST API والـ Routes

إعداد الشبكة العام

- Base URL: `https://medicard-api-v2.medicardeg.com/api/`
- `Accept-Language`: يضاف تلقائيًا بقيمة `ar` أو `en`.
- `Authorization`: يضيفه `AuthInterceptor` عند وجود session. بعض clients القديمة تمرره صراحة كـ `Bearer <token>`.
- timeout الافتراضي في `DioFactory`: راجع المصدر عند تعديل سياسة المهلة أو retry.
- URLs الكاملة الموضحة في الجداول تتجاوز الـ base URL ويجب تغييرها صراحة عند تبديل البيئة.

Endpoints الرئيسية

المصدر: `lib/service/medicard_api_service.dart`.

الغرض	Function	Path	Method
إنشاء حساب	`register`	`Auth/register`	POST
تسجيل الدخول وإرجاع session	`login`	`Auth/login`	POST
إرسال OTP	`sendOtp`	`Auth/forgot-password/send-otp`	POST
التحقق من OTP	`verifyOtp`	`Auth/forgot-password/verify-otp`	POST
تعيين كلمة مرور جديدة	`resetPassword`	`Auth/forgot-password/reset`	POST
بيانات home القديمة بالبطاقة	`getHomeInfo`	`{lang}/Home/GetHomeInfo`	GET
بيانات الملف القديمة	`getPersonalInfo`	`{lang}/Profile/GetPersonalInfo`	GET

قنوات الدعم	`getSupportContacts`	`{lang}/ContactUs/Get`	GET
home للمستخدم مع الموقع الاختياري	`getHomeData`	`mobile/MobileHome`	GET
تحديث profile القديم	`updateProfile`	`{lang}/Profile/UpdateProfile`	PUT multipart
profile v2	`getProfile`	`Profile`	GET
العدادات والنقاط والملخص	`getProfileSummary`	`Profile/summary`	GET
بيانات البطاقة الرقمية	`getCardData`	`Profile/card-data`	GET
حذف صورة الملف	`deleteProfileImage`	`Profile/profile/image`	DELETE
تفعيل بطاقة عضو	`activateMemberCard`	`CardActivation/member-card/activate`	POST
تحديث profile v2 والصورة	`updateProfileV2`	`Profile`	PUT multipart
تغيير كلمة المرور	`changePassword`	`Auth/change-password`	POST
حفظ لغة المستخدم على الخادم	`changeLanguage`	`Profile/language`	PUT
HTML للسياسات والشروط	`getContent`	`Content/{slug}`	GET
قائمة أفراد الأسرة	`getFamilyMembers`	`FamilyMembers`	GET
فرد واحد	`getFamilyMember`	`FamilyMembers/{id}`	GET
إضافة فرد	`createFamilyMember`	`FamilyMembers`	POST
تعديل فرد	`updateFamilyMember`	`FamilyMembers/{id}`	PUT
حذف فرد	`deleteFamilyMember`	`FamilyMembers/{id}`	DELETE
home العام مع الموقع الاختياري	`getGuestHomeData`	`GuestMobileHome/guest`	GET

Packages وProvider

المصدر: `lib/medicard_network/service/medicard_network_api_service.dart`.

الغرض	Function	Path	Method
تفاصيل مقدم الخدمة	`getProviderPage`	`ProviderPage/{providerId}`	GET
الفروع والمواقع	`getProviderBranchLocations`	`ProviderPage/{providerId}/branchLocations`	GET
خدمات مقدم الخدمة مع search/paging	`getProviderServices`	`https://medicard-api-v2.medicardeg.com/services`	GET
التقييمات	`getProviderReviews`	`ProviderPage/{providerId}/reviews`	GET
باقات مقدم الخدمة	`getProviderPackages`	`ProviderPage/{providerId}/packages`	GET
تفاصيل باقة	`getPackageDetails`	`Package/{packageId}`	GET
كل الباقات مع search/ paging	`getAllPackages`	`Package`	GET
فئات خدمات مقدم الخدمة	`getProviderCategories`	`CategoryService/providerCategories`	GET
خدمات فئة لدى مقدم محدد	`getServicesByCategory`	`ProviderPage/servicesByCategory`	GET

Search

المصدر: `lib/features/search/data/data_sources/search_remote_data_source.dart`.

أهم المدخلات	Function	Path	Method
keyword/category/location/ sort/page	`searchProviders`	`Search/providers`	GET
لا يوجد	`getCategories`	`Lookups/categories`	GET
لا يوجد	`getServiceCategories`	`CategoryService/categories`	GET
لا يوجد	`getGovernorates`	`Lookups/governorates`	GET
`governorateId`	`getCities`	`Lookups/cities`	GET

GET	`Cart/category/{categoryId}/services`	`getCategoryServices`	category/page؛ يمكن تخطي auth للعرض العام
GET	`GuestMobileHome/servicesByCategory`	`getGuestCategoryServices`	category/search/page

Cart

المصدر: `lib/features/cart/data/api/cart_remote_data_source.dart`.

الغرض	Function	Path	Method
السلة الحالية	`getCart`	`Cart`	GET
إضافة خدمة	`addToCart`	`Cart`	POST
حذف خدمة	`removeFromCart`	`Cart?ServiceId=...`	DELETE
خدمات الفئة وحالة الاختيار	`getCategoryServices`	`Cart/category/{categoryId}/services`	GET
مسح السلة	`clearCart`	`Cart/clear`	DELETE
مقدمو الخدمة وتغطية العناصر	`getChooseProviders`	`Cart/choose-providers`	GET
تعيين provider للسلة؛ URL كامل خارج `/api/`	`updateCartProvider`	`https://medicard-api-v2.medicardeg.com/update`	PUT

Reviews وInvoices

المصدر: `lib/features/activity/data/data_source/invoice_remote_data_source.dart`.

الغرض	Function	Path	Method
invoices حسب tab مع pagination	`getMyInvoices`	`Invoice/my-invoices`	GET
إنشاء invoice من باقة	`createFromPackage`	`Invoice/from-package`	POST
إنشاء invoice من السلة	`createFromCart`	`ProviderBranchView/invoices/create-from-cart`	POST

تفاصيل invoice	getInvoiceDetails	ProviderBranchView/invoices/{invoiceId}/details	GET
إرسال تقييم	createReview	Review/reviews	POST
إلغاء invoice	cancelInvoice	Invoice/cancel/{invoiceId}	PUT

Notifications

المصدر: `lib/features/notifications/data/notification_client.dart`.

الغرض	Function	Path	Method
القائمة مع pagination	getNotifications	Notification	GET
إرسال إشعار عبر API	sendNotification	Notification/send	POST
تعليم إشعار كمقروء	markAsRead	Notification/{id}/read	PUT
تعليم الكل كمقروء	markAllAsRead	Notification/read-all	PUT
ربط FCM token بالمستخدم والمنصة	registerDeviceToken	Notification/register-device-token	POST

GoRouter paths

المصدر: `lib/core/routing/app_router.dart`.

`state.extra` أو query	الشاشة / السلوك	Path
إلى guest أو authenticated home حسب alias	redirect	`/`, `/Home`, `/home`
إلى `/guest-home`	redirect حاليًا	`/onboarding`
لا يوجد	`GuestHomeScreen`	`/guest-home`
لا يوجد	`MediCardLoginScreen`	`/medicard-login`
لا يوجد	`RegisterScreen`	`/medicard-register`

`/forgot-password`	`ForgotPasswordScreen`	لا يوجد
`/forgot-password/verify`	`OtpVerificationScreen`	map: `phoneNumber`, `otp`
`/forgot-password/reset`	`ResetPasswordScreen`	map: `phoneNumber`, `token`
`/medicard-activation`	`ActivateCardScreen`	لا يوجد
`/medicard-home`	`HomeScreen`	يفتح تبويب النشاط query `tab=activity`
`/notifications`	`NotificationsScreen`	لا يوجد
`/medicard-profile`	`ProfileScreen`	لا يوجد
`/medicard-edit-profile-new`	`EditProfileScreen`	map: `profileData`, و`profileCubit?`
`/change-password`	`ChangePasswordScreen`	لا يوجد
`/search`	`SearchScreen`	map اختياري: `userLatitude`, `userLongitude`, `categoryId`, `initialTabIndex`, `enableCartFeatures`
`/search/providers`	providers tab	نفس مدخلات search عدا tab ثابت 0
`/search/services`	services tab	نفس مدخلات search عدا tab ثابت 1
`/packages`	`AllPackagesScreen`	لا يوجد
`/privacy-policy`	`ContentScreen`	النوع ثابت
`/refund-policy`	`ContentScreen`	النوع ثابت
`/terms-and-conditions`	`ContentScreen`	النوع ثابت
`/family-members`	القائمة	لا يوجد
`/family-members/add`	add screen	`FamilyMembersCubit?` لإعادة الاستخدام

`/family-members/edit`	edit screen	map: `member`, `cubit?` أو `FamilyMemberModel`
`/cart`	`CartScreen`	map اختياري: `providerId`, `selectedBranch`
`/checkout`	`CheckoutScreen`	map اختياري: `cubit`, `providerId`, `selectedBranch`, وبيانات package
`/invoice-details`	`InvoiceDetailsScreen`	map: `invoice`, `tab?`
`/choose-provider`	`ChooseProviderScreen`	`CartCubit?`
`/pending-appointment-details`	تفاصيل الموعد	`AppointmentModel`
`/provider-page`	`ProviderPageScreen`	map: `providerId` أو `int providerId`, `autoOpenPackageId?`
`/provider-branches`	`BranchesScreen`	map: `branches`, `providerName?` أو list فروع
`/provider-reviews`	`ReviewsScreen`	map: `cubit`, `providerId`, `providerName?`

قواعد مهمة عند التنقل

- `state.extra` غير متسلسل؛ لا تعتمد عليه وحده في deep links خارج التطبيق.
- افحص النوع قبل cast عندما يسمح route بأكثر من شكل، كما يحدث في provider/family routes.
- استخدم `BlocProvider.value` فقط لإعادة استخدام Cubit موجود ولا تغلقه الشاشة الجديدة.
- أضف alias داخل `normalizeLocation` إذا كان تغيير path يجب أن يحافظ على روابط قديمة.

دليل التطوير والصيانة

إعداد بيئة التطوير

```
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter run
```

لـ iOS عند تغير pods:

```
cd ios
pod install
cd ..
```

استخدم إعدادات Firebase الموجودة للمشروع الصحيح. لا تضع secrets أو tokens في source أو logs أو screenshots.

إضافة Feature جديدة

البنية المقترحة:

```
lib/features/example/
├── data/
│   ├── data_source/
│   ├── dto/ أو models/
│   └── repository/
├── domain/
│   ├── models/
│   ├── repository/
│   └── usecases/
└── presentation/
    ├── cubit/
    ├── pages/
    └── widgets/
```

الترتيب العملي:

1. عرّف domain model وعقد repository.
2. عرّف DTO وRetrofit data source.
3. نفّذ mapping داخل repository implementation وأعد `ApiResult<T>` أو النوع المتبع في feature.
4. أضف use case إذا كانت العملية business action مستقلة.
5. أنشئ state وCubit، مع guards للطلبات المتزامنة إذا كان الضغط المكرر ممكنًا.
6. سجّل data source/repository/use case/Cubit في `setupServiceLocator()` بالـ lifetime المناسب.
7. أضف route وأنشئ Cubit عند حدود الشاشة.
8. أضف مفاتيح `ar.json` و`en.json` واختبارات النجاح والفشل والحواف.
9. حدّث `FEATURES.md` و`API_AND_ROUTES.md` ثم ولّد `CODE_REFERENCE.md`.

إضافة endpoint أو model

مثال مبسط:

```
@GET('Example/{id}')
Future<ExampleResponseDto> getExample(@Path('id') int id);
```

بعد تعديل Retrofit/JSON/Freezed:

```
dart run build_runner build --delete-conflicting-outputs
```

قواعد المشروع:

- لا تعدّل `*.g.dart` أو `*.freezed.dart`.
- DTO يعكس عقد الخادم ويتعامل مع null/اختلاف الأنواع دفاعيًا عند الحاجة.
- حوّل DTO إلى domain model في data layer.
- افحص `succeeded` و`data` ولا تعتبر HTTP 200 نجاحًا منطقيًا تلقائيًا.
- استخدم `parseApiError`/`AppFailure` بدل عرض `DioException.toString()` للمستخدم.
- لا تضف Authorization يدويًا إلا إذا كان client الحالي يتطلبه؛ افهم دور `AuthInterceptor` أولًا.

- استخدم `@Extras({'skipAuthInterceptor': true})` أو الآلية الموجودة فقط للـ endpoints العامة المقصودة.

إضافة route

أضف `GoRoute` في `AppRouter.createRouter`، ووثّق:

- path الثابت.
- query parameters القابلة للتسلسل.
- نوع وشكل `state.extra`.
- من يملك Cubit ومن يغلقه.
- fallback عند فتح المسار دون extra، إن كان deep-link محتملًا.

تجنب cast مباشر غير محمي إذا كانت الشاشة قابلة للفتح من notification أو deep link. لا تمرّر كائنات domain داخل URL؛ استخدم identifier وأعد تحميل البيانات عند الحاجة.

إدارة الحالة بصورة آمنة

- لا تصدر state بعد `close()`؛ افحص `isClosed` بعد العمليات الطويلة.
- احتفظ بالبيانات السابقة أثناء refresh عندما يكون إخفاؤها تجربة سيئة.
- امنع duplicate requests باستخدام in-flight future أو request id.
- عند optimistic update، خزّن snapshot لاسترجاعه عند الفشل.
- عند pagination، تجاهل response قديم إذا تغيّر query/filter أثناء الطلب.
- عملية مالية أو إنشاء invoice لا تُعاد تلقائيًا بعد timeout؛ صالح النتيجة مع الخادم أولًا.
- استخدم `CartSyncService` لتزامن السلة بدل استدعاء Cubit شاشة أخرى مباشرة.

الترجمة والتنسيق

أي نص للمستخدم يجب أن يكون له مفتاح في الملفين:

- `assets/translations/ar.json`
- `assets/translations/en.json`

استخدم `AppTextStyles` و`AppColors` و`ScreenUtil` بدل أرقام وتصميمات متكررة. اختبر RTL وLTR والنص الطويل. تنسيق التاريخ يمر عبر `AppDateFormatter/intl`، والمبالغ عبر helpers الموجودة في feature.

الاختبارات الحالية

الاختبارات الموجودة تغطي نقاطًا عالية المخاطر، منها:

- logout حتى عند فشل خطوات التنظيف.
- Cart checkout reliability، منع الإنشاء المكرر، unknown outcome، وترتيب responses.
- Invoice details/cancel/review.
- Search pagination/filter/location وتزامن السلة.
- Content parsing/link handling/retry.

- Notification optimistic read rollback وتسجيل FCM token.
- Profile summary دون كسر بيانات profile.
- Provider page scoped rebuilds وتفاصيل الباقة.

شغّل:

```
flutter test
```

ولاختبار ملف واحد:

```
flutter test test/features/cart/presentation/cubit/cart_cubit_test.dart
```

عند إصلاح bug، أضف test يفشل قبل الإصلاح إن أمكن، خصوصًا للـ Cubits وrepository mappings وحالات السباق.

فحوصات قبل الدمج

```
dart format --set-exit-if-changed lib test tool
flutter analyze
flutter test
dart run tool/generate_code_reference.dart
git diff --check
```

ثم راجع يدويًا:

- لا يوجد token/password/PII في logs.
- ملفات التوليد متزامنة مع مصادرها.
- الترجمتان تحتويان المفاتيح الجديدة.
- loading/empty/error/success كلها قابلة للوصول.
- back navigation وdeep links لا تنهار عند missing extra.
- iOS permissions/configs وAndroid محدثة إذا أضيف plugin أو capability.

مرجع الدوال المولّد

`tool/generate_code_reference.dart` يستخدم Dart analyzer لقراءة AST وليس regex، لذلك يفهرس التصريحات متعددة الأسطر والـ constructors/getters/private methods بصورة صحيحة. شغّله بعد أي تغيير في واجهات الكود:

```
dart run tool/generate_code_reference.dart
```

الناتج هو `docs/CODE_REFERENCE.md`. الوصف فيه مختصر ومبني على اسم الدالة ومسارها؛ أضف شرح التدفق والقرارات إلى المستند اليدوي المناسب بدل تعديل الناتج مباشرة.

مرجع الكود الكامل

هذا الملف مولّد آليًا من شجرة بناء Dart. لا تعدّله يدويًا؛ شغّل `dart run tool/generate_code_reference.dart` بعد تغيير الواجهات أو الدوال.

طريقة قراءة المرجع

- يغطي كل ملفات Dart المكتوبة يدويًا داخل `lib/`، بما فيها الدوال الخاصة.
- يستبعد ملفات `*.g.dart` و`*.freezed.dart` لأنها ناتج توليد وليست مصدرًا يُعدّل يدويًا.
- اضغط على أي توقيع للانتقال إلى مكانه في المصدر. وسم خاص يعني أن الاسم يبدأ بـ `_`.
- الوصف المختصر يوضح غرض الدالة من اسمها وسياق ملفها؛ المصدر المرتبط هو المرجع النهائي للتفاصيل التنفيذية.

الفهرس

- Core: auth
- Core: cache
- Core: constants
- Core: cubit
- Core: di
- Core: helpers
- Core: network
- Core: routing
- Core: services
- Core: theming
- Core: utils
- Core: widgets
- Module: data
- Module: domain
- Feature: activate_card
- Feature: activity
- Feature: auth
- Feature: cart
- Feature: content
- Feature: ecard
- Feature: family_members
- Feature: guest_home
- Feature: help_center
- Feature: home
- Feature: notifications
- Feature: packages
- Feature: profile
- Feature: search
- Module: firebase_options.dart
- Module: main.dart
- Module: medicard_network
- Module: network
- Module: presentation
- Module: repository
- Module: service

Core: auth

core/auth/auth_refresh_service.dart

- `class AuthRefreshService` — الكلاس الذي يجمع مسؤوليات auth refresh service.
- `AuthRefreshService({required AuthSessionStorage sessionStorage, required Dio refreshDio})` — ينشئ كائنًا من AuthRefreshService ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<AuthSession?> refreshToken()` — يعيد تحميل أو محاولة refresh token داخل AuthRefreshService.
- `Future<AuthSession?> _performRefresh()` — خاص — ينفّذ منطق perform refresh داخل AuthRefreshService.

core/auth/auth_session.dart

- `class AuthSession` — الكلاس الذي يجمع مسؤوليات auth session.
- `const AuthSession({required this.accessToken, required this.refreshToken, this.accessTokenExpiresAt, this.refreshTokenExpiresAt})` — ينشئ كائنًا من AuthSession ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool get hasAccessToken` — يفحص شرط has access token ويعيد النتيجة داخل AuthSession.
- `bool get hasRefreshToken` — يفحص شرط has refresh token ويعيد النتيجة داخل AuthSession.
- `bool get isRefreshTokenExpired` — يفحص شرط is refresh token expired ويعيد النتيجة داخل AuthSession.
- `bool get isAccessTokenExpiredOrAboutToExpire` — يفحص شرط is access token expired or about to expire ويعيد النتيجة داخل AuthSession.
- `AuthSession copyWith({String? accessToken, String? refreshToken, DateTime? accessTokenExpiresAt, DateTime? refreshTokenExpiresAt})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

core/auth/auth_session_storage.dart

- `class AuthSessionStorage` — الكلاس الذي يجمع مسؤوليات auth session storage.
- `const AuthSessionStorage()` — ينشئ كائنًا من AuthSessionStorage ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> saveSession(AuthSession session)` — ينفّذ منطق save session داخل AuthSessionStorage.
- `Future<void> updateTokens({required String accessToken, required String refreshToken, required DateTime? accessTokenExpiresAt, required DateTime? refreshTokenExpiresAt})` — يحدّث update tokens داخل AuthSessionStorage.
- `Future<AuthSession?> getSession()` — يجلب get session من مصدر البيانات المناسب داخل AuthSessionStorage.
- `Future<String?> getAccessToken()` — يجلب get access token من مصدر البيانات المناسب داخل AuthSessionStorage.
- `Future<String?> getRefreshToken()` — يجلب get refresh token من مصدر البيانات المناسب داخل AuthSessionStorage.
- `Future<void> clearSession()` — يحذف أو يمسح clear session داخل AuthSessionStorage.

core/auth/jwt_helper.dart

- `DateTime? getJwtExpiryDate(String token)` — يجلب get jwt expiry date من مصدر

البيانات المناسب داخل هذا الملف.

core/auth/refresh_token_dto.dart

- `class RefreshTokenRequestDto` — الكلاس الذي يجمع مسؤوليات `refresh token request dto`.
- `const RefreshTokenRequestDto({required this.refreshToken})` — ينشئ كائنًا من `RefreshTokenRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class RefreshTokenResponseDto` — الكلاس الذي يجمع مسؤوليات `refresh token response dto`.
- `const RefreshTokenResponseDto({this.token, this.refreshToken, this.refreshTokenExpiresAt, this.expiresAt})` — ينشئ كائنًا من `RefreshTokenResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory RefreshTokenResponseDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `RefreshTokenResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `static DateTime? _parseDate(dynamic value)` — خاص — يحلّل `parse date` ويحوّله لصيغة قابلة للاستخدام داخل `RefreshTokenResponseDto`.
- `class RefreshTokenEnvelopeDto` — الكلاس الذي يجمع مسؤوليات `refresh token envelope dto`.
- `const RefreshTokenEnvelopeDto({this.httpStatusCode, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `RefreshTokenEnvelopeDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory RefreshTokenEnvelopeDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `RefreshTokenEnvelopeDto` ويهيّئ الاعتماديات والقيم المطلوبة.

Core: cache

core/cache/cache_service.dart

- `class CacheService` — الكلاس الذي يجمع مسؤوليات `cache service`.
- `static Future<void> cacheSupportContactsData(dynamic data)` — ينفّذ منطق `cache support contacts data` داخل `CacheService`.
- `static Future<dynamic> getCachedSupportContactsData()` — يجلب `get cached support contacts data` من مصدر البيانات المناسب داخل `CacheService`.
- `static Future<void> clearSupportContactsCache()` — يحذف أو يمسح `clear support contacts cache` داخل `CacheService`.

core/cache/network_cache_service.dart

- `class NetworkCacheService` — الكلاس الذي يجمع مسؤوليات `network cache service`.
- `static Future<void> cacheSupportContactsData(dynamic data)` — ينفّذ منطق `cache support contacts data` داخل `NetworkCacheService`.
- `static Future<dynamic> getCachedSupportContactsData()` — يجلب `get cached support contacts data` من مصدر البيانات المناسب داخل `NetworkCacheService`.
- `static Future<void> clearSupportContactsCache()` — يحذف أو يمسح `clear support contacts cache` داخل `NetworkCacheService`.

Core: constants

core/constants/api_result.dart

- `class ApiResult` — الكلاس الذي يجمع مسؤوليات `api result`.
- `const factory ApiResult.success(T data)` — ينشئ كائنًا من `ApiResult` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ApiResult.failure(String message)` — ينشئ كائنًا من `ApiResult` ويهيّئ الاعتماديات والقيم المطلوبة.

core/constants/app_assets.dart

- `class AppAssets` — الكلاس الذي يجمع مسؤوليات `app assets`.

core/constants/constants.dart

- `class SharedPrefKeys` — الكلاس الذي يجمع مسؤوليات `shared pref keys`.

Core: cubit

core/cubit/app_refresh_cubit.dart

- `class AppRefreshCubit` — الكلاس الذي يجمع مسؤوليات `app refresh cubit`.
- `AppRefreshCubit()` — ينشئ كائنًا من `AppRefreshCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `void triggerLanguageRefresh()` — ينفّذ منطق `trigger language refresh` داخل `AppRefreshCubit`.
- `void triggerSessionExpired()` — ينفّذ منطق `trigger session expired` داخل `AppRefreshCubit`.
- `void triggerProfileSummaryRefresh()` — ينفّذ منطق `trigger profile summary refresh` داخل `AppRefreshCubit`.
- `void triggerInvoicesRefresh()` — ينفّذ منطق `trigger invoices refresh` داخل `AppRefreshCubit`.

core/cubit/app_refresh_state.dart

- `class AppRefreshState` — الكلاس الذي يجمع مسؤوليات `app refresh state`.
- `const factory AppRefreshState.initial()` — ينشئ كائنًا من `AppRefreshState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory AppRefreshState.languageChanged()` — ينشئ كائنًا من `AppRefreshState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory AppRefreshState.sessionExpired()` — ينشئ كائنًا من `AppRefreshState` ويهيّئ الاعتماديات والقيم المطلوبة.

core/cubit/localization_cubit.dart

- `class LocalizationCubit` — الكلاس الذي يجمع مسؤوليات `localization cubit`.
- `LocalizationCubit({required AppLanguageManager languageManager, required AppRefreshCubit appRefreshCubit})` — ينشئ كائنًا من `LocalizationCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> changeLanguage(BuildContext context, String language)` — يحدّث `change language` داخل `LocalizationCubit`.

core/cubit/localization_state.dart

- `class LocalizationState` — الكلاس الذي يجمع مسؤوليات `localization state`.
- `const factory LocalizationState.initial(String language)` — ينشئ كائنًا من `LocalizationState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory LocalizationState.loading()` — ينشئ كائنًا من `LocalizationState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory LocalizationState.loaded(String language)` — ينشئ كائنًا من

`LocalizationState` ويهيّئ الاعتماديات والقيم المطلوبة.

- `const factory LocalizationState.error(String message)` — ينشئ كائنًا من `LocalizationState` ويهيّئ الاعتماديات والقيم المطلوبة.

Core: di

`core/di/service_locator.dart`

- `Future<void> setupServiceLocator()` — يحدّث `setup service locator` داخل هذا الملف.

Core: helpers

`core/helpers/shared_pref_helper.dart`

- `class SharedPrefHelper` — الكلاس الذي يجمع مسؤوليات `shared pref helper`.
- `SharedPrefHelper._()` — ينشئ كائنًا من `SharedPrefHelper` ويهيّئ الاعتماديات والقيم المطلوبة.
- `static Future<SharedPreferences> _getPrefs()` — خاص — يجلب `get prefs` من مصدر البيانات المناسب داخل `SharedPrefHelper`.
- `static Future<void> setData(String key, dynamic value)` — يحدّث `set data` داخل `SharedPrefHelper`.
- `static Future<String> getString(String key)` — يجلب `get string` من مصدر البيانات المناسب داخل `SharedPrefHelper`.
- `static Future<bool> getBool(String key)` — يجلب `get bool` من مصدر البيانات المناسب داخل `SharedPrefHelper`.
- `static Future<int> getInt(String key)` — يجلب `get int` من مصدر البيانات المناسب داخل `SharedPrefHelper`.
- `static Future<double> getDouble(String key)` — يجلب `get double` من مصدر البيانات المناسب داخل `SharedPrefHelper`.
- `static Future<void> removeData(String key)` — يحذف أو يمسح `remove data` داخل `SharedPrefHelper`.
- `static Future<void> clearAllData()` — يحذف أو يمسح `clear all data` داخل `SharedPrefHelper`.
- `static Future<void> setSecuredString(String key, String value)` — يحدّث `set secured string` داخل `SharedPrefHelper`.
- `static Future<String> getSecuredString(String key)` — يجلب `get secured string` من مصدر البيانات المناسب داخل `SharedPrefHelper`.
- `static Future<void> clearAllSecuredData()` — يحذف أو يمسح `clear all secured data` داخل `SharedPrefHelper`.
- `static Future<void> removeSecuredData(String key)` — يحذف أو يمسح `remove secured data` داخل `SharedPrefHelper`.

Core: network

`core/network/api_error_handler.dart`

- `class ErrorHandler` — الكلاس الذي يجمع مسؤوليات `error handler`.
- `static String handle(dynamic error, [BuildContext? context])`](../lib/core/network/api_error_handler.dart#L7) — يعالج حدث `handle` داخل `ErrorHandler`.

`core/network/app_failure.dart`

- `enum AppFailureType` — يعرّف القيم الثابتة الخاصة بـ `app failure type`.
- `class AppFailure` — الكلاس الذي يجمع مسؤوليات `app failure`.
- `const AppFailure(this.type, {this.message, this.statusCode})` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.network()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.timeout()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.connectionTimeout()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.transferTimeout()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.unauthorized()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.forbidden()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.notFound()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.server({int? statusCode})` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.cancelled()` — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const AppFailure.unexpected([String? message])`](../lib/core/network/app_failure.dart#L31) — ينشئ كائنًا من `AppFailure` ويهيّئ الاعتماديات والقيم المطلوبة.

`core/network/auth_interceptor.dart`

- `bool _isPublicPath(String path)` — خاص — يفحص شرط `is public path` ويعيد النتيجة داخل هذا الملف.
- `class AuthInterceptor` — الكلاس الذي يجمع مسؤوليات `auth interceptor`.
- `AuthInterceptor({required AuthSessionStorage sessionStorage, required AuthRefreshService refreshService, required this.onSessionExpired})` — ينشئ كائنًا من `AuthInterceptor` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> onRequest(RequestOptions options, RequestInterceptorHandler handler)` — يعالج حدث `on request` داخل `AuthInterceptor`.
- `void onResponse(Response response, ResponseInterceptorHandler handler)` — يعالج حدث `on response` داخل `AuthInterceptor`.
- `Future<void> onError(DioException err, ErrorInterceptorHandler handler)` — يعالج حدث `on error` داخل `AuthInterceptor`.

`core/network/connectivity_service.dart`

- `class ConnectivityService` — الكلاس الذي يجمع مسؤوليات `connectivity service`.
- `ConnectivityService._()` — ينشئ كائنًا من `ConnectivityService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool get isOnline` — يفحص شرط `is online` ويعيد النتيجة داخل `ConnectivityService`.
- `Stream<bool> get connectivityStream` — ينفّذ منطق `connectivity stream` داخل `ConnectivityService`.
- `Future<void> initialize()` — ينفّذ منطق `initialize` داخل `ConnectivityService`.
- `Future<bool> checkConnectivity()` — ينفّذ منطق `check connectivity` داخل `ConnectivityService`.
- `bool _updateConnectivityStatus(List<ConnectivityResult> results)` — خاص — يحدّث `update connectivity status` داخل `ConnectivityService`.
- `void dispose()` — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.

core/network/dio_factory.dart

- `class DioFactory` — الكلاس الذي يجمع مسؤوليات `dio factory`.
- `DioFactory._()` — ينشئ كائنًا من `DioFactory` ويهيّئ الاعتماديات والقيم المطلوبة.
- `static Interceptor get _acceptLanguageInterceptor` — خاص — ينفّذ منطق `accept language interceptor` داخل `DioFactory`.
- `static Interceptor get _logger` — خاص — ينفّذ منطق `logger` داخل `DioFactory`.
- `static Future<Dio> getDio()` — يجلب `get dio` من مصدر البيانات المناسب داخل `DioFactory`.
- `static Dio createRefreshDio()` — ينشئ أو يضيف `create refresh dio` داخل `DioFactory`.
- `static Future<Dio> getDioForMedicard()` — يجلب `get dio for medicard` من مصدر البيانات المناسب داخل `DioFactory`.
- `static Future<Dio> getDioForLogin()` — يجلب `get dio for login` من مصدر البيانات المناسب داخل `DioFactory`.
- `static void setTokenIntoHeaderAfterLogin(String token)` — يحدّث `set token into header after login` داخل `DioFactory`.
- `static void clearTokenFromHeader()` — يحذف أو يمسح `clear token from header` داخل `DioFactory`.
- `static Future<void> setAcceptLanguage(String language)` — يحدّث `set accept language` داخل `DioFactory`.
- `static void _addCommonInterceptors(Dio dio)` — خاص — ينشئ أو يضيف `add common interceptors` داخل `DioFactory`.

core/network/error_mapper.dart

- `AppFailure mapDioException(DioException exception)` — يحوّل `map dio exception` بين طبقات أو صيغ البيانات داخل هذا الملف.

core/network/error_mapping_interceptor.dart

- `class ErrorMappingInterceptor` — الكلاس الذي يجمع مسؤوليات `error mapping interceptor`.
- `void onError(DioException err, ErrorInterceptorHandler handler)` — يعالج حدث `on error` داخل `ErrorMappingInterceptor`.

core/network/retry_inceptor.dart

- `class RetryInterceptor` — الكلاس الذي يجمع مسؤوليات `retry interceptor`.
- `RetryInterceptor({this.maxRetries = 3, this.baseDelay = const Duration(seconds: 2)})` — ينشئ كائنًا من `RetryInterceptor` ويهيّئ الاعتماديات والقيم المطلوبة.
- `void onError(DioException err, ErrorInterceptorHandler handler)` — يعالج حدث `on error` داخل `RetryInterceptor`.
- `bool _shouldRetry(DioException err)` — خاص — يفحص شرط `should retry` ويعيد النتيجة داخل `RetryInterceptor`.
- `bool _isIdempotent(String method)` — خاص — يفحص شرط `is idempotent` ويعيد النتيجة داخل `RetryInterceptor`.
- `Future<Dio> _getDioInstance()` — خاص — يجلب `get dio instance` من مصدر البيانات المناسب داخل `RetryInterceptor`.

Core: routing

core/routing/app_router.dart

- `class AppRouter` — الكلاس الذي يجمع مسؤوليات `app router`.
- `static Future<String> getInitialLocation()` — يجلب `get initial location` من مصدر البيانات المناسب داخل `AppRouter`.
- `static Future<String> _loginOrOnboarding()` — خاص — ينفّذ تسجيل الدخول ويتعامل مع نتيجة المصادقة داخل `AppRouter`.
- `static String normalizeLocation(String location)` — ينفّذ منطق `normalize` `location` داخل `AppRouter`.
- `static GoRouter createRouter(String initialLocation)` — ينشئ أو يضيف `create` `router` داخل `AppRouter`.
- `class _LocationObserver` — خاص — الكلاس الذي يجمع مسؤوليات `location observer`.
- `void _update()` — خاص — يحدّث `update` داخل `_LocationObserver`.
- `void didPush(Route<dynamic> route, Route<dynamic>? previousRoute)` — خاص — ينفّذ منطق `did push` داخل `_LocationObserver`.
- `void didPop(Route<dynamic> route, Route<dynamic>? previousRoute)` — خاص — ينفّذ منطق `did pop` داخل `_LocationObserver`.
- `void didReplace({Route<dynamic>? newRoute, Route<dynamic>? oldRoute})` — خاص — ينفّذ منطق `did replace` داخل `_LocationObserver`.
- `void didRemove(Route<dynamic> route, Route<dynamic>? previousRoute)` — خاص — ينفّذ منطق `did remove` داخل `_LocationObserver`.

Core: services

`core/services/app_language_manager.dart`

- `class AppLanguageManager` — الكلاس الذي يقدّم خدمة `app language manager` لباقي التطبيق.
- `Future<void> get initializationFuture` — ينفّذ منطق `initialization future` داخل `AppLanguageManager`.
- `Stream<String> get languageStream` — ينفّذ منطق `language stream` داخل `AppLanguageManager`.
- `AppLanguageManager._()` — ينشئ كائنًا من `AppLanguageManager` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> initialize()` — ينفّذ منطق `initialize` داخل `AppLanguageManager`.
- `String get currentLanguage` — ينفّذ منطق `current language` داخل `AppLanguageManager`.
- `bool get isInitialized` — يفحص شرط `is initialized` ويعيد النتيجة داخل `AppLanguageManager`.
- `Future<void> setLanguage(String language)` — يحدّث `set language` داخل `AppLanguageManager`.
- `void dispose()` — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.

`core/services/haptic_feedback_service.dart`

- `class HapticFeedbackService` — الكلاس الذي يقدّم خدمة `haptic feedback service` لباقي التطبيق.
- `const HapticFeedbackService._()` — ينشئ كائنًا من `HapticFeedbackService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `void lightImpact()` — ينفّذ منطق `light impact` داخل `HapticFeedbackService`.
- `void mediumImpact()` — ينفّذ منطق `medium impact` داخل `HapticFeedbackService`.
- `void heavyImpact()` — ينفّذ منطق `heavy impact` داخل `HapticFeedbackService`.
- `void selectionClick()` — يغيّر الاختيار الخاص بـ `selection click` داخل

HapticFeedbackService.

- `void vibrate()` — ينفّذ منطق `vibrate` داخل HapticFeedbackService.
- `void success()` — ينفّذ منطق `success` داخل HapticFeedbackService.
- `void error()` — ينفّذ منطق `error` داخل HapticFeedbackService.
- `void warning()` — ينفّذ منطق `warning` داخل HapticFeedbackService.

core/services/location_repository.dart

- `class LocationRepository` — الكلاس الذي يقدّم خدمة `location repository` لباقي التطبيق.
- `LocationResult? get cached` — ينفّذ منطق `cached` داخل LocationRepository.
- `bool get hasLocation` — يفحص شرط `has location` ويعيد النتيجة داخل LocationRepository.
- `Future<LocationResult> resolve(LocationService service, {Future<bool> Function()? showRationaleDialog})` — ينفّذ منطق `resolve` داخل LocationRepository.
- `void cache(LocationResult result)` — ينفّذ منطق `cache` داخل LocationRepository.
- `void clear()` — يحذف أو يمسح `clear` داخل LocationRepository.

core/services/location_service.dart

- `enum LocationAccessStatus` — يعرّف القيم الثابتة الخاصة بـ `location access status`.
- `class LocationResult` — الكلاس الذي يقدّم خدمة `location result` لباقي التطبيق.
- `const LocationResult({required this.status, this.latitude, this.longitude})` — ينشئ كائنًا من LocationResult ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool get hasCoordinates` — يفحص شرط `has coordinates` ويعيد النتيجة داخل LocationResult.
- `factory LocationResult.granted(double lat, double lng)` — ينشئ كائنًا من LocationResult ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory LocationResult.status(LocationAccessStatus s)` — ينشئ كائنًا من LocationResult ويهيّئ الاعتماديات والقيم المطلوبة.
- `class LocationService` — الكلاس الذي يقدّم خدمة `location service` لباقي التطبيق.
- `Future<LocationResult> resolveLocation({bool requestIfNeeded = false, Future<bool> Function()? showRationaleDialog})` — ينفّذ منطق `resolve location` داخل LocationService.
- `Future<LocationResult> tryGetLocationSilently()` — ينفّذ منطق `try get location` silently داخل LocationService.
- `Future<LocationResult> retryWithPermissionRequest()` — يعيد تحميل أو محاولة `retry with permission request` داخل LocationService.
- `Future<bool> openAppSettings()` — يعرض أو يفتح `open app settings` داخل LocationService.
- `Future<_GpsEnableResult> _requestGpsEnable()` — خاص — ينفّذ منطق `request gps enable` داخل LocationService.
- `bool _isGranted(LocationPermission p)` — خاص — يفحص شرط `is granted` ويعيد النتيجة داخل LocationService.
- `Future<LocationResult> _fetchCoordinates()` — خاص — يجلب `fetch coordinates` من مصدر البيانات المناسب داخل LocationService.
- `enum _GpsEnableResult` — خاص — يعرّف القيم الثابتة الخاصة بـ `gps enable result`.

core/services/url_launcher_service.dart

- `class UrlLauncherService` — الكلاس الذي يقدّم خدمة `url launcher service` لباقي التطبيق.
- `Future<bool> launchURL(String url)` — ينفّذ الانتقال أو الفتح الخاص بـ `launch url` داخل

.UrlLauncherService

- `Future<bool> launchEmail(String email, {String? subject, String? body})` — ينفّذ الانتقال أو الفتح الخاص بـ `launch email` داخل `UrlLauncherService`.
- `Future<bool> launchWhatsApp(String phone, {String? message})` — ينفّذ الانتقال أو الفتح الخاص بـ `launch whats app` داخل `UrlLauncherService`.
- `Future<bool> makePhoneCall(String phone)` — ينفّذ منطق `make phone call` داخل `UrlLauncherService`.
- `Future<bool> launchSMS(String phone, {String? message})` — ينفّذ الانتقال أو الفتح الخاص بـ `launch sms` داخل `UrlLauncherService`.
- `Future<bool> launchFacebook(String url)` — ينفّذ الانتقال أو الفتح الخاص بـ `launch facebook` داخل `UrlLauncherService`.

Core: theming

core/theming/app_colors.dart

- `class AppColors` — الكلاس الذي يجمع مسؤوليات `app colors`.

core/theming/app_snack_bar.dart

- `enum SnackBarType` — يعرّف القيم الثابتة الخاصة بـ `snack bar type`.
- `void showAppSnackBar(BuildContext context, String message, {bool isError = false, Duration duration = const Duration(seconds: 3), SnackBarType? type, VoidCallback? onAction, String? actionLabel})` — يعرض أو يفتح `show app snack bar` داخل هذا الملف.
- `class _SnackBarConfig` — خاص — الكلاس الذي يجمع مسؤوليات `snack bar config`.
- `const _SnackBarConfig({required this.gradient, required this.icon, required this.title, required this.shadowColor})` — خاص — ينشئ كائنًا من `SnackBarConfig` ويهيّئ الاعتماديات والقيم المطلوبة.
- `_SnackBarConfig _getSnackBarConfig(SnackBarType type)` — خاص — يجلب `get snack bar config` من مصدر البيانات المناسب داخل هذا الملف.
- `void showSimpleToast(BuildContext context, String message)` — يعرض أو يفتح `show simple toast` داخل هذا الملف.

core/theming/app_text_styles.dart

- `class AppFontSizes` — الكلاس الذي يجمع مسؤوليات `app font sizes`.
- `class AppTextStyles` — الكلاس الذي يجمع مسؤوليات `app text styles`.
- `static TextStyle _baseStyle(BuildContext context, {required double fontSize, required FontWeight fontWeight, required Color color})` — خاص — ينفّذ منطق `base style` داخل `AppTextStyles`.
- `static TextStyle custom(BuildContext context, {required double fontSize, FontWeight fontWeight = FontWeight.w400, Color color = AppColors.blackClr})` — ينفّذ منطق `custom` داخل `AppTextStyles`.
- `static TextStyle of({double? fontSize, FontWeight? fontWeight, Color? color, double? height, double? letterSpacing, TextDecoration? decoration, FontStyle? fontStyle, Color? decorationColor, double? decorationThickness})` — ينفّذ منطق `of` داخل `AppTextStyles`.
- `static TextStyle font20BlackSemiBold(BuildContext context)` — ينفّذ منطق `font20 black semi bold` داخل `AppTextStyles`.
- `static TextStyle font20WhiteSemiBold(BuildContext context)` — ينفّذ منطق `font20 white semi bold` داخل `AppTextStyles`.

- `static TextStyle font18BlackSemiBold(BuildContext context)` — ينفّذ منطق `font18 black semi bold` داخل `AppTextStyles`.
- `static TextStyle font18BlackMedium(BuildContext context)` — ينفّذ منطق `font18 black medium` داخل `AppTextStyles`.
- `static TextStyle font16BlackRegular(BuildContext context)` — ينفّذ منطق `font16 black regular` داخل `AppTextStyles`.
- `static TextStyle font16BlackMedium(BuildContext context)` — ينفّذ منطق `font16 black medium` داخل `AppTextStyles`.
- `static TextStyle font16GreyRegular(BuildContext context)` — ينفّذ منطق `font16 grey regular` داخل `AppTextStyles`.
- `static TextStyle font16WhiteRegular(BuildContext context)` — ينفّذ منطق `font16 white regular` داخل `AppTextStyles`.
- `static TextStyle font16WhiteMedium(BuildContext context)` — ينفّذ منطق `font16 white medium` داخل `AppTextStyles`.
- `static TextStyle font16BlueMedium(BuildContext context)` — ينفّذ منطق `font16 blue medium` داخل `AppTextStyles`.
- `static TextStyle font16GreenMedium(BuildContext context)` — ينفّذ منطق `font16 green medium` داخل `AppTextStyles`.
- `static TextStyle font14BlackRegular(BuildContext context)` — ينفّذ منطق `font14 black regular` داخل `AppTextStyles`.
- `static TextStyle font14BlackMedium(BuildContext context)` — ينفّذ منطق `font14 black medium` داخل `AppTextStyles`.
- `static TextStyle font14GreyRegular(BuildContext context)` — ينفّذ منطق `font14 grey regular` داخل `AppTextStyles`.
- `static TextStyle font14WhiteRegular(BuildContext context)` — ينفّذ منطق `font14 white regular` داخل `AppTextStyles`.
- `static TextStyle font14WhiteMedium(BuildContext context)` — ينفّذ منطق `font14 white medium` داخل `AppTextStyles`.
- `static TextStyle font14PrimaryMedium(BuildContext context)` — ينفّذ منطق `font14 primary medium` داخل `AppTextStyles`.
- `static TextStyle font14BlueRegular(BuildContext context)` — ينفّذ منطق `font14 blue regular` داخل `AppTextStyles`.
- `static TextStyle font14BlueMedium(BuildContext context)` — ينفّذ منطق `font14 blue medium` داخل `AppTextStyles`.
- `static TextStyle font14GreenRegular(BuildContext context)` — ينفّذ منطق `font14 green regular` داخل `AppTextStyles`.
- `static TextStyle font12BlackRegular(BuildContext context)` — ينفّذ منطق `font12 black regular` داخل `AppTextStyles`.
- `static TextStyle font12BlackMedium(BuildContext context)` — ينفّذ منطق `font12 black medium` داخل `AppTextStyles`.
- `static TextStyle font12GreyRegular(BuildContext context)` — ينفّذ منطق `font12 grey regular` داخل `AppTextStyles`.
- `static TextStyle font12WhiteRegular(BuildContext context)` — ينفّذ منطق `font12 white regular` داخل `AppTextStyles`.
- `static TextStyle font12BlueRegular(BuildContext context)` — ينفّذ منطق `font12 blue regular` داخل `AppTextStyles`.
- `static TextStyle font12BlueMedium(BuildContext context)` — ينفّذ منطق `font12`

`blue medium` داخل `AppTextStyles`.

- `static TextStyle font10BlackRegular(BuildContext context)` — ينفّذ منطق `font10 black regular` داخل `AppTextStyles`.
- `static TextStyle font10BlackMedium(BuildContext context)` — ينفّذ منطق `font10 black medium` داخل `AppTextStyles`.
- `static TextStyle font10GreyRegular(BuildContext context)` — ينفّذ منطق `font10 grey regular` داخل `AppTextStyles`.
- `static TextStyle font10WhiteRegular(BuildContext context)` — ينفّذ منطق `font10 white regular` داخل `AppTextStyles`.
- `static TextStyle font8WhiteRegular(BuildContext context)` — ينفّذ منطق `font8 white regular` داخل `AppTextStyles`.
- `static TextStyle font8GreyRegular(BuildContext context)` — ينفّذ منطق `font8 grey regular` داخل `AppTextStyles`.
- `extension TextStyleExtensions` — يضيف وظائف مساعدة إلى نوع موجود دون تعديل تعريفه.
- `TextStyle get bold` — ينفّذ منطق `bold` داخل `TextStyleExtensions`.
- `TextStyle get semiBold` — ينفّذ منطق `semi bold` داخل `TextStyleExtensions`.
- `TextStyle get medium` — ينفّذ منطق `medium` داخل `TextStyleExtensions`.
- `TextStyle withColor(Color color)` — ينفّذ منطق `with color` داخل `TextStyleExtensions`.
- `TextStyle withSize(double size)` — ينفّذ منطق `with size` داخل `TextStyleExtensions`.
- `TextStyle withValues({double? opacity})` — ينفّذ منطق `with values` داخل `TextStyleExtensions`.

core/theming/app_toast.dart

- `enum ToastType` — يعرّف القيم الثابتة الخاصة بـ `toast type`.
- `Color _backgroundColorFor(ToastType type)` — خاص — ينفّذ منطق `background color for` داخل هذا الملف.
- `IconData _iconFor(ToastType type)` — خاص — ينفّذ منطق `icon for` داخل هذا الملف.
- `void showToast(String message, {ToastType type = ToastType.info, bool isError = false})` — يعرض أو يفتح `show toast` داخل هذا الملف.
- `void showSuccessToast(String message)` — يعرض أو يفتح `show success toast` داخل هذا الملف.
- `void showErrorToast(String message)` — يعرض أو يفتح `show error toast` داخل هذا الملف.
- `void showInfoToast(String message)` — يعرض أو يفتح `show info toast` داخل هذا الملف.
- `void showWarningToast(String message)` — يعرض أو يفتح `show warning toast` داخل هذا الملف.
- `void showLoginRequiredMessage(BuildContext context)` — يعرض أو يفتح `show login required message` داخل هذا الملف.
- `class _LoginRequiredToast` — خاص — الكلاس الذي يجمع مسؤوليات `login required toast`.
- `const _LoginRequiredToast({required this.onDismiss, required this.onLoginTap})` — خاص — ينشئ كائنًا من `LoginRequiredToast` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_LoginRequiredToast> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _LoginRequiredToastState` — خاص — الكلاس الذي يجمع مسؤوليات `login required`

toast state.

- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _dismiss()` — خاص — ينفّذ منطق `dismiss` داخل `_LoginRequiredToastState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ToastWidget` — خاص — الكلاس الذي يجمع مسؤوليات `toast widget`.
- `const _ToastWidget({required this.message, required this.type, required this.backgroundColor, required this.icon, required this.onDismiss})` — خاص — ينشئ كائنًا من `ToastWidget` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_ToastWidget> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ToastWidgetState` — خاص — الكلاس الذي يجمع مسؤوليات `toast widget state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _dismiss()` — خاص — ينفّذ منطق `dismiss` داخل `_ToastWidgetState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

core/theming/font_manager.dart

- `class FontManager` — الكلاس الذي يجمع مسؤوليات `font manager`.
- `static String getFontFamily(BuildContext context)` — يجلب `get font family` من مصدر البيانات المناسب داخل `FontManager`.

Core: utils

core/utils/api_error_parser.dart

- `String parseApiError(dynamic error)` — يحلّل `parse api error` ويحوّله لصيغة قابلة للاستخدام داخل هذا الملف.
- `String _extractMessages(dynamic value)` — خاص — ينفّذ منطق `extract messages` داخل هذا الملف.
- `String _pickRelevantFieldError(dynamic errors)` — خاص — ينفّذ منطق `pick relevant field error` داخل هذا الملف.
- `String friendlyApiMessage(String message)` — ينفّذ منطق `friendly api message` داخل هذا الملف.
- `bool _isGenericMessage(String message)` — خاص — يفحص شرط `is generic message` ويعيد النتيجة داخل هذا الملف.
- `String? _messageForStatusCode(int? statusCode)` — خاص — ينفّذ منطق `message for status code` داخل هذا الملف.
- `String? _messageForAppFailure(AppFailure failure)` — خاص — ينفّذ منطق `message for app failure` داخل هذا الملف.

core/utils/app_button.dart

- `class AppButton` — الكلاس الذي يجمع مسؤوليات `app button`.
- `const AppButton({super.key, required this.text, required this.onPressed, this.width = 343, this.height = 48, this.isEnabled = true, this.isLoading = false, this.backgroundColor, this.textColor, this.useGradient = false, this.gradient, this.icon, this.iconSize})` — ينشئ كائنًا من `AppButton` ويهيّئ الاعتماديات والقيم المطلوبة.

- `State<AppButton> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AppButtonState` — خاص — الكلاس الذي يجمع مسؤوليات `app button state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _handleTapDown(TapDownDetails details)` — خاص — يعالج حدث `handle tap down` داخل `_AppButtonState`.
- `void _handleTapUp(TapUpDetails details)` — خاص — يعالج حدث `handle tap up` داخل `_AppButtonState`.
- `void _handleTapCancel()` — خاص — يعالج حدث `handle tap cancel` داخل `_AppButtonState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class AppOutlineButton` — الكلاس الذي يجمع مسؤوليات `app outline button`.
- `const AppOutlineButton({super.key, required this.text, required this.onPressed, this.width = 343, this.height = 48, this.isEnabled = true, this.isLoading = false, this.borderColor, this.textColor, this.icon})` — ينشئ كائنًا من `AppOutlineButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<AppOutlineButton> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AppOutlineButtonState` — خاص — الكلاس الذي يجمع مسؤوليات `app outline button state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _handleTapDown(TapDownDetails details)` — خاص — يعالج حدث `handle tap down` داخل `_AppOutlineButtonState`.
- `void _handleTapUp(TapUpDetails details)` — خاص — يعالج حدث `handle tap up` داخل `_AppOutlineButtonState`.
- `void _handleTapCancel()` — خاص — يعالج حدث `handle tap cancel` داخل `_AppOutlineButtonState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

core/utils/app_date_formatter.dart

- `class AppDateFormatter` — الكلاس الذي يجمع مسؤوليات `app date formatter`.
- `static DateTime? tryParse(String? value)` — ينفّذ منطق `try parse` داخل `AppDateFormatter`.
- `static String _safe(String? value, String Function(DateTime) builder)` — خاص — ينفّذ منطق `safe` داخل `AppDateFormatter`.
- `static String _localeCode(String? languageCode)` — خاص — ينفّذ منطق `locale code` داخل `AppDateFormatter`.
- `static String formatDateTime(String? value, String? languageCode)` — ينسّق `format date time` للعرض أو الإرسال داخل `AppDateFormatter`.
- `static String formatDate(String? value, String? languageCode)` — ينسّق `format date` للعرض أو الإرسال داخل `AppDateFormatter`.
- `static String formatTime(String? value, String? languageCode)` — ينسّق `format time` للعرض أو الإرسال داخل `AppDateFormatter`.
- `static String formatDateTimeShort(String? value, String? languageCode)` — ينسّق `format date time short` للعرض أو الإرسال داخل `AppDateFormatter`.

core/utils/app_error_handler.dart

- `class AppErrorHandler` — الكلاس الذي يجمع مسؤوليات app error handler.
- `static String messageFromFailure(AppFailure failure)` — ينفّذ منطق message from failure داخل AppErrorHandler.
- `static String getUserFriendlyMessage(String error)` — يجلب get user friendly message من مصدر البيانات المناسب داخل AppErrorHandler.
- `static bool _isTechnicalMessage(String message)` — خاص — يفحص شرط is technical message ويعيد النتيجة داخل AppErrorHandler.
- `static String _resolveMessage(Object error)` — خاص — ينفّذ منطق resolve message داخل AppErrorHandler.
- `static void showErrorSnackBar(BuildContext context, Object error, {VoidCallback? onRetry, Duration duration = const Duration(seconds: 4)})` — يعرض أو يفتح show error snack bar داخل AppErrorHandler.
- `static void showErrorDialog(BuildContext context, Object error, {VoidCallback? onRetry})` — يعرض أو يفتح show error dialog داخل AppErrorHandler.
- `static void logError(Object error, {StackTrace? stackTrace})` — ينفّذ منطق log error داخل AppErrorHandler.
- `static void handleApiResult(BuildContext context, ApiResult<T> result, {VoidCallback? onRetry, bool showDialog = false})` — يعالج حدث handle api result داخل AppErrorHandler.
- `static String? getErrorMessageFromResult(ApiResult<T> result)` — يجلب get error message from result من مصدر البيانات المناسب داخل AppErrorHandler.
- `static bool isFailure(ApiResult<T> result)` — يفحص شرط is failure ويعيد النتيجة داخل AppErrorHandler.

core/utils/app_text_field.dart

- `class AppTextField` — الكلاس الذي يجمع مسؤوليات app text field.
- `const AppTextField({super.key, this.controller, this.hintText, this.isPassword = false, this.prefixImagePath, this.prefixIcon, this.errorText, this.keyboardType = TextInputType.text, this.onChanged, this.enabled = true, this.validator, this.label, this.showSuccessState = false, this.maxLength, this.inputFormatters})` — ينشئ كائنًا من AppTextField ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<AppTextField> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AppTextFieldState` — خاص — الكلاس الذي يجمع مسؤوليات app text field state.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onFocusChange()` — خاص — يعالج حدث on focus change داخل _AppTextFieldState.
- `OutlineInputBorder _buildBorder(Color color, double width)` — خاص — ينفّذ منطق build border داخل _AppTextFieldState.
- `Color _getBorderColor(bool hasError)` — خاص — يجلب get border color من مصدر البيانات المناسب داخل _AppTextFieldState.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget? _buildPrefixIcon(bool hasError)` — خاص — ينفّذ منطق build prefix icon داخل _AppTextFieldState.
- `Widget? _buildSuffixIcon(bool hasError)` — خاص — ينفّذ منطق build suffix icon

داخل `_AppTextFieldState`.

core/utils/custom_showcase.dart

- `class CustomShowcase` — الكلاس الذي يجمع مسؤوليات `custom showcase`.
- `const CustomShowcase({super.key, required this.targetKey, required this.child})` — ينشئ كائنًا من `CustomShowcase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class CustomShowcaseOverlay` — الكلاس الذي يجمع مسؤوليات `custom showcase overlay`.
- `const CustomShowcaseOverlay({super.key, required this.targetKeys, required this.descriptions, required this.currentIndex, required this.child, this.onNext, this.onDismiss})` — ينشئ كائنًا من `CustomShowcaseOverlay` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<CustomShowcaseOverlay> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _CustomShowcaseOverlayState` — خاص — الكلاس الذي يجمع مسؤوليات `custom showcase overlay state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void didUpdateWidget(CustomShowcaseOverlay oldWidget)` — خاص — يتعامل مع تغيّر إعدادات الـ widget مع الاحتفاظ بالحالة.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ShowcasePainter` — خاص — الكلاس الذي يجمع مسؤوليات `showcase painter`.
- `_ShowcasePainter({required this.targetPosition, required this.targetSize, required this.screenSize})` — خاص — ينشئ كائنًا من `ShowcasePainter` ويهيّئ الاعتماديات والقيم المطلوبة.
- `void paint(Canvas canvas, Size size)` — خاص — ينفّذ منطق `paint` داخل `_ShowcasePainter`.
- `bool shouldRepaint(_ShowcasePainter oldDelegate)` — خاص — يفحص شرط `should repaint` ويعيد النتيجة داخل `_ShowcasePainter`.

core/utils/egyptian_phone_field.dart

- `class EgyptianPhoneField` — الكلاس الذي يجمع مسؤوليات `egyptian phone field`.
- `const EgyptianPhoneField({super.key, required this.controller, this.validator, this.hintText, required this.phoneRequiredKey, required this.phoneInvalidKey, this.onChanged, this.maxLength})` — ينشئ كائنًا من `EgyptianPhoneField` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<EgyptianPhoneField> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _EgyptianPhoneFieldState` — خاص — الكلاس الذي يجمع مسؤوليات `egyptian phone field state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `String? _defaultValidator(String? value)` — خاص — ينفّذ منطق `default validator` داخل `_EgyptianPhoneFieldState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

core/utils/error_state_widget.dart

- `class ErrorStateWidget` — الكلاس الذي يجمع مسؤوليات `error state widget`.
- `const ErrorStateWidget({super.key, required this.error, this.title, this.onRetry, this.icon, this.showDetails = false})` — ينشئ كائنًا من `ErrorStateWidget` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _getErrorMessage(BuildContext context)` — خاص — يجلب `get error` `message` من مصدر البيانات المناسب داخل `ErrorStateWidget`.
- `class CompactErrorWidget` — الكلاس الذي يجمع مسؤوليات `compact error widget`.
- `const CompactErrorWidget({super.key, required this.error, this.onRetry})` — ينشئ كائنًا من `CompactErrorWidget` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _getErrorMessage(BuildContext context)` — خاص — يجلب `get error` `message` من مصدر البيانات المناسب داخل `CompactErrorWidget`.

core/utils/image_url_resolver.dart

- `String? resolveImageUrl(String? url)` — ينفّذ منطق `resolve image url` داخل هذا الملف.

core/utils/language_helper.dart

- `class LanguageHelper` — الكلاس الذي يجمع مسؤوليات `language helper`.
- `static String getLanguageCode(BuildContext context)` — يجلب `get language` `code` من مصدر البيانات المناسب داخل `LanguageHelper`.
- `static String convertArabicToEnglishNumerals(String input)` — ينفّذ منطق `convert arabic to english numerals` داخل `LanguageHelper`.

core/utils/platform_type.dart

- `enum PlatformType` — يعرّف القيم الثابتة الخاصة بـ `platform type`.
- `const PlatformType(this.value)` — ينشئ كائنًا من `PlatformType` ويهيّئ الاعتماديات والقيم المطلوبة.
- `static PlatformType get current` — ينفّذ منطق `current` داخل `PlatformType`.

core/utils/relation_helper.dart

- `class RelationHelper` — الكلاس الذي يجمع مسؤوليات `relation helper`.
- `static String getLocalizedRelation(int? relation)` — يجلب `get localized` `relation` من مصدر البيانات المناسب داخل `RelationHelper`.
- `static List<Map<String, dynamic>> getRelationsList()` — يجلب `get relations` `list` من مصدر البيانات المناسب داخل `RelationHelper`.

core/utils/shimmer_card.dart

- `class SmoothShimmer` — الكلاس الذي يجمع مسؤوليات `smooth shimmer`.
- `const SmoothShimmer({super.key, required this.child, this.baseColor, this.highlightColor, this.period = const Duration(milliseconds: 1500), this.direction = ShimmerDirection.ltr})` — ينشئ كائنًا من `SmoothShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ShimmerCard` — الكلاس الذي يجمع مسؤوليات `shimmer card`.
- `const ShimmerCard({super.key, this.width, required this.height, this.borderRadius, this.margin})` — ينشئ كائنًا من `ShimmerCard` ويهيّئ الاعتماديات والقيم

المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ShimmerListItem` — الكلاس الذي يجمع مسؤوليات `shimmer list item`.
- `const ShimmerListItem({super.key, this.showAvatar = true, this.lines = 2, this.avatarSize, this.padding})` — ينشئ كائنًا من `ShimmerListItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ShimmerApprovalCard` — الكلاس الذي يجمع مسؤوليات `shimmer approval card`.
- `const ShimmerApprovalCard({super.key})` — ينشئ كائنًا من `ShimmerApprovalCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ShimmerGridItem` — الكلاس الذي يجمع مسؤوليات `shimmer grid item`.
- `const ShimmerGridItem({super.key, this.width, this.height})` — ينشئ كائنًا من `ShimmerGridItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class PulseShimmer` — الكلاس الذي يجمع مسؤوليات `pulse shimmer`.
- `const PulseShimmer({super.key, required this.child, this.duration = const Duration(milliseconds: 1200)})` — ينشئ كائنًا من `PulseShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<PulseShimmer> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _PulseShimmerState` — خاص — الكلاس الذي يجمع مسؤوليات `pulse shimmer state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Core: widgets

core/widgets/app_blue_header.dart

- `class AppBlueHeader` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const AppBlueHeader({super.key, this.title, this.subtitle, this.showBack = true, this.onBack, this.actions, this.bottomChild, this.bottomRadius = 24, this.color = const Color(0xFF3872B1), this.padding, this.contentHeight})` — ينشئ كائنًا من `AppBlueHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

core/widgets/app_simple_header.dart

- `class AppSimpleHeader` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const AppSimpleHeader({super.key, this.title, this.subtitle, this.showBack = true, this.onBack, this.actions, this.bottomChild})` — ينشئ كائنًا من `AppSimpleHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `void _handleBack(BuildContext context)` — خاص — يعالج حدث `handle back` داخل `AppSimpleHeader`.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

core/widgets/auth_shared_widgets.dart

- `class AuthFieldLabel` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const AuthFieldLabel(this.text, {super.key, this.isRequired = false})` — ينشئ كائنًا من `AuthFieldLabel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class AuthGenderOption` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const AuthGenderOption({super.key, required this.label, required this.selected, required this.onTap})` — ينشئ كائنًا من `AuthGenderOption` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `InputDecoration authInputDecoration({required String hintText, Widget? suffixIcon})` — ينفّذ منطق `auth input decoration` داخل هذا الملف.
- `class AuthBlueHeader` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const AuthBlueHeader({super.key, this.showBackButton = false, this.onBack, this.logoAsset = AppAssets.mediLogo, required this.title})` — ينشئ كائنًا من `AuthBlueHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class AuthPrimaryButton` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const AuthPrimaryButton({super.key, required this.label, required this.isLoading, required this.onPressed})` — ينشئ كائنًا من `AuthPrimaryButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Module: data

data/auth_login_request_model.dart

- `class AuthLoginRequestModel` — الكلاس الذي يمثل بيانات `auth login request model` وينقلها بين الطبقات.
- `const AuthLoginRequestModel({this.cardNumber, this.phoneNumber, required this.password, this.platformType = 1})` — ينشئ كائنًا من `AuthLoginRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AuthLoginRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AuthLoginRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

data/auth_register_request_model.dart

- `class AuthRegisterRequestModel` — الكلاس الذي يمثل بيانات `auth register request model` وينقلها بين الطبقات.
- `const AuthRegisterRequestModel({required this.firstName, required this.lastName, required this.phoneNumber, required this.isMale, required this.birthDate, required this.password, required this.confirmPassword, this.nationalId, this.passportNumber})` — ينشئ كائنًا من

`AuthRegisterRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory AuthRegisterRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AuthRegisterRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/auth_response_model.dart`

- `class AuthResponseModel` — الكلاس الذي يمثل بيانات `auth response model` وينقلها بين الطبقات.
- `const AuthResponseModel({this.httpStatusCode, this.errors, required this.succeeded, this.message, this.data})` — ينشئ كائنًا من `AuthResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AuthResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AuthResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `Object? _readSucceeded(Map<dynamic, dynamic> json, String key)` — خاص — ينفّذ منطق `read succeeded` داخل هذا الملف.
- `class AuthTokenDataModel` — الكلاس الذي يمثل بيانات `auth token data model` وينقلها بين الطبقات.
- `const AuthTokenDataModel({this.token, this.expiresAt, this.refreshToken, this.refreshTokenExpiresAt, this.success, this.message})` — ينشئ كائنًا من `AuthTokenDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AuthTokenDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AuthTokenDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `static DateTime? _parseDateTimeOrNull(String? value)` — خاص — يحلّل `parse date time or null` ويحوّله لصيغة قابلة للاستخدام داخل `AuthTokenDataModel`.
- `static String? _dateTimeToStringOrNull(DateTime? dt)` — خاص — ينفّذ منطق `date time to string or null` داخل `AuthTokenDataModel`.

`data/card_home_info_response_model.dart`

- `class CardHomeInfoResponseModel` — الكلاس الذي يمثل بيانات `card home info response model` وينقلها بين الطبقات.
- `const CardHomeInfoResponseModel({required this.success, required this.timestamp, required this.message, this.data})` — ينشئ كائنًا من `CardHomeInfoResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardHomeInfoResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardHomeInfoResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CardHomeInfoDataModel` — الكلاس الذي يمثل بيانات `card home info data model` وينقلها بين الطبقات.
- `const CardHomeInfoDataModel({required this.cardId, required this.firstName, required this.lastName, required this.expireDate, this.memberPhoto})` — ينشئ كائنًا من `CardHomeInfoDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardHomeInfoDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardHomeInfoDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/card_login_request_model.dart`

- `class CardLoginRequestModel` — الكلاس الذي يمثل بيانات `card login request model` وينقلها بين الطبقات.
- `const CardLoginRequestModel({required this.cardNo, required`

`this.password})` — ينشئ كائنًا من `CardLoginRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory CardLoginRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardLoginRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/card_login_response_model.dart`

- `class CardLoginResponseModel` — الكلاس الذي يمثل بيانات `card login response model` وينقلها بين الطبقات.
- `const CardLoginResponseModel({required this.success, required this.timestamp, required this.message, this.data})` — ينشئ كائنًا من `CardLoginResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardLoginResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardLoginResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CardLoginDataModel` — الكلاس الذي يمثل بيانات `card login data model` وينقلها بين الطبقات.
- `const CardLoginDataModel({required this.cardId, required this.firstName, required this.lasName})` — ينشئ كائنًا من `CardLoginDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardLoginDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardLoginDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/card_personal_info_response_model.dart`

- `class CardPersonalInfoResponseModel` — الكلاس الذي يمثل بيانات `card personal info response model` وينقلها بين الطبقات.
- `const CardPersonalInfoResponseModel({required this.success, required this.timestamp, required this.message, this.data})` — ينشئ كائنًا من `CardPersonalInfoResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardPersonalInfoResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardPersonalInfoResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CardPersonalInfoDataModel` — الكلاس الذي يمثل بيانات `card personal info data model` وينقلها بين الطبقات.
- `const CardPersonalInfoDataModel({required this.firstName, required this.lastName, this.image, required this.cardId, required this.mobile, required this.birthdate, required this.isMale, this.passport, this.email, this.nationalId, required this.activatedDate, required this.expireDate})` — ينشئ كائنًا من `CardPersonalInfoDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardPersonalInfoDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardPersonalInfoDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/card_support_contact_response_model.dart`

- `class CardSupportContactResponseModel` — الكلاس الذي يمثل بيانات `card support contact response model` وينقلها بين الطبقات.
- `const CardSupportContactResponseModel({required this.success, required this.timestamp, required this.message, required this.data})` — ينشئ كائنًا من `CardSupportContactResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory CardSupportContactResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardSupportContactResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CardSupportContactData` — الكلاس الذي يمثل بيانات `card support contact data` وينقلها بين الطبقات.
- `const CardSupportContactData({required this.email, required this.hotLine, required this.whatsApp, required this.website, required this.linkedIn, required this.facebook, required this.instagram})` — ينشئ كائنًا من `CardSupportContactData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardSupportContactData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardSupportContactData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String valueOf(String key)` — ينفّذ منطق `value of` داخل `CardSupportContactData`.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/card_update_profile_request_model.dart`

- `class CardUpdateProfileRequestModel` — الكلاس الذي يمثل بيانات `card update profile request model` وينقلها بين الطبقات.
- `const CardUpdateProfileRequestModel({required this.cardNo, this.firstName, this.lastName, this.phoneNumber, this.email, this.nationalId, this.passportNumber, this.birthdate, this.isMale, this.profileImage})` — ينشئ كائنًا من `CardUpdateProfileRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardUpdateProfileRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardUpdateProfileRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`data/card_update_profile_response_model.dart`

- `class CardUpdateProfileResponseModel` — الكلاس الذي يمثل بيانات `card update profile response model` وينقلها بين الطبقات.
- `const CardUpdateProfileResponseModel({required this.success, required this.timestamp, required this.message, this.data})` — ينشئ كائنًا من `CardUpdateProfileResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardUpdateProfileResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardUpdateProfileResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CardUpdateProfileDataModel` — الكلاس الذي يمثل بيانات `card update profile data model` وينقلها بين الطبقات.
- `const CardUpdateProfileDataModel({required this.cardId, required this.firstName, required this.lastName, required this.mobile, this.email, this.nationalId, this.passportNumber, required this.birthdate, required this.isMale, this.profileImage})` — ينشئ كائنًا من `CardUpdateProfileDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardUpdateProfileDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardUpdateProfileDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

Module: domain

`domain/medicard_repository.dart`

- `class MedicardRepository` — الكلاس الذي يجمع مسؤوليات `medicard repository`.

- `Future<ApiResult<AuthResponseModel>> register(AuthRegisterRequestModel request)` — يسجّل `register` لدى النظام أو الخدمة داخل `MedicardRepository`.
- `Future<ApiResult<AuthResponseModel>> login(AuthLoginRequestModel request)` — ينفّذ تسجيل الدخول ويتعامل مع نتيجة المصادقة داخل `MedicardRepository`.
- `Future<ApiResult<OtpResponseModel>> sendOtp(String phoneNumber)` — يرسل `send otp` إلى الطبقة أو الخدمة المطلوبة داخل `MedicardRepository`.
- `Future<ApiResult<OtpResponseModel>> verifyOtp({required String phoneNumber, required String otp})` — ينفّذ منطق `verify otp` داخل `MedicardRepository`.
- `Future<ApiResult<OtpResponseModel>> resetPassword({required String phoneNumber, required String password, required String confirmPassword})` — ينفّذ منطق `reset password` داخل `MedicardRepository`.
- `Future<ApiResult<CardHomeInfoResponseModel>> getHomeInfo(String cardNo, String lang)` — يجلب `get home info` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<CardPersonalInfoResponseModel>> getPersonalInfo(String cardNo, String lang)` — يجلب `get personal info` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<CardSupportContactResponseModel>> getSupportContacts(String lang)` — يجلب `get support contacts` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<CardUpdateProfileResponseModel>> updateProfile(CardUpdateProfileRequestModel request, String lang)` — يحدّث `update profile` داخل `MedicardRepository`.
- `Future<ApiResult<HomeResponseModel>> getHomeData({double? latitude, double? longitude})` — يجلب `get home data` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<ProfileResponseModel>> getProfile()` — يجلب `get profile` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<ProfileSummaryResponseModel>> getProfileSummary()` — يجلب `get profile summary` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<ProfileResponseModel>> deleteProfileImage()` — يحذف أو يمسح `delete profile image` داخل `MedicardRepository`.
- `Future<ApiResult<MemberCardActivationResponseModel>> activateMemberCard({required String cardNumber})` — ينفّذ منطق `activate member card` داخل `MedicardRepository`.
- `Future<ApiResult<CardDataResponseModel>> getCardData()` — يجلب `get card data` من مصدر البيانات المناسب داخل `MedicardRepository`.
- `Future<ApiResult<ProfileResponseModel>> updateProfileV2({required String firstName, required String lastName, required String phoneNumber, required String nationalId, String? passportNumber, required bool isMale, required String birthDate, String? imagePath, bool removeImage = false, String? currentImageUrl})` — يحدّث `update profile v2` داخل `MedicardRepository`.
- `Future<ApiResult<ChangePasswordResponseModel>> changePassword({required String currentPassword, required String newPassword, required String confirmNewPassword})` — يحدّث `change password` داخل `MedicardRepository`.
- `Future<ApiResult<ChangeLanguageResponseModel>> changeLanguage({required String language})` — يحدّث `change language` داخل `MedicardRepository`.
- `Future<ApiResult<GuestHomeResponseModel>> getGuestHomeData({double? latitude, double? longitude})` — يجلب `get guest home data` من مصدر البيانات المناسب

داخل `MedicardRepository`.

`domain/usecases/get_home_info_usecase.dart`

- `class GetHomeInfoUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get home info use case`.
- `GetHomeInfoUseCase(this._repository)` — ينشئ كائنًا من `GetHomeInfoUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<CardHomeInfoResponseModel>> call(String cardNo, String lang)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`domain/usecases/get_personal_info_usecase.dart`

- `class GetPersonalInfoUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get personal info use case`.
- `GetPersonalInfoUseCase(this._repository)` — ينشئ كائنًا من `GetPersonalInfoUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<CardPersonalInfoResponseModel>> call(String cardNo, String lang)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`domain/usecases/get_support_contacts_usecase.dart`

- `class GetSupportContactsUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get support contacts use case`.
- `GetSupportContactsUseCase(this._repository)` — ينشئ كائنًا من `GetSupportContactsUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<CardSupportContactResponseModel>> call(String lang)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`domain/usecases/login_card_usecase.dart`

- `class LoginCardUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `login card use case`.
- `LoginCardUseCase(this._repository)` — ينشئ كائنًا من `LoginCardUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<AuthResponseModel>> call(AuthLoginRequestModel request)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`domain/usecases/logout_usecase.dart`

- `class LogoutUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `logout use case`.
- `const LogoutUseCase()` — ينشئ كائنًا من `LogoutUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`domain/usecases/register_usecase.dart`

- `class RegisterUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `register use case`.
- `RegisterUseCase(this._repository)` — ينشئ كائنًا من `RegisterUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<AuthResponseModel>> call(AuthRegisterRequestModel request)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`domain/usecases/update_profile_usecase.dart`

- `class UpdateProfileUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `update profile use case`.
- `UpdateProfileUseCase(this._repository)` — ينشئ كائنًا من `UpdateProfileUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<CardUpdateProfileResponseModel>> call(CardUpdateProfileRequestModel request, String lang)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

Feature: activate_card

features/activate_card/data/models/activate_card_request_model.dart

- `class ActivateCardRequestModel` — الكلاس الذي يمثل بيانات `activate card request` وينقلها بين الطبقات `model`.
- `const ActivateCardRequestModel({required this.firstName, required this.lastName, required this.cardNumber, required this.isMale, required this.phoneNumber, required this.birthDate, required this.password, required this.confirmPassword, this.nationalId, this.passportNumber})` — ينشئ كائنًا من `ActivateCardRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activate_card/data/models/activate_card_response_model.dart

- `class ActivateCardResponseModel` — الكلاس الذي يمثل بيانات `activate card response` وينقلها بين الطبقات `model`.
- `const ActivateCardResponseModel({required this.httpStatusCode, this.errors, required this.succeeded, required this.message, this.data})` — ينشئ كائنًا من `ActivateCardResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ActivateCardResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ActivateCardResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ActivateCardData` — الكلاس الذي يمثل بيانات `activate card data` وينقلها بين الطبقات.
- `const ActivateCardData({required this.token, required this.expiresAt})` — ينشئ كائنًا من `ActivateCardData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ActivateCardData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ActivateCardData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activate_card/data/repository/activate_card_repository_impl.dart

- `class ActivateCardRepositoryImpl` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات `activate card repository impl`.
- `Future<Either<String, ActivateCardResponseModel>> activateCard(ActivateCardRequestModel request)` — ينفّذ منطق `activate card` داخل `ActivateCardRepositoryImpl`.
- `String? _backendMessageFrom(dynamic value)` — خاص — ينفّذ منطق `backend message` `from` داخل `ActivateCardRepositoryImpl`.
- `String _extractBackendMessages(dynamic value)` — خاص — ينفّذ منطق `extract` `backend messages` داخل `ActivateCardRepositoryImpl`.

features/activate_card/domain/repository/activate_card_repository.dart

- `class ActivateCardRepository` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات `activate card repository`.
- `Future<Either<String, ActivateCardResponseModel>> activateCard(ActivateCardRequestModel request)` — ينفّذ منطق `activate card` داخل `ActivateCardRepository`.

features/activate_card/domain/usecases/activate_card_usecase.dart

- `class ActivateCardUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `activate card use case`.
- `ActivateCardUseCase(this._repository)` — ينشئ كائنًا من `ActivateCardUseCase` ويهيّئ

الاعتماديات والقيم المطلوبة.

- `Future<Either<String, ActivateCardResponseModel>> call(ActivateCardRequestModel request)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/activate_card/presentation/cubit/activate_card_cubit.dart

- `class ActivateCardCubit` — الكلاس الذي يدير حالة وتفاعلات activate card cubit.
- `ActivateCardCubit(this._useCase)` — ينشئ كائنًا من ActivateCardCubit ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> activateCard(ActivateCardRequestModel request)` — ينفّذ منطق activate card داخل ActivateCardCubit.

features/activate_card/presentation/cubit/activate_card_state.dart

- `class ActivateCardState` — الكلاس الذي يدير حالة وتفاعلات activate card state.
- `class ActivateCardInitial` — الكلاس الذي يدير حالة وتفاعلات activate card initial.
- `class ActivateCardLoading` — الكلاس الذي يدير حالة وتفاعلات activate card loading.
- `class ActivateCardSuccess` — الكلاس الذي يدير حالة وتفاعلات activate card success.
- `ActivateCardSuccess(this.response)` — ينشئ كائنًا من ActivateCardSuccess ويهيّئ الاعتماديات والقيم المطلوبة.
- `class ActivateCardError` — الكلاس الذي يدير حالة وتفاعلات activate card error.
- `ActivateCardError(this.message)` — ينشئ كائنًا من ActivateCardError ويهيّئ الاعتماديات والقيم المطلوبة.

features/activate_card/presentation/pages/activate_card_screen.dart

- `class ActivateCardScreen` — الكلاس الذي يمثل شاشة activate card screen.
- `const ActivateCardScreen({super.key})` — ينشئ كائنًا من ActivateCardScreen ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ActivateCardScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ActivateCardScreenState` — خاص — الكلاس الذي يمثل شاشة activate card screen state.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onPasswordChanged()` — خاص — يعالج حدث on password changed داخل _ActivateCardScreenState.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildTextField({Key? fieldKey, required TextEditingController controller, required String hintText, String? fieldName, TextInputType? keyboardType, bool required = true, int? maxLength, TextCapitalization textCapitalization = TextCapitalization.none, List<TextInputFormatter>? inputFormatters, String? Function(String?)? validator, ValueChanged<String>? onChanged})` — خاص — ينفّذ منطق build text field داخل _ActivateCardScreenState.
- `Widget _buildPasswordField({Key? fieldKey, required TextEditingController controller, required String hintText, required bool obscure, required VoidCallback onToggle, String? fieldName, String? Function(String?)? validator})` — خاص — ينفّذ منطق build password field داخل _ActivateCardScreenState.
- `Widget _buildPasswordRequirements()` — خاص — ينفّذ منطق build password requirements داخل _ActivateCardScreenState.
- `Widget _buildPasswordRequirement({required String label, required bool`

`build password` منطق ينفّذ — خاص — `isMet, required bool showInvalid})` `requirement` داخل `_ActivateCardScreenState`.

- `Widget _buildRequiredHint(String text)` — خاص — ينفّذ منطق `build required hint` داخل `_ActivateCardScreenState`.
- `Widget _buildBirthDatePicker(BuildContext context)` — خاص — ينفّذ منطق `build` `birth date picker` داخل `_ActivateCardScreenState`.
- `DateTime get _selectedBirthDate` — خاص — يغيّر الاختيار الخاص بـ `selected birth date` داخل `_ActivateCardScreenState`.
- `DateTime? _parseBirthDate(String? value)` — خاص — يحلّل `parse birth date` ويحوّله لصيغة قابلة للاستخدام داخل `_ActivateCardScreenState`.
- `String? _validateFirstName(String? value)` — خاص — يتحقق من صحة `validate` `first name` داخل `_ActivateCardScreenState`.
- `String? _validateLastName(String? value)` — خاص — يتحقق من صحة `validate last` `name` داخل `_ActivateCardScreenState`.
- `String? _validateCardNumber(String? value)` — خاص — يتحقق من صحة `validate` `card number` داخل `_ActivateCardScreenState`.
- `String? _validatePhoneNumber(String? value)` — خاص — يتحقق من صحة `validate` `phone number` داخل `_ActivateCardScreenState`.
- `String? _validateNationalId(String? value)` — خاص — يتحقق من صحة `validate` `national id` داخل `_ActivateCardScreenState`.
- `String? _validatePassportNumber(String? value)` — خاص — يتحقق من صحة `validate` `passport number` داخل `_ActivateCardScreenState`.
- `bool _isPasswordValid(String password)` — خاص — يفحص شرط `is password valid` ويعيد النتيجة داخل `_ActivateCardScreenState`.
- `bool _hasMinimumLength(String password)` — خاص — يفحص شرط `has minimum` `length` ويعيد النتيجة داخل `_ActivateCardScreenState`.
- `bool _hasLetter(String password)` — خاص — يفحص شرط `has letter` ويعيد النتيجة داخل `_ActivateCardScreenState`.
- `bool _hasNumber(String password)` — خاص — يفحص شرط `has number` ويعيد النتيجة داخل `_ActivateCardScreenState`.
- `bool _hasSpecialCharacter(String password)` — خاص — يفحص شرط `has special` `character` ويعيد النتيجة داخل `_ActivateCardScreenState`.
- `String _normalize(String value)` — خاص — ينفّذ منطق `normalize` داخل `_ActivateCardScreenState`.
- `Widget _buildGenderSelector()` — خاص — ينفّذ منطق `build gender selector` داخل `_ActivateCardScreenState`.

Feature: activity

`features/activity/data/data_source/invoice_remote_data_source.dart`

- `class InvoiceRemoteDataSource` — الكلاس الذي يعرّف الاتصال بمصدر البيانات أو REST API.
- `factory InvoiceRemoteDataSource(Dio dio, {String baseUrl})` — ينشئ كائنًا من `InvoiceRemoteDataSource` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<InvoicesResponseDto> getMyInvoices({@Header('Accept-Language') required String language, @Query('pageNumber') int pageNumber = 1, @Query('pageSize') int pageSize = 10, @Query('Tab') int? tab})` — يجلب `get` `my invoices` من مصدر البيانات المناسب داخل `InvoiceRemoteDataSource`.
- `Future<CreateInvoiceResponseDto> createFromPackage(@Body()`

`CreatePackageInvoiceRequestDto request)` — ينشئ أو يضيف `create from package` داخل `InvoiceRemoteDataSource`.

- `Future<CreateInvoiceResponseDto> createFromCart(@Body()` `CreateCartInvoiceRequestDto request)` — ينشئ أو يضيف `create from cart` داخل `InvoiceRemoteDataSource`.
- `Future<InvoiceDetailsResponseDto> getInvoiceDetails(@Path('invoiceId')` `int invoiceId)` — يجلب `get invoice details` من مصدر البيانات المناسب داخل `InvoiceRemoteDataSource`.
- `Future<CreateInvoiceResponseDto> createReview(@Body()` `CreateReviewRequestDto request)` — ينشئ أو يضيف `create review` داخل `InvoiceRemoteDataSource`.
- `Future<CreateInvoiceResponseDto> cancelInvoice(@Path('invoiceId') int` `invoiceId)` — يفحص شرط `cancel invoice` ويعيد النتيجة داخل `InvoiceRemoteDataSource`.

features/activity/data/dto/create_cart_invoice_request_dto.dart

- `class CreateCartInvoiceRequestDto` — الكلاس الذي يمثل بيانات `create cart invoice request dto` وينقلها بين الطبقات.
- `const CreateCartInvoiceRequestDto({this.cardNumber, required this.isCash, required this.providerId, required this.providerBranchId, this.familyId})` — ينشئ كائنًا من `CreateCartInvoiceRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CreateCartInvoiceRequestDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CreateCartInvoiceRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activity/data/dto/create_invoice_response_dto.dart

- `class CreateInvoiceResponseDto` — الكلاس الذي يمثل بيانات `create invoice response dto` وينقلها بين الطبقات.
- `const CreateInvoiceResponseDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `CreateInvoiceResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CreateInvoiceResponseDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CreateInvoiceResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activity/data/dto/create_package_invoice_request_dto.dart

- `class CreatePackageInvoiceRequestDto` — الكلاس الذي يمثل بيانات `create package invoice request dto` وينقلها بين الطبقات.
- `const CreatePackageInvoiceRequestDto({required this.packageId, required this.providerBranchId, required this.isCash, this.cardNumber})` — ينشئ كائنًا من `CreatePackageInvoiceRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CreatePackageInvoiceRequestDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CreatePackageInvoiceRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activity/data/dto/create_review_request_dto.dart

- `class CreateReviewRequestDto` — الكلاس الذي يمثل بيانات `create review request dto` وينقلها بين الطبقات.
- `const CreateReviewRequestDto({required this.invoiceId, this.rating, this.ratingText})` — ينشئ كائنًا من `CreateReviewRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activity/data/dto/invoice_details_response_dto.dart

- `class InvoiceDetailsResponseDto` — الكلاس الذي يمثل بيانات `invoice details response` وينقلها بين الطبقات.
- `const InvoiceDetailsResponseDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `InvoiceDetailsResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoiceDetailsResponseDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoiceDetailsResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceDetailsDto` — الكلاس الذي يمثل بيانات `invoice details dto` وينقلها بين الطبقات.
- `const InvoiceDetailsDto({this.invoiceId, this.status, this.date, this.totalBefore, this.totalAfter, this.discountPercentage, this.familyId, this.familyFirstName, this.familyLastName, this.items})` — ينشئ كائنًا من `InvoiceDetailsDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoiceDetailsDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoiceDetailsDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceServiceItemsPageDto` — الكلاس الذي يمثل بيانات `invoice service items page dto` وينقلها بين الطبقات.
- `const InvoiceServiceItemsPageDto({required this.items})` — ينشئ كائنًا من `InvoiceServiceItemsPageDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoiceServiceItemsPageDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoiceServiceItemsPageDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceServiceItemDto` — الكلاس الذي يمثل بيانات `invoice service item dto` وينقلها بين الطبقات.
- `const InvoiceServiceItemDto({this.id, this.serviceName, this.serviceNameEn, this.priceBefore, this.priceAfter, this.discountPercentage})` — ينشئ كائنًا من `InvoiceServiceItemDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoiceServiceItemDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoiceServiceItemDto` ويهيّئ الاعتماديات والقيم المطلوبة.

features/activity/data/dto/invoice_item_dto.dart

- `class InvoiceItemDto` — الكلاس الذي يمثل بيانات `invoice item dto` وينقلها بين الطبقات.
- `const InvoiceItemDto({this.invoiceId, this.providerName, this.branchName, this.status, this.date, this.providerImageUrl, this.isReviewed, this.isFromPackage, this.priceAfter, this.packagePrice, this.familyId, this.familyFirstName, this.familyLastName})` — ينشئ كائنًا من `InvoiceItemDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoiceItemDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoiceItemDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activity/data/dto/invoices_response_dto.dart

- `class InvoicesResponseDto` — الكلاس الذي يمثل بيانات `invoices response dto` وينقلها بين الطبقات.
- `const InvoicesResponseDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `InvoicesResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoicesResponseDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoicesResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

- `class InvoicesPageDto` — الكلاس الذي يمثل بيانات `invoices page dto` وينقلها بين الطبقات.
- `const InvoicesPageDto({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `InvoicesPageDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory InvoicesPageDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `InvoicesPageDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/activity/data/repositories/invoice_repository_impl.dart

- `class InvoiceRepositoryImpl` — الكلاس الذي يمثل بيانات `invoice repository impl` وينقلها بين الطبقات.
- `InvoiceRepositoryImpl(this._dataSource)` — ينشئ كائنًا من `InvoiceRepositoryImpl` ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool _isAmbiguousCreationError(Object error)` — خاص — يفحص شرط `is ambiguous creation error` ويعيد النتيجة داخل `InvoiceRepositoryImpl`.
- `String _failureMessage({dynamic errors, String? message, required String fallback})` — خاص — ينفّذ منطق `failure message` داخل `InvoiceRepositoryImpl`.
- `String? _readableError(dynamic errors)` — خاص — ينفّذ منطق `readable error` داخل `InvoiceRepositoryImpl`.
- `Future<ApiResult<InvoicesPage>> getMyInvoices({int pageNumber = 1, int pageSize = 10, int? tab})` — يجلب `get my invoices` من مصدر البيانات المناسب داخل `InvoiceRepositoryImpl`.
- `Future<ApiResult<CreatedInvoice>> createFromPackage({required int packageId, required int providerBranchId, bool isCash = true, String? cardNumber})` — ينشئ أو يضيف `create from package` داخل `InvoiceRepositoryImpl`.
- `Future<ApiResult<CreatedInvoice>> createFromCart({required int providerId, required int providerBranchId, required bool isCash, int? familyId, String? cardNumber})` — ينشئ أو يضيف `create from cart` داخل `InvoiceRepositoryImpl`.
- `String? _beneficiaryName({String? firstName, String? lastName})` — خاص — ينفّذ منطق `beneficiary name` داخل `InvoiceRepositoryImpl`.
- `CreatedInvoice? _createdInvoiceFromData(dynamic data)` — خاص — ينشئ أو يضيف `created invoice from data` داخل `InvoiceRepositoryImpl`.
- `String? _validInvoiceId(dynamic value)` — خاص — ينفّذ منطق `valid invoice id` داخل `InvoiceRepositoryImpl`.
- `Future<ApiResult<InvoiceDetails>> getInvoiceDetails(int invoiceId)` — يجلب `get invoice details` من مصدر البيانات المناسب داخل `InvoiceRepositoryImpl`.
- `Future<ApiResult<void>> createReview({required int invoiceId, int? rating, String? ratingText})` — ينشئ أو يضيف `create review` داخل `InvoiceRepositoryImpl`.
- `Future<ApiResult<void>> cancelInvoice(int invoiceId)` — يفحص شرط `cancel invoice` ويعيد النتيجة داخل `InvoiceRepositoryImpl`.

features/activity/domain/models/created_invoice.dart

- `class CreatedInvoice` — الكلاس الذي يمثل بيانات `created invoice` وينقلها بين الطبقات.
- `const CreatedInvoice({required this.invoiceId, this.paymentLink})` — ينشئ كائنًا من `CreatedInvoice` ويهيّئ الاعتماديات والقيم المطلوبة.

features/activity/domain/models/invoice_details.dart

- `class InvoiceDetails` — الكلاس الذي يمثل بيانات `invoice details` وينقلها بين الطبقات.

- `const InvoiceDetails({required this.invoiceId, required this.status, required this.date, required this.totalBefore, required this.totalAfter, required this.discountPercentage, this.beneficiaryName, required this.services})` — ينشئ كائنًا من `InvoiceDetails` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceServiceItem` — الكلاس الذي يمثل بيانات `invoice service item` وينقلها بين الطبقات.
- `const InvoiceServiceItem({required this.id, required this.serviceName, required this.priceBefore, required this.priceAfter, required this.discountPercentage})` — ينشئ كائنًا من `InvoiceServiceItem` ويهيّئ الاعتماديات والقيم المطلوبة.

features/activity/domain/models/invoice_item.dart

- `class InvoiceItem` — الكلاس الذي يمثل بيانات `invoice item` وينقلها بين الطبقات.
- `bool get isPending` — يفحص شرط `is pending` ويعيد النتيجة داخل `InvoiceItem`.
- `bool get isPaid` — يفحص شرط `is paid` ويعيد النتيجة داخل `InvoiceItem`.
- `bool get isUsed` — يفحص شرط `is used` ويعيد النتيجة داخل `InvoiceItem`.
- `bool get isCancelled` — يفحص شرط `is cancelled` ويعيد النتيجة داخل `InvoiceItem`.
- `const InvoiceItem({required this.invoiceId, required this.providerName, required this.branchName, required this.status, required this.date, this.providerImageUrl, this.isReviewed = false, this.isFromPackage = false, this.priceAfter, this.packagePrice, this.beneficiaryName})` — ينشئ كائنًا من `InvoiceItem` ويهيّئ الاعتماديات والقيم المطلوبة.

features/activity/domain/models/invoices_page.dart

- `class InvoicesPage` — الكلاس الذي يمثل بيانات `invoices page` وينقلها بين الطبقات.
- `const InvoicesPage({required this.items, required this.pageNumber, required this.pageSize, required this.totalCount, required this.totalPages})` — ينشئ كائنًا من `InvoicesPage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool get hasMore` — يفحص شرط `has more` ويعيد النتيجة داخل `InvoicesPage`.

features/activity/domain/repositories/invoice_repository.dart

- `class InvoiceRepository` — الكلاس الذي يجمع مسؤوليات `invoice repository`.
- `Future<ApiResult<InvoicesPage>> getMyInvoices({int pageNumber = 1, int pageSize = 10, int? tab})` — يجلب من مصدر البيانات المناسب داخل `get my invoices` `InvoiceRepository`.
- `Future<ApiResult<CreatedInvoice>> createFromPackage({required int packageId, required int providerBranchId, bool isCash = true, String? cardNumber})` — ينشئ أو يضيف `create from package` داخل `InvoiceRepository`.
- `Future<ApiResult<CreatedInvoice>> createFromCart({required int providerId, required int providerBranchId, required bool isCash, int? familyId, String? cardNumber})` — ينشئ أو يضيف `create from cart` داخل `InvoiceRepository`.
- `Future<ApiResult<InvoiceDetails>> getInvoiceDetails(int invoiceId)` — يجلب من مصدر البيانات المناسب داخل `get invoice details` `InvoiceRepository`.
- `Future<ApiResult<void>> createReview({required int invoiceId, int? rating, String? ratingText})` — ينشئ أو يضيف `create review` داخل `InvoiceRepository`.
- `Future<ApiResult<void>> cancelInvoice(int invoiceId)` — يفحص شرط `cancel` `invoice` ويعيد النتيجة داخل `InvoiceRepository`.

features/activity/domain/usecases/cancel_invoice_usecase.dart

- `class CancelInvoiceUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `cancel invoice use`

case.

- `CancelInvoiceUseCase(this._repository)` — ينشئ كائنًا من `CancelInvoiceUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<void>> call(int invoiceId)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/activity/domain/usecases/create_review_usecase.dart

- `class CreateReviewUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `create review use case`.
- `CreateReviewUseCase(this._repository)` — ينشئ كائنًا من `CreateReviewUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<void>> call({required int invoiceId, int? rating, String? ratingText})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/activity/domain/usecases/get_invoice_details_usecase.dart

- `class GetInvoiceDetailsUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get invoice details use case`.
- `GetInvoiceDetailsUseCase(this._repository)` — ينشئ كائنًا من `GetInvoiceDetailsUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<InvoiceDetails>> call(int invoiceId)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/activity/presentation/cubit/activity_cubit.dart

- `class ActivityCubit` — الكلاس الذي يدير حالة وتفاعلات `activity cubit`.
- `ActivityCubit(this._repository)` — ينشئ كائنًا من `ActivityCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadInvoices({int tab = 1, bool forceRefresh = false})` — يحمّل `load invoices` ويحدّث الحالة المرتبطة داخل `ActivityCubit`.
- `Future<void> loadMore()` — يحمّل `load more` ويحدّث الحالة المرتبطة داخل `ActivityCubit`.
- `Future<void> _performLoadInvoices({required int tab})` — خاص — ينفّذ منطق `perform load invoices` داخل `ActivityCubit`.
- `Future<void> _performLoadMore()` — خاص — ينفّذ منطق `perform load more` داخل `ActivityCubit`.

features/activity/presentation/cubit/activity_state.dart

- `class ActivityState` — الكلاس الذي يدير حالة وتفاعلات `activity state`.
- `const factory ActivityState.initial()` — ينشئ كائنًا من `ActivityState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ActivityState.loading()` — ينشئ كائنًا من `ActivityState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ActivityState.success({required List<InvoiceItem> invoices, @Default(false) bool isLoadingMore, @Default(false) bool hasMore})` — ينشئ كائنًا من `ActivityState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ActivityState.failure(String error)` — ينشئ كائنًا من `ActivityState` ويهيّئ الاعتماديات والقيم المطلوبة.

features/activity/presentation/cubit/invoice_details_cubit.dart

- `class InvoiceDetailsCubit` — الكلاس الذي يدير حالة وتفاعلات `invoice details cubit`.
- `InvoiceDetailsCubit(this._getInvoiceDetails, this._cancelInvoice, this._createReview)` — ينشئ كائنًا من `InvoiceDetailsCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadDetails(int invoiceId)` — يحمّل `load details` ويحدّث الحالة المرتبطة

داخل `InvoiceDetailsCubit`.

- `Future<String?> cancelInvoice(int invoiceId)` — يفحص شرط `cancel invoice` ويعيد النتيجة داخل `InvoiceDetailsCubit`.
- `Future<String?> submitReview({required int invoiceId, int? rating, String? ratingText})` — يرسل `submit review` إلى الطبقة أو الخدمة المطلوبة داخل `InvoiceDetailsCubit`.

features/activity/presentation/cubit/invoice_details_state.dart

- `class InvoiceDetailsState` — الكلاس الذي يدير حالة وتفاعلات `invoice details state`.
- `const InvoiceDetailsState()` — ينشئ كائنًا من `InvoiceDetailsState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceDetailsInitial` — الكلاس الذي يدير حالة وتفاعلات `invoice details initial`.
- `const InvoiceDetailsInitial()` — ينشئ كائنًا من `InvoiceDetailsInitial` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceDetailsLoading` — الكلاس الذي يدير حالة وتفاعلات `invoice details loading`.
- `const InvoiceDetailsLoading()` — ينشئ كائنًا من `InvoiceDetailsLoading` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceDetailsLoaded` — الكلاس الذي يدير حالة وتفاعلات `invoice details loaded`.
- `const InvoiceDetailsLoaded(this.details)` — ينشئ كائنًا من `InvoiceDetailsLoaded` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class InvoiceDetailsLoadFailure` — الكلاس الذي يدير حالة وتفاعلات `invoice details load failure`.
- `const InvoiceDetailsLoadFailure(this.message)` — ينشئ كائنًا من `InvoiceDetailsLoadFailure` ويهيّئ الاعتماديات والقيم المطلوبة.

features/activity/presentation/pages/invoice_details_screen.dart

- `class InvoiceDetailsScreen` — الكلاس الذي يمثل شاشة `invoice details screen`.
- `const InvoiceDetailsScreen({super.key, required this.invoice, this.tab = 0})` — ينشئ كائنًا من `InvoiceDetailsScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _DetailsBody` — خاص — الكلاس الذي يمثل شاشة `details body`.
- `const _DetailsBody({required this.invoice, required this.details, required this.tab})` — خاص — ينشئ كائنًا من `DetailsBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _servicesText(List<InvoiceServiceItem> services)` — خاص — ينفّذ منطق `services text` داخل `_DetailsBody`.
- `class _StatusBadge` — خاص — الكلاس الذي يمثل شاشة `status badge`.
- `const _StatusBadge({required this.status})` — خاص — ينشئ كائنًا من `StatusBadge` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ProviderHeader` — خاص — الكلاس الذي يمثل شاشة `provider header`.
- `const _ProviderHeader({required this.invoice})` — خاص — ينشئ كائنًا من `ProviderHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

- `class _InfoCard` — خاص — الكلاس الذي يمثل شاشة `info card`.
- `const _InfoCard({required this.children})` — خاص — ينشئ كائنًا من `InfoCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _InfoRow` — خاص — الكلاس الذي يمثل شاشة `info row`.
- `const _InfoRow({required this.label, required this.value, this.valueColor, this.isBold = false})` — خاص — ينشئ كائنًا من `InfoRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _DetailsActions` — خاص — الكلاس الذي يمثل شاشة `details actions`.
- `const _DetailsActions({required this.invoice, required this.details, required this.tab})` — خاص — ينشئ كائنًا من `DetailsActions` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_DetailsActions> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _DetailsActionsState` — خاص — الكلاس الذي يمثل شاشة `details actions state`.
- `bool get _canCancel` — خاص — يفحص شرط `can cancel` ويعيد النتيجة داخل `_DetailsActionsState`.
- `bool get _canReview` — خاص — يفحص شرط `can review` ويعيد النتيجة داخل `_DetailsActionsState`.
- `Future<void> _onCancelTapped()` — خاص — يعالج حدث `on cancel tapped` داخل `_DetailsActionsState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ReviewDialog` — خاص — الكلاس الذي يمثل شاشة `review dialog`.
- `const _ReviewDialog({required this.invoiceId})` — خاص — ينشئ كائنًا من `ReviewDialog` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_ReviewDialog> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ReviewDialogState` — خاص — الكلاس الذي يمثل شاشة `review dialog state`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Future<void> _submit()` — خاص — يرسل `submit` إلى الطبقة أو الخدمة المطلوبة داخل `_ReviewDialogState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _money(double value)` — خاص — ينفّذ منطق `money` داخل هذا الملف.
- `double _discountPercent(InvoiceDetails details)` — خاص — ينفّذ منطق `discount percent` داخل هذا الملف.
- `String _localizedStatus(String status)` — خاص — ينفّذ منطق `localized status` داخل هذا الملف.

Feature: auth

`features/auth/data/models/otp_response_model.dart`

- `class OtpResponseModel` — الكلاس الذي يمثل بيانات `otp response model` وينقلها بين الطبقات.
- `const OtpResponseModel({this.httpStatusCode, this.errors, required this.succeeded, this.message, this.data})` — ينشئ كائنًا من `OtpResponseModel` ويهيّئ

الاعتماديات والقيم المطلوبة.

- `factory OtpResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `OtpResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `Object? _readSucceeded(Map<dynamic, dynamic> json, String key)` — خاص — ينفّذ منطق `read succeeded` داخل هذا الملف.

`features/auth/data/models/reset_password_request_model.dart`

- `class ResetPasswordRequestModel` — الكلاس الذي يمثل بيانات `reset password request` وينقلها بين الطبقات.
- `const ResetPasswordRequestModel({required this.phoneNumber, required this.password, required this.confirmPassword})` — ينشئ كائنًا من `ResetPasswordRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ResetPasswordRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ResetPasswordRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/auth/data/models/send_otp_request_model.dart`

- `class SendOtpRequestModel` — الكلاس الذي يمثل بيانات `send otp request model` وينقلها بين الطبقات.
- `const SendOtpRequestModel({required this.phoneNumber})` — ينشئ كائنًا من `SendOtpRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory SendOtpRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `SendOtpRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/auth/data/models/verify_otp_request_model.dart`

- `class VerifyOtpRequestModel` — الكلاس الذي يمثل بيانات `verify otp request model` وينقلها بين الطبقات.
- `const VerifyOtpRequestModel({required this.phoneNumber, required this.otp})` — ينشئ كائنًا من `VerifyOtpRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory VerifyOtpRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `VerifyOtpRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/auth/domain/usecases/reset_password_usecase.dart`

- `class ResetPasswordUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `reset password use case`.
- `ResetPasswordUseCase(this._repository)` — ينشئ كائنًا من `ResetPasswordUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<OtpResponseModel>> call({required String phoneNumber, required String password, required String confirmPassword})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`features/auth/domain/usecases/send_otp_usecase.dart`

- `class SendOtpUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `send otp use case`.
- `SendOtpUseCase(this._repository)` — ينشئ كائنًا من `SendOtpUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<OtpResponseModel>> call(String phoneNumber)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/auth/domain/usecases/verify_otp_usecase.dart

- `class VerifyOtpUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: verify otp use case.
- `VerifyOtpUseCase(this._repository)` — ينشئ كائنًا من VerifyOtpUseCase ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<OtpResponseModel>> call({required String phoneNumber, required String otp})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/auth/presentation/cubit/forgot_password_cubit.dart

- `class ForgotPasswordCubit` — الكلاس الذي يدير حالة وتفاعلات forgot password cubit.
- `ForgotPasswordCubit(this._sendOtpUseCase, this._verifyOtpUseCase, this._resetPasswordUseCase)` — ينشئ كائنًا من ForgotPasswordCubit ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> sendOtp(String phoneNumber)` — يرسل send otp إلى الطبقة أو الخدمة المطلوبة داخل ForgotPasswordCubit.
- `Future<void> verifyOtp({required String phoneNumber, required String otp})` — ينفّذ منطق verify otp داخل ForgotPasswordCubit.
- `Future<void> resetPassword({required String phoneNumber, required String password, required String confirmPassword})` — ينفّذ منطق reset password داخل ForgotPasswordCubit.

features/auth/presentation/cubit/forgot_password_state.dart

- `class ForgotPasswordState` — الكلاس الذي يدير حالة وتفاعلات forgot password state.
- `const factory ForgotPasswordState.initial()` — ينشئ كائنًا من ForgotPasswordState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ForgotPasswordState.loading()` — ينشئ كائنًا من ForgotPasswordState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ForgotPasswordState.otpSent(String phoneNumber, String otp)` — ينشئ كائنًا من ForgotPasswordState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ForgotPasswordState.otpVerified(String phoneNumber, String token)` — ينشئ كائنًا من ForgotPasswordState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ForgotPasswordState.resetSuccess()` — ينشئ كائنًا من ForgotPasswordState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ForgotPasswordState.failed(String error)` — ينشئ كائنًا من ForgotPasswordState ويهيّئ الاعتماديات والقيم المطلوبة.

features/auth/presentation/screens/forgot_password_screen.dart

- `class ForgotPasswordScreen` — الكلاس الذي يمثل شاشة forgot password screen.
- `const ForgotPasswordScreen({super.key})` — ينشئ كائنًا من ForgotPasswordScreen ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ForgotPasswordScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ForgotPasswordScreenState` — خاص — الكلاس الذي يمثل شاشة forgot password screen state.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onSendOtpPressed(BuildContext context)` — خاص — يعالج حدث on send otp pressed داخل _ForgotPasswordScreenState.
- `bool _isAccountExistsInfoMessage(String message)` — خاص — يفحص شرط is account exists info message ويعيد النتيجة داخل _ForgotPasswordScreenState.
- `String? _validatePhone(String? value)` — خاص — يتحقق من صحة validate phone

داخل `_ForgotPasswordScreenState`.

- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/auth/presentation/screens/otp_verification_screen.dart

- `class OtpVerificationScreen` — الكلاس الذي يمثل شاشة `otp verification screen`.
- `const OtpVerificationScreen({super.key, required this.phoneNumber, required this.otp})` — ينشئ كائنًا من `OtpVerificationScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<OtpVerificationScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _OtpVerificationScreenState` — خاص — الكلاس الذي يمثل شاشة `otp verification screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _startCountdown()` — خاص — ينفّذ منطق `start countdown` داخل `_OtpVerificationScreenState`.
- `void _startResendCooldown()` — خاص — ينفّذ منطق `start resend cooldown` داخل `_OtpVerificationScreenState`.
- `void _restartCountdown(String otp)` — خاص — ينفّذ منطق `restart countdown` داخل `_OtpVerificationScreenState`.
- `void _onVerifyPressed(BuildContext context)` — خاص — يعالج حدث `on verify pressed` داخل `_OtpVerificationScreenState`.
- `String? _validateOtp(String? value)` — خاص — يتحقق من صحة `validate otp` داخل `_OtpVerificationScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _OtpPreviewCard` — خاص — الكلاس الذي يمثل شاشة `otp preview card`.
- `const _OtpPreviewCard({required this.otp, required this.secondsLeft})` — خاص — ينشئ كائنًا من `OtpPreviewCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/auth/presentation/screens/reset_password_screen.dart

- `class ResetPasswordScreen` — الكلاس الذي يمثل شاشة `reset password screen`.
- `const ResetPasswordScreen({super.key, required this.phoneNumber, required this.token})` — ينشئ كائنًا من `ResetPasswordScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ResetPasswordScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ResetPasswordScreenState` — خاص — الكلاس الذي يمثل شاشة `reset password screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onPasswordChanged()` — خاص — يعالج حدث `on password changed` داخل `_ResetPasswordScreenState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onResetPressed(BuildContext context)` — خاص — يعالج حدث `on reset pressed` داخل `_ResetPasswordScreenState`.
- `bool _hasMinimumLength(String password)` — خاص — يفحص شرط `has minimum length` ويعيد النتيجة داخل `_ResetPasswordScreenState`.
- `bool _hasLetter(String password)` — خاص — يفحص شرط `has letter` ويعيد النتيجة داخل

_ResetPasswordScreenState.

- `bool _hasNumber(String password)` — خاص — يفحص شرط `has number` ويعيد النتيجة داخل _ResetPasswordScreenState.
- `bool _hasSpecialCharacter(String password)` — خاص — يفحص شرط `has special character` ويعيد النتيجة داخل _ResetPasswordScreenState.
- `bool _isPasswordValid(String password)` — خاص — يفحص شرط `is password valid` ويعيد النتيجة داخل _ResetPasswordScreenState.
- `String? _validatePassword(String? value)` — خاص — يتحقق من صحة `validate password` داخل _ResetPasswordScreenState.
- `String? _validateConfirm(String? value)` — خاص — يتحقق من صحة `validate confirm` داخل _ResetPasswordScreenState.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildPasswordRequirements()` — خاص — ينفّذ منطق `build password requirements` داخل _ResetPasswordScreenState.
- `Widget _buildPasswordRequirement({required String label, required bool isMet, required bool showInvalid})` — خاص — ينفّذ منطق `build password requirement` داخل _ResetPasswordScreenState.

Feature: cart

features/cart/data/api/cart_remote_data_source.dart

- `class CartRemoteDataSource` — الكلاس الذي يعرّف الاتصال بمصدر البيانات أو REST API.
- `factory CartRemoteDataSource(Dio dio, {String baseUrl})` — ينشئ كائنًا من CartRemoteDataSource ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<CartDto> getCart(@Header('Authorization') String token)` — يجلب `get cart` من مصدر البيانات المناسب داخل CartRemoteDataSource.
- `Future<UpdateCartProviderResponseDto> addToCart(@Header('Authorization') String token, @Body() AddCartRequestDto request)` — ينشئ أو يضيف `add to cart` داخل CartRemoteDataSource.
- `Future<UpdateCartProviderResponseDto> removeFromCart(@Header('Authorization') String token, @Query('ServiceId') int serviceId)` — يحذف أو يمسح `remove from cart` داخل CartRemoteDataSource.
- `Future<CategoryServicesDto> getCategoryServices(@Header('Authorization') String token, @Path('categoryId') int categoryId)` — يجلب `get category services` من مصدر البيانات المناسب داخل CartRemoteDataSource.
- `Future<UpdateCartProviderResponseDto> clearCart(@Header('Authorization') String token)` — يحذف أو يمسح `clear cart` داخل CartRemoteDataSource.
- `Future<ChooseProvidersDto> getChooseProviders(@Header('Authorization') String token)` — يجلب `get choose providers` من مصدر البيانات المناسب داخل CartRemoteDataSource.
- `Future<UpdateCartProviderResponseDto> updateCartProvider(@Header('Authorization') String token, @Body() UpdateCartProviderRequestDto request)` — يحدّث `update cart provider` داخل CartRemoteDataSource.

features/cart/data/dto/add_cart_request_dto.dart

- `class AddCartRequestDto` — الكلاس الذي يمثل بيانات `add cart request dto` وينقلها بين الطبقات.

- `const AddCartRequestDto({required this.serviceId, required this.categoryId, this.providerId, this.cardNumber, this.price})` — ينشئ كائنًا من `AddCartRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AddCartRequestDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AddCartRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/cart/data/dto/cart_dto.dart

- `class CartDto` — الكلاس الذي يمثل بيانات `cart dto` وينقلها بين الطبقات.
- `const CartDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `CartDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CartDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CartDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `List<String>? _stringList(dynamic value)` — خاص — ينفّذ منطق `string list` داخل هذا الملف.
- `class CartDataDto` — الكلاس الذي يمثل بيانات `cart data dto` وينقلها بين الطبقات.
- `const CartDataDto({this.items, this.totalCount, this.categoryCounters})` — ينشئ كائنًا من `CartDataDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CartDataDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CartDataDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/cart/data/dto/cart_item_dto.dart

- `class CartItemDto` — الكلاس الذي يمثل بيانات `cart item dto` وينقلها بين الطبقات.
- `const CartItemDto({this.cartItemId, this.serviceId, this.serviceName, this.categoryId, this.categoryName, this.providerId, this.providerName, this.priceAfter, this.priceBefore, this.discountPercentage})` — ينشئ كائنًا من `CartItemDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CartItemDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CartItemDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/cart/data/dto/category_counter_dto.dart

- `class CategoryCounterDto` — الكلاس الذي يمثل بيانات `category counter dto` وينقلها بين الطبقات.
- `const CategoryCounterDto({this.categoryId, this.categoryName, this.count})` — ينشئ كائنًا من `CategoryCounterDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryCounterDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryCounterDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/cart/data/dto/category_service_dto.dart

- `class CategoryServiceDto` — الكلاس الذي يمثل بيانات `category service dto` وينقلها بين الطبقات.
- `const CategoryServiceDto({this.serviceId, this.serviceNameEn, this.serviceNameAr, this.isSelected})` — ينشئ كائنًا من `CategoryServiceDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String? get serviceName` — ينفّذ منطق `service name` داخل `CategoryServiceDto`.
- `factory CategoryServiceDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا

من `CategoryServiceDto` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/cart/data/dto/category_services_dto.dart`

- `class CategoryServicesDto` — الكلاس الذي يمثل بيانات `category services dto` وينقلها بين الطبقات.
- `const CategoryServicesDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `CategoryServicesDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryServicesDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryServicesDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CategoryServicesDataDto` — الكلاس الذي يمثل بيانات `category services data dto` وينقلها بين الطبقات.
- `const CategoryServicesDataDto({this.categoryId, this.categoryName, this.services, this.totalSelectedCount})` — ينشئ كائنًا من `CategoryServicesDataDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `List<CategoryServiceDto> get items` — ينفّذ منطق `items` داخل `CategoryServicesDataDto`.
- `factory CategoryServicesDataDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryServicesDataDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CategoryServicesItemsWrapperDto` — الكلاس الذي يمثل بيانات `category services items wrapper dto` وينقلها بين الطبقات.
- `const CategoryServicesItemsWrapperDto({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `CategoryServicesItemsWrapperDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryServicesItemsWrapperDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryServicesItemsWrapperDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/cart/data/dto/choose_providers_dto.dart`

- `class ChooseProvidersDto` — الكلاس الذي يمثل بيانات `choose providers dto` وينقلها بين الطبقات.
- `const ChooseProvidersDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ChooseProvidersDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ChooseProvidersDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ChooseProvidersDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ChooseProvidersDataDto` — الكلاس الذي يمثل بيانات `choose providers data dto` وينقلها بين الطبقات.
- `const ChooseProvidersDataDto({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `ChooseProvidersDataDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ChooseProvidersDataDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ChooseProvidersDataDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderOptionDto` — الكلاس الذي يمثل بيانات `provider option dto` وينقلها بين الطبقات.

- `const ProviderOptionDto({this.providerId, this.providerName, this.providerImageUrl, this.coveredCount, this.uncoveredCount, this.totalCount, this.totalPriceBefore, this.totalPriceAfter, this.coveredServices, this.uncoveredServices})` — ينشئ كائنًا من `ProviderOptionDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderOptionDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderOptionDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderCoveredServiceDto` — الكلاس الذي يمثل بيانات `provider covered service` وينقلها بين الطبقات `dto`.
- `const ProviderCoveredServiceDto({this.cartItemId, this.serviceId, this.serviceName, this.priceBefore, this.priceAfter, this.discountPercentage})` — ينشئ كائنًا من `ProviderCoveredServiceDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderCoveredServiceDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderCoveredServiceDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/cart/data/dto/update_cart_provider_request_dto.dart

- `class UpdateCartProviderRequestDto` — الكلاس الذي يمثل بيانات `update cart provider` وينقلها بين الطبقات `request dto`.
- `const UpdateCartProviderRequestDto({required this.cartId, required this.providerId})` — ينشئ كائنًا من `UpdateCartProviderRequestDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/cart/data/dto/update_cart_provider_response_dto.dart

- `class UpdateCartProviderResponseDto` — الكلاس الذي يمثل بيانات `update cart provider` وينقلها بين الطبقات `response dto`.
- `const UpdateCartProviderResponseDto({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `UpdateCartProviderResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory UpdateCartProviderResponseDto.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `UpdateCartProviderResponseDto` ويهيّئ الاعتماديات والقيم المطلوبة.

features/cart/data/repository/cart_repository_impl.dart

- `class CartRepositoryImpl` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات `cart repository impl`.
- `CartRepositoryImpl(this._dataSource)` — ينشئ كائنًا من `CartRepositoryImpl` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<String> get _bearerToken` — خاص — ينفّذ منطق `bearer token` داخل `CartRepositoryImpl`.
- `String _failureMessage({List<String>? errors, String? message, required String fallback})` — خاص — ينفّذ منطق `failure message` داخل `CartRepositoryImpl`.
- `Cart _mapCartDtoToDomain(CartDto dto)` — خاص — يحوّل `map cart dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.
- `CartItem _mapCartItemDtoToDomain(CartItemDto dto)` — خاص — يحوّل `map cart item dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.
- `CategoryCounter _mapCategoryCounterDtoToDomain(CategoryCounterDto dto)` — خاص — يحوّل `map category counter dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.

- `CategoryServices _mapCategoryServicesDtoToDomain(CategoryServicesDto dto)` — خاص — يحوّل `map category services dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.
- `CategoryService _mapCategoryServiceDtoToDomain(CategoryServiceDto dto)` — خاص — يحوّل `map category service dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.
- `Future<ApiResult<Cart>> getCart()` — يجلب `get cart` من مصدر البيانات المناسب داخل `CartRepositoryImpl`.
- `Future<ApiResult<void>> addToCart({required int serviceId, required int categoryId, int? providerId, String? cardNumber, double? price})` — ينشئ أو يضيف `add to cart` داخل `CartRepositoryImpl`.
- `Future<ApiResult<void>> removeFromCart(int serviceId)` — يحذف أو يمسح `remove from cart` داخل `CartRepositoryImpl`.
- `Future<ApiResult<void>> updateCartProvider({required int cartId, required int providerId})` — يحدّث `update cart provider` داخل `CartRepositoryImpl`.
- `Future<ApiResult<CategoryServices>> getCategoryServices(int categoryId)` — يجلب `get category services` من مصدر البيانات المناسب داخل `CartRepositoryImpl`.
- `Future<ApiResult<void>> clearCart()` — يحذف أو يمسح `clear cart` داخل `CartRepositoryImpl`.
- `Future<ApiResult<List<ProviderOption>>> getChooseProviders()` — يجلب `get choose providers` من مصدر البيانات المناسب داخل `CartRepositoryImpl`.
- `ProviderOption _mapProviderOptionDtoToDomain(ProviderOptionDto dto)` — خاص — يحوّل `map provider option dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.
- `ProviderCoveredService _mapProviderCoveredServiceDtoToDomain(ProviderCoveredServiceDto dto)` — خاص — يحوّل `map provider covered service dto to domain` بين طبقات أو صيغ البيانات داخل `CartRepositoryImpl`.

`features/cart/domain/models/cart.dart`

- `class Cart` — الكلاس الذي يمثل بيانات `cart` وينقلها بين الطبقات.
- `const Cart({required this.items, required this.totalCount, required this.categoryCounters})` — ينشئ كائنًا من `Cart` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const Cart.empty()` — ينشئ كائنًا من `Cart` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Cart removeService(int serviceId)` — يحذف أو يمسح `remove service` داخل `Cart`.
- `Cart removeProvider(int providerId)` — يحذف أو يمسح `remove provider` داخل `Cart`.
- `Cart assignProvider(ProviderOption provider)` — ينفّذ منطق `assign provider` داخل `Cart`.

`features/cart/domain/models/cart_item.dart`

- `class CartItem` — الكلاس الذي يمثل بيانات `cart item` وينقلها بين الطبقات.
- `const CartItem({required this.cartItemId, required this.serviceId, required this.serviceName, required this.categoryId, this.categoryName = '', this.providerId, this.providerName, this.priceAfter, this.priceBefore, this.discountPercentage})` — ينشئ كائنًا من `CartItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `CartItem copyWith({int? providerId, String? providerName, double? priceAfter, double? priceBefore, double? discountPercentage})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

features/cart/domain/models/category_counter.dart

- `class CategoryCounter` — الكلاس الذي يمثل بيانات `category counter` وينقلها بين الطبقات.
- `const CategoryCounter({required this.categoryId, required this.categoryName, required this.count})` — ينشئ كائنًا من `CategoryCounter` ويهيّئ الاعتماديات والقيم المطلوبة.

features/cart/domain/models/category_service.dart

- `class CategoryService` — الكلاس الذي يمثل بيانات `category service` وينقلها بين الطبقات.
- `const CategoryService({required this.serviceId, required this.serviceName, required this.isSelected})` — ينشئ كائنًا من `CategoryService` ويهيّئ الاعتماديات والقيم المطلوبة.

features/cart/domain/models/category_services.dart

- `class CategoryServices` — الكلاس الذي يمثل بيانات `category services` وينقلها بين الطبقات.
- `const CategoryServices({required this.items, this.totalSelectedCount = 0})` — ينشئ كائنًا من `CategoryServices` ويهيّئ الاعتماديات والقيم المطلوبة.

features/cart/domain/models/provider_covered_service.dart

- `class ProviderCoveredService` — الكلاس الذي يمثل بيانات `provider covered service` وينقلها بين الطبقات.
- `const ProviderCoveredService({required this.cartItemId, required this.serviceId, required this.serviceName, this.priceBefore, this.priceAfter, this.discountPercentage})` — ينشئ كائنًا من `ProviderCoveredService` ويهيّئ الاعتماديات والقيم المطلوبة.

features/cart/domain/models/provider_option.dart

- `class ProviderOption` — الكلاس الذي يمثل بيانات `provider option` وينقلها بين الطبقات.
- `const ProviderOption({required this.providerId, required this.providerName, this.providerImageUrl, required this.coveredCount, required this.uncoveredCount, required this.totalCount, required this.totalPriceBefore, required this.totalPriceAfter, required this.coveredServices, required this.uncoveredServices})` — ينشئ كائنًا من `ProviderOption` ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool get coversAll` — ينفّذ منطق `covers all` داخل `ProviderOption`.

features/cart/domain/repository/cart_repository.dart

- `class CartRepository` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات `cart repository`.
- `Future<ApiResult<Cart>> getCart()` — يجلب `get cart` من مصدر البيانات المناسب داخل `CartRepository`.
- `Future<ApiResult<void>> addToCart({required int serviceId, required int categoryId, int? providerId, String? cardNumber, double? price})` — ينشئ أو يضيف `add to cart` داخل `CartRepository`.
- `Future<ApiResult<void>> removeFromCart(int serviceId)` — يحذف أو يمسح `remove from cart` داخل `CartRepository`.
- `Future<ApiResult<void>> updateCartProvider({required int cartId, required int providerId})` — يحدّث `update cart provider` داخل `CartRepository`.
- `Future<ApiResult<CategoryServices>> getCategoryServices(int categoryId)` — يجلب `get category services` من مصدر البيانات المناسب داخل `CartRepository`.
- `Future<ApiResult<void>> clearCart()` — يحذف أو يمسح `clear cart` داخل `CartRepository`.
- `Future<ApiResult<List<ProviderOption>>> getChooseProviders()` — يجلب `get choose providers` من مصدر البيانات المناسب داخل `CartRepository`.

features/cart/domain/services/cart_sync_service.dart

- `class CartSyncEvent` — الكلاس الذي يقدّم خدمة `cart sync event` لباقي التطبيق.
- `const CartSyncEvent({this.serviceId, this.categoryId, this.isSelected, this.cartCleared = false, this.selectedServiceIds, this.categorySelectedCounts, this.refreshCart = true})` — ينشئ كائنًا من `CartSyncEvent` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class CartSyncService` — الكلاس الذي يقدّم خدمة `cart sync service` لباقي التطبيق.
- `Stream<CartSyncEvent> get stream` — ينفّذ منطق `stream` داخل `CartSyncService`.
- `void serviceSelectionChanged({required int serviceId, int? categoryId, required bool isSelected})` — ينفّذ منطق `service selection changed` داخل `CartSyncService`.
- `void cartCleared()` — ينفّذ منطق `cart cleared` داخل `CartSyncService`.
- `void selectionSnapshotChanged({required Iterable<int> selectedServiceIds, Map<int, int>? categorySelectedCounts, bool refreshCart = true})` — يغيّر الاختيار الخاص بـ `selection snapshot changed` داخل `CartSyncService`.
- `void dispose()` — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.

features/cart/presentation/cubit/cart_cubit.dart

- `enum UnknownInvoiceResolution` — يعرّف القيم الثابتة الخاصة بـ `unknown invoice resolution`.
- `class CartCubit` — الكلاس الذي يدير حالة وتفاعلات `cart cubit`.
- `CartCubit(this._repository, this._invoiceRepository, this._familyMembersRepository, this._networkRepository, {required CartSyncService cartSyncService, this.providerId})` — ينشئ كائنًا من `CartCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> close()` — يغلق الـ Cubit أو الـ streams والموارد التابعة له.
- `Future<void> refreshCart()` — يعيد تحميل أو محاولة `refresh cart` داخل `CartCubit`.
- `Future<void> _refreshAfterCurrent()` — خاص — يعيد تحميل أو محاولة `refresh after current` داخل `CartCubit`.
- `Future<void> _performCartRefresh()` — خاص — ينفّذ منطق `perform cart refresh` داخل `CartCubit`.
- `void _broadcastCartSnapshot(Cart cart, {bool refreshCart = false})` — خاص — ينفّذ منطق `broadcast cart snapshot` داخل `CartCubit`.
- `Future<void> loadCategoryServices(int categoryId)` — يحمّل `load category services` ويحدّث الحالة المرتبطة داخل `CartCubit`.
- `Future<void> addToCart({required int serviceId, required int categoryId})` — ينشئ أو يضيف `add to cart` داخل `CartCubit`.
- `Future<void> removeFromCart({required int serviceId, required int categoryId})` — يحذف أو يمسح `remove from cart` داخل `CartCubit`.
- `Future<void> loadCart()` — يحمّل `load cart` ويحدّث الحالة المرتبطة داخل `CartCubit`.
- `Future<ApiResult<Cart>> _getCartWithRetry()` — خاص — يجلب `get cart with retry` من مصدر البيانات المناسب داخل `CartCubit`.
- `Future<void> loadCartAndProviders()` — يحمّل `load cart and providers` ويحدّث الحالة المرتبطة داخل `CartCubit`.
- `Future<void> clearCart()` — يحذف أو يمسح `clear cart` داخل `CartCubit`.
- `Future<void> loadProviders()` — يحمّل `load providers` ويحدّث الحالة المرتبطة داخل `CartCubit`.

- `void selectProvider(ProviderOption provider)` — يغيّر الاختيار الخاص بـ `select` `provider` داخل `CartCubit`.
- `void selectCheckoutBranch(ProviderBranch branch)` — يغيّر الاختيار الخاص بـ `select` `checkout branch` داخل `CartCubit`.
- `void selectFamilyMember(FamilyMemberModel? member)` — يغيّر الاختيار الخاص بـ `select` `family member` داخل `CartCubit`.
- `void selectPaymentMethod(CartPaymentMethod method)` — يغيّر الاختيار الخاص بـ `select` `payment method` داخل `CartCubit`.
- `Future<void> loadCheckout({int? initialProviderId, ProviderBranch? initialBranch, int? packageId, String? packageName, double? packagePriceBefore, double? packagePriceAfter})` — يحمّل `load checkout` ويحدّث الحالة المرتبطة داخل `CartCubit`.
- `Future<ProviderOption?> _resolveCheckoutProvider({required Cart? cart, required int? initialProviderId})` — خاص — ينفّذ منطق `resolve checkout` `provider` داخل `CartCubit`.
- `int? _singleCartProviderId(Cart? cart)` — خاص — ينفّذ منطق `single cart` `provider id` داخل `CartCubit`.
- `Future<ProviderOption?> _freshProviderOption(int providerId)` — خاص — ينفّذ منطق `fresh provider option` داخل `CartCubit`.
- `Future<void> loadProviderBranches(int providerId, {ProviderBranch? initialBranch})` — يحمّل `load provider branches` ويحدّث الحالة المرتبطة داخل `CartCubit`.
- `Future<void> loadFamilyMembers()` — يحمّل `load family members` ويحدّث الحالة المرتبطة داخل `CartCubit`.
- `Future<bool> createCartInvoice()` — ينشئ أو يضيف `create cart invoice` داخل `CartCubit`.
- `Future<UnknownInvoiceResolution> reconcileUnknownInvoiceOutcome()` — ينفّذ منطق `reconcile unknown invoice outcome` داخل `CartCubit`.
- `void _clearUnknownOutcome()` — خاص — يحذف أو يمسح `clear unknown outcome` داخل `CartCubit`.
- `Future<bool> createPackageInvoice()` — ينشئ أو يضيف `create package invoice` داخل `CartCubit`.
- `Future<bool> assignProvider(ProviderOption provider)` — ينفّذ منطق `assign` `provider` داخل `CartCubit`.
- `Future<Cart?> _reconcileAssignedServices(ProviderOption provider, List<ProviderCoveredService> confirmedServices)` — خاص — ينفّذ منطق `reconcile` `assigned services` داخل `CartCubit`.

`features/cart/presentation/cubit/cart_state.dart`

- `enum CartPaymentMethod` — يعرّف القيم الثابتة الخاصة بـ `cart payment method`.
- `class CartState` — الكلاس الذي يدير حالة وتفاعلات `cart state`.
-

```
const CartState({this.cart, this.isCartLoading = false, this.cartError,
        this.categoryServices, this.isCategoryServicesLoading = false,
        this.categoryServicesError, this.pendingServiceIds = const {},
           this.providers = const [], this.isProvidersLoading = false,
               this.providersError, this.isAssigningProvider = false,
 this.assignProviderError, this.selectedProvider, this.checkoutProvider,
               this.checkoutBranches = const [], this.selectedBranch,
this.isCheckoutLoading = false, this.checkoutError, this.familyMembers =
 const [], this.isFamilyMembersLoading = false, this.familyMembersError,
 this.selectedFamilyMember, this.paymentMethod = CartPaymentMethod.cash,
   this.isCreatingInvoice = false, this.isInvoiceOutcomeUnknown = false,
```

this.createInvoiceError, this.createdInvoiceId, this.createdInvoicePaymentLink, this.checkoutPackageId, this.checkoutPackageName, this.checkoutPackagePriceBefore, this.checkoutPackagePriceAfter})](../lib/features/cart/presentation/cubit/cart_state.dart#L47)
— ينشئ كائنًا من `CartState` ويهيّئ الاعتماديات والقيم المطلوبة.

- `CartState copyWith({Cart? cart, bool? isCartLoading, String? cartError, bool resetCartError = false, CategoryServices? categoryServices, bool? isCategoryServicesLoading, String? categoryServicesError, bool resetCategoryServicesError = false, Set<int>? pendingServiceIds, List<ProviderOption>? providers, bool? isProvidersLoading, String? providersError, bool resetProvidersError = false, bool? isAssigningProvider, String? assignProviderError, bool resetAssignProviderError = false, ProviderOption? selectedProvider, bool resetSelectedProvider = false, ProviderOption? checkoutProvider, bool resetCheckoutProvider = false, List<ProviderBranch>? checkoutBranches, ProviderBranch? selectedBranch, bool resetSelectedBranch = false, bool? isCheckoutLoading, String? checkoutError, bool resetCheckoutError = false, List<FamilyMemberModel>? familyMembers, bool? isFamilyMembersLoading, String? familyMembersError, bool resetFamilyMembersError = false, FamilyMemberModel? selectedFamilyMember, bool resetSelectedFamilyMember = false, CartPaymentMethod? paymentMethod, bool? isCreatingInvoice, bool? isInvoiceOutcomeUnknown, String? createInvoiceError, bool resetCreateInvoiceError = false, String? createdInvoiceId, bool resetCreatedInvoiceId = false, String? createdInvoicePaymentLink, bool resetCreatedInvoicePaymentLink = false, int? checkoutPackageId, bool resetCheckoutPackageId = false, String? checkoutPackageName, bool resetCheckoutPackageName = false, double? checkoutPackagePriceBefore, bool resetCheckoutPackagePriceBefore = false, double? checkoutPackagePriceAfter, bool resetCheckoutPackagePriceAfter = false})`
  — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

features/cart/presentation/pages/cart_screen.dart

- `class CartScreen` — الكلاس الذي يمثل شاشة `cart screen`.
- `const CartScreen({super.key, this.initialProviderId, this.initialBranch})`
  — ينشئ كائنًا من `CartScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<CartScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _CartScreenState` — خاص — الكلاس الذي يمثل شاشة `cart screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/pages/checkout_screen.dart

- `class CheckoutScreen` — الكلاس الذي يمثل شاشة `checkout screen`.
- `const CheckoutScreen({super.key, this.initialProviderId, this.initialBranch, this.packageId, this.packageName, this.packagePriceBefore, this.packagePriceAfter})`
  — ينشئ كائنًا من `CheckoutScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<CheckoutScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _CheckoutScreenState` — خاص — الكلاس الذي يمثل شاشة `checkout screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void didChangeAppLifecycleState(AppLifecycleState state)` — خاص — ينفّذ منطق

`did change app lifecycle state` داخل `_CheckoutScreenState`.

- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildBody(BuildContext context, CartState state)` — خاص — ينفّذ منطق `build body` داخل `_CheckoutScreenState`.

features/cart/presentation/widgets/cart_body.dart

- `class CartBody` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CartBody({super.key, required this.items, required this.categoryCounters, this.initialProviderId, this.initialBranch, this.syncError, required this.isBusy})` — ينشئ كائنًا من `CartBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _SummaryStrip` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _SummaryStrip({required this.itemCount, required this.total, required this.isBusy})` — خاص — ينشئ كائنًا من `SummaryStrip` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Future<void> _showClearCartSheet(BuildContext context)` — خاص — يعرض أو يفتح `show clear cart sheet` داخل هذا الملف.
- `class _NeedsProviderBanner` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _NeedsProviderBanner({required this.onTap})` — خاص — ينشئ كائنًا من `NeedsProviderBanner` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ProviderGroupSection` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ProviderGroupSection({required this.providerId, required this.items, this.initialBranch})` — خاص — ينشئ كائنًا من `ProviderGroupSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/cart_empty_view.dart

- `class CartEmptyView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CartEmptyView({super.key, required this.onExplore})` — ينشئ كائنًا من `CartEmptyView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/cart_error_view.dart

- `class CartErrorView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CartErrorView({super.key, required this.message, required this.onRetry})` — ينشئ كائنًا من `CartErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/cart_item_card.dart

- `class CartItemCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CartItemCard({super.key, required this.item, this.showDivider =` —

`true})` — ينشئ كائنًا من `CartItemCard` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _Tag` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _Tag({required this.label, required this.bg, required this.fg})` — خاص — ينشئ كائنًا من `Tag` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _PriceLine` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _PriceLine({required this.item})` — خاص — ينشئ كائنًا من `PriceLine` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/cart_sync_warning.dart

- `class CartSyncWarning` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CartSyncWarning({super.key, required this.message})` — ينشئ كائنًا من `CartSyncWarning` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/category_badges_row.dart

- `class CategoryBadgesRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CategoryBadgesRow({super.key, required this.counters})` — ينشئ كائنًا من `CategoryBadgesRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class CategoryChip` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CategoryChip({super.key, required this.counter})` — ينشئ كائنًا من `CategoryChip` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_branch_selector.dart

- `class CheckoutBranchSelector` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutBranchSelector({super.key, required this.state})` — ينشئ كائنًا من `CheckoutBranchSelector` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_dropdown_frame.dart

- `class CheckoutDropdownFrame` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutDropdownFrame({super.key, required this.child})` — ينشئ كائنًا من `CheckoutDropdownFrame` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_error_view.dart

- `class CheckoutErrorView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutErrorView({super.key, required this.message, required` —

`this.onRetry})` — ينشئ كائنًا من `CheckoutErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_map_preview.dart

- `class CheckoutMapPreview` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutMapPreview({super.key, required this.branch})` — ينشئ كائنًا من `CheckoutMapPreview` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<CheckoutMapPreview> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class CheckoutMapPreviewState` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `LatLng? get _point` — خاص — ينفّذ منطق `point` داخل `CheckoutMapPreviewState`.
- `void didUpdateWidget(covariant CheckoutMapPreview oldWidget)` — يتعامل مع تغيّر إعدادات الـ widget مع الاحتفاظ بالحالة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_patient_dropdown.dart

- `class CheckoutPatientDropdown` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutPatientDropdown({super.key, required this.state})` — ينشئ كائنًا من `CheckoutPatientDropdown` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _memberName(FamilyMemberModel member)` — خاص — ينفّذ منطق `member name` داخل `CheckoutPatientDropdown`.

features/cart/presentation/widgets/checkout_pay_button.dart

- `class CheckoutPayButton` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutPayButton({super.key, required this.state})` — ينشئ كائنًا من `CheckoutPayButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Future<void> _reconcile(BuildContext context)` — خاص — ينفّذ منطق `reconcile` داخل `CheckoutPayButton`.

features/cart/presentation/widgets/checkout_payment_dropdown.dart

- `class CheckoutPaymentDropdown` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutPaymentDropdown({super.key, required this.state})` — ينشئ كائنًا من `CheckoutPaymentDropdown` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_payment_summary.dart

- `class CheckoutPaymentSummary` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutPaymentSummary({super.key, required this.state})` — ينشئ كائنًا من `CheckoutPaymentSummary` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/cart/presentation/widgets/checkout_section_title.dart

- `class CheckoutSectionTitle` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.

- `const CheckoutSectionTitle({super.key, required this.text})` — ينشئ كائنًا من `CheckoutSectionTitle` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`features/cart/presentation/widgets/checkout_summary_row.dart`

- `class CheckoutSummaryRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CheckoutSummaryRow({super.key, required this.label, required this.value, this.valueColor, this.isBold = false})` — ينشئ كائنًا من `CheckoutSummaryRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String checkoutMoney(double value)` — ينفّذ منطق `checkout money` داخل هذا الملف.

`features/cart/presentation/widgets/floating_cart_button.dart`

- `class FloatingCartButton` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const FloatingCartButton({super.key})` — ينشئ كائنًا من `FloatingCartButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<FloatingCartButton> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _FloatingCartButtonState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onLocationChanged()` — خاص — يعالج حدث `on location changed` داخل `_FloatingCartButtonState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Future<void> _refreshCount()` — خاص — يعيد تحميل أو محاولة `refresh count` داخل `_FloatingCartButtonState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: content

`features/content/data/models/content_response_model.dart`

- `class ContentResponseModel` — الكلاس الذي يمثل بيانات `content response model` وينقلها بين الطبقات.
- `const ContentResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ContentResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ContentResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ContentResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ContentData` — الكلاس الذي يمثل بيانات `content data` وينقلها بين الطبقات.
- `const ContentData({this.id, this.description = ''})` — ينشئ كائنًا من `ContentData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ContentData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ContentData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `static int? _idFromJson(Object? value)` — خاص — ينفّذ منطق `id from json` داخل `ContentData`.

- `static String _descriptionFromJson(Object? value)` — خاص — ينفّذ منطق `description from json` داخل `ContentData`.

features/content/data/repositories/content_repository_impl.dart

- `class ContentRepositoryImpl` — الكلاس الذي يمثل بيانات `content repository impl` وينقلها بين الطبقات.
- `ContentRepositoryImpl(this.apiService)` — ينشئ كائنًا من `ContentRepositoryImpl` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ContentData>> getContent(ContentType contentType)` — يجلب `get content` من مصدر البيانات المناسب داخل `ContentRepositoryImpl`.
- `String _messageForFailure(AppFailure failure)` — خاص — ينفّذ منطق `message for failure` داخل `ContentRepositoryImpl`.

features/content/domain/models/content_type.dart

- `enum ContentType` — يعرّف القيم الثابتة الخاصة بـ `content type`.
- `const ContentType({required this.slug, required this.titleKey})` — ينشئ كائنًا من `ContentType` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String get apiPath` — ينفّذ منطق `api path` داخل `ContentType`.

features/content/domain/repositories/content_repository.dart

- `class ContentRepository` — الكلاس الذي يجمع مسؤوليات `content repository`.
- `Future<ApiResult<ContentData>> getContent(ContentType contentType)` — يجلب `get content` من مصدر البيانات المناسب داخل `ContentRepository`.

features/content/domain/use_cases/get_content_use_case.dart

- `class GetContentUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get content use case`.
- `GetContentUseCase(this.repository)` — ينشئ كائنًا من `GetContentUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ContentData>> call(ContentType contentType)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/content/presentation/cubit/content_cubit.dart

- `class ContentCubit` — الكلاس الذي يدير حالة وتفاعلات `content cubit`.
- `ContentCubit(this._getContentUseCase)` — ينشئ كائنًا من `ContentCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadContent(ContentType contentType, {bool force = false})` — يحمّل `load content` ويحدّث الحالة المرتبطة داخل `ContentCubit`.
- `Future<void> retry(ContentType contentType)` — يعيد تحميل أو محاولة `retry` داخل `ContentCubit`.
- `Future<void> _performLoad(ContentType contentType)` — خاص — ينفّذ منطق `perform load` داخل `ContentCubit`.

features/content/presentation/cubit/content_state.dart

- `class ContentState` — الكلاس الذي يدير حالة وتفاعلات `content state`.
- `const factory ContentState.initial()` — ينشئ كائنًا من `ContentState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ContentState.loading()` — ينشئ كائنًا من `ContentState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ContentState.success(ContentData data)` — ينشئ كائنًا من `ContentState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ContentState.failed(String error)` — ينشئ كائنًا من `ContentState`

ويهيّئ الاعتماديات والقيم المطلوبة.

features/content/presentation/pages/content_screen.dart

- `class ContentScreen` — الكلاس الذي يمثل شاشة `content screen`.
- `const ContentScreen({super.key, required this.contentType})` — ينشئ كائنًا من `ContentScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ContentScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ContentScreenState` — خاص — الكلاس الذي يمثل شاشة `content screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _HtmlContentView` — خاص — الكلاس الذي يمثل شاشة `html content view`.
- `const _HtmlContentView({required this.html, required this.isRtl, required this.linkHandler})` — خاص — ينشئ كائنًا من `HtmlContentView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ContentEmptyView` — خاص — الكلاس الذي يمثل شاشة `content empty view`.
- `const _ContentEmptyView({required this.contentType})` — خاص — ينشئ كائنًا من `ContentEmptyView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/content/presentation/utils/content_link_handler.dart

- `enum ContentLinkAction` — يعرّف القيم الثابتة الخاصة بـ `content link action`.
- `class ContentLinkHandler` — الكلاس الذي يجمع مسؤوليات `content link handler`.
- `const ContentLinkHandler(this._launcher)` — ينشئ كائنًا من `ContentLinkHandler` ويهيّئ الاعتماديات والقيم المطلوبة.
- `static ContentLinkAction classifyUrl(String? rawUrl)` — ينفّذ منطق `classify` داخل `ContentLinkHandler`.
- `ContentLinkAction classify(String? rawUrl)` — ينفّذ منطق `classify` داخل `ContentLinkHandler`.
- `Future<bool> handle(BuildContext context, String? rawUrl)` — يعالج حدث `url` داخل `ContentLinkHandler`.

features/content/presentation/widgets/content_error_view.dart

- `class ContentErrorView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ContentErrorView({super.key, required this.onRetry, required this.message})` — ينشئ كائنًا من `ContentErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/content/presentation/widgets/content_shimmer.dart

- `class ContentShimmer` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ContentShimmer({super.key})` — ينشئ كائنًا من `ContentShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: ecard

features/ecard/data/models/card_data_response_model.dart

- `class CardDataResponseModel` — الكلاس الذي يمثل بيانات `card data response model` وينقلها بين الطبقات.
- `const CardDataResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `CardDataResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardDataResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardDataResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CardData` — الكلاس الذي يمثل بيانات `card data` وينقلها بين الطبقات.
- `const CardData({this.cardNumber, this.expireAt, this.firstName, this.lastName, this.planType})` — ينشئ كائنًا من `CardData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CardData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CardData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `String get fullName` — ينفّذ منطق `full name` داخل `CardData`.
- `String get formattedCardNumber` — ينسّق `formatted card number` للعرض أو الإرسال داخل `CardData`.
- `String get formattedExpiry` — ينسّق `formatted expiry` للعرض أو الإرسال داخل `CardData`.

features/ecard/domain/usecases/get_card_data_usecase.dart

- `class GetCardDataUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get card data use case`.
- `GetCardDataUseCase(this._repository)` — ينشئ كائنًا من `GetCardDataUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<CardDataResponseModel>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/ecard/presentation/cubit/ecard_cubit.dart

- `class ECardCubit` — الكلاس الذي يدير حالة وتفاعلات `ecard cubit`.
- `ECardCubit(this._getCardDataUseCase)` — ينشئ كائنًا من `ECardCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadCardData({bool forceRefresh = false})` — يحمّل `load card data` ويحدّث الحالة المرتبطة داخل `ECardCubit`.
- `Future<void> _performLoad()` — خاص — ينفّذ منطق `perform load` داخل `ECardCubit`.

features/ecard/presentation/cubit/ecard_state.dart

- `class ECardState` — الكلاس الذي يدير حالة وتفاعلات `ecard state`.
- `const factory ECardState.initial()` — ينشئ كائنًا من `ECardState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ECardState.loading()` — ينشئ كائنًا من `ECardState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ECardState.loaded(CardData? data)` — ينشئ كائنًا من `ECardState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ECardState.failed(String error)` — ينشئ كائنًا من `ECardState` ويهيّئ الاعتماديات والقيم المطلوبة.

Feature: family_members

features/family_members/data/models/create_family_member_request_model.dart

- `class CreateFamilyMemberRequestModel` — الكلاس الذي يمثل بيانات `create family member request model` وينقلها بين الطبقات.
- `const CreateFamilyMemberRequestModel({this.firstName, this.lastName, this.relation, this.imageUrl})` — ينشئ كائنًا من `CreateFamilyMemberRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CreateFamilyMemberRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CreateFamilyMemberRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/family_members/data/models/family_member_model.dart

- `class FamilyMemberModel` — الكلاس الذي يمثل بيانات `family member model` وينقلها بين الطبقات.
- `const FamilyMemberModel({this.id, this.fullName, this.firstName, this.lastName, this.relation, this.imageUrl})` — ينشئ كائنًا من `FamilyMemberModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory FamilyMemberModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `FamilyMemberModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/family_members/data/models/family_member_operation_response_model.dart

- `class FamilyMemberOperationResponseModel` — الكلاس الذي يمثل بيانات `family member operation response model` وينقلها بين الطبقات.
- `const FamilyMemberOperationResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `FamilyMemberOperationResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory FamilyMemberOperationResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `FamilyMemberOperationResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/family_members/data/models/get_family_member_response_model.dart

- `class GetFamilyMemberResponseModel` — الكلاس الذي يمثل بيانات `get family member response model` وينقلها بين الطبقات.
- `const GetFamilyMemberResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `GetFamilyMemberResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GetFamilyMemberResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GetFamilyMemberResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/family_members/data/models/get_family_members_response_model.dart

- `class GetFamilyMembersResponseModel` — الكلاس الذي يمثل بيانات `get family members response model` وينقلها بين الطبقات.
- `const GetFamilyMembersResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `GetFamilyMembersResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GetFamilyMembersResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GetFamilyMembersResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/family_members/data/repository/family_members_repository_impl.dart

- `class FamilyMembersRepositoryImpl` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات family members repository impl.
- `FamilyMembersRepositoryImpl(this.apiService)` — ينشئ كائنًا من FamilyMembersRepositoryImpl ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<List<FamilyMemberModel>> getFamilyMembers()` — يجلب get family members من مصدر البيانات المناسب داخل FamilyMembersRepositoryImpl.
- `Future<FamilyMemberModel> getFamilyMember(int id)` — يجلب get family member من مصدر البيانات المناسب داخل FamilyMembersRepositoryImpl.
- `Future<void> createFamilyMember(String firstName, String lastName, int relation, String? imageUrl)` — ينشئ أو يضيف create family member داخل FamilyMembersRepositoryImpl.
- `Future<void> updateFamilyMember(int id, String firstName, String lastName, int relation, String? imageUrl)` — يحدّث update family member داخل FamilyMembersRepositoryImpl.
- `Future<void> deleteFamilyMember(int id)` — يحذف أو يمسح delete family member داخل FamilyMembersRepositoryImpl.

features/family_members/domain/repository/family_members_repository.dart

- `class FamilyMembersRepository` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات family members repository.
- `Future<List<FamilyMemberModel>> getFamilyMembers()` — يجلب get family members من مصدر البيانات المناسب داخل FamilyMembersRepository.
- `Future<FamilyMemberModel> getFamilyMember(int id)` — يجلب get family member من مصدر البيانات المناسب داخل FamilyMembersRepository.
- `Future<void> createFamilyMember(String firstName, String lastName, int relation, String? imageUrl)` — ينشئ أو يضيف create family member داخل FamilyMembersRepository.
- `Future<void> updateFamilyMember(int id, String firstName, String lastName, int relation, String? imageUrl)` — يحدّث update family member داخل FamilyMembersRepository.
- `Future<void> deleteFamilyMember(int id)` — يحذف أو يمسح delete family member داخل FamilyMembersRepository.

features/family_members/presentation/cubit/family_members_cubit.dart

- `class FamilyMembersCubit` — الكلاس الذي يدير حالة وتفاعلات family members cubit.
- `FamilyMembersCubit(this.repository)` — ينشئ كائنًا من FamilyMembersCubit ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadFamilyMembers()` — يحمّل load family members ويحدّث الحالة المرتبطة داخل FamilyMembersCubit.
- `Future<void> createFamilyMember({required String firstName, required String lastName, required int relation, String? imageUrl})` — ينشئ أو يضيف create family member داخل FamilyMembersCubit.
- `Future<void> updateFamilyMember({required int id, required String firstName, required String lastName, required int relation, String? imageUrl})` — يحدّث update family member داخل FamilyMembersCubit.
- `Future<void> deleteFamilyMember({required int id})` — يحذف أو يمسح delete family member داخل FamilyMembersCubit.

features/family_members/presentation/cubit/family_members_state.dart

- `class FamilyMembersState` — الكلاس الذي يدير حالة وتفاعلات family members state.
- `const factory FamilyMembersState.initial()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.loading()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.success(List<FamilyMemberModel> members)` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.failed(String error)` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.creating()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.created()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.updating()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.updated()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.deleting()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory FamilyMembersState.deleted()` — ينشئ كائنًا من FamilyMembersState ويهيّئ الاعتماديات والقيم المطلوبة.

features/family_members/presentation/pages/add_edit_family_member_screen.dart

- `class AddEditFamilyMemberScreen` — الكلاس الذي يمثل شاشة add edit family member screen.
- `const AddEditFamilyMemberScreen({super.key, this.member})` — ينشئ كائنًا من AddEditFamilyMemberScreen ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<AddEditFamilyMemberScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AddEditFamilyMemberScreenState` — خاص — الكلاس الذي يمثل شاشة add edit family member screen state.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _submit()` — خاص — يرسل submit إلى الطبقة أو الخدمة المطلوبة داخل _AddEditFamilyMemberScreenState.
- `void _showDeleteDialog()` — خاص — يعرض أو يفتح show delete dialog داخل _AddEditFamilyMemberScreenState.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildTextField({required TextEditingController controller, required String hint, String? Function(String?)? validator, List<TextInputFormatter>? inputFormatters, String? focusKey})` — خاص — ينفّذ منطق build text field داخل _AddEditFamilyMemberScreenState.
- `String? _validateName(String? value)` — خاص — يتحقق من صحة validate name داخل _AddEditFamilyMemberScreenState.
- `Widget _buildRelationDropdown()` — خاص — ينفّذ منطق build relation dropdown داخل _AddEditFamilyMemberScreenState.

- `class _FieldLabel` — خاص — الكلاس الذي يمثل شاشة `field label`.
- `const _FieldLabel(this.label)` — خاص — ينشئ كائنًا من `FieldLabel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/family_members/presentation/pages/family_members_list_screen.dart

- `class FamilyMembersListScreen` — الكلاس الذي يمثل شاشة `family members list` `screen`.
- `const FamilyMembersListScreen({super.key})` — ينشئ كائنًا من `FamilyMembersListScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<FamilyMembersListScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _FamilyMembersListScreenState` — خاص — الكلاس الذي يمثل شاشة `family members` `list screen state`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildEmptyState(BuildContext context)` — خاص — ينفّذ منطق `build empty` `state` داخل `_FamilyMembersListScreenState`.
- `class _FamilyMemberListItem` — خاص — الكلاس الذي يمثل شاشة `family member list` `item`.
- `const _FamilyMemberListItem({required this.member, this.showDivider = true})` — خاص — ينشئ كائنًا من `FamilyMemberListItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `void _showDeleteDialog(BuildContext context, FamilyMemberModel member)` — خاص — يعرض أو يفتح `show delete dialog` داخل `_FamilyMemberListItem`.
- `Widget _buildAvatar(String? imageUrl)` — خاص — ينفّذ منطق `build avatar` داخل `_FamilyMemberListItem`.
- `Widget _buildPlaceholderAvatar()` — خاص — ينفّذ منطق `build placeholder avatar` داخل `_FamilyMemberListItem`.
- `class _CircleIconButton` — خاص — الكلاس الذي يمثل شاشة `circle icon button`.
- `const _CircleIconButton({required this.icon, required this.foreground, required this.background, required this.onTap})` — خاص — ينشئ كائنًا من `CircleIconButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: guest_home

features/guest_home/data/models/guest_home_response_model.dart

- `class GuestHomeResponseModel` — الكلاس الذي يمثل بيانات `guest home response model` وينقلها بين الطبقات.
- `const GuestHomeResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `GuestHomeResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestHomeResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestHomeResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class GuestHomeDataModel` — الكلاس الذي يمثل بيانات `guest home data model` وينقلها بين الطبقات.
- `const GuestHomeDataModel({this.sliders, this.categories, this.nearestProviders, this.packages})` — ينشئ كائنًا من `GuestHomeDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestHomeDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestHomeDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class GuestSlider` — الكلاس الذي يمثل بيانات `guest slider` وينقلها بين الطبقات.
- `const GuestSlider({this.id, this.imageUrl, this.providerId, this.providerName})` — ينشئ كائنًا من `GuestSlider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestSlider.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestSlider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class GuestCategory` — الكلاس الذي يمثل بيانات `guest category` وينقلها بين الطبقات.
- `const GuestCategory({this.id, this.name, this.imageUrl})` — ينشئ كائنًا من `GuestCategory` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestCategory.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestCategory` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class GuestNearestProvider` — الكلاس الذي يمثل بيانات `guest nearest provider` وينقلها بين الطبقات.
- `const GuestNearestProvider({this.providerId, this.providerBranchId, this.providerName, this.providerImageUrl, this.branchName, this.fullAddress, this.mapUrl, this.distanceKm})` — ينشئ كائنًا من `GuestNearestProvider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestNearestProvider.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestNearestProvider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class GuestPackage` — الكلاس الذي يمثل بيانات `guest package` وينقلها بين الطبقات.
- `const GuestPackage({this.packageId, this.providerId, this.providerName, this.name, this.imageUrl, this.description, this.priceBefore, this.priceAfter, this.discountPercentage, this.servicesCount})` — ينشئ كائنًا من `GuestPackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestPackage.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestPackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/guest_home/domain/usecases/get_guest_home_data_usecase.dart`

- `class GetGuestHomeDataUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get guest home data use case`.
- `GetGuestHomeDataUseCase(this._repository)` — ينشئ كائنًا من `GetGuestHomeDataUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<GuestHomeResponseModel>> call({double? latitude, double? longitude})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`features/guest_home/presentation/cubit/guest_home_cubit.dart`

- `class GuestHomeCubit` — الكلاس الذي يدير حالة وتفاعلات `guest home cubit`.

- `GuestHomeCubit(this._getGuestHomeDataUseCase, this._locationRepository)` — ينشئ كائنًا من `GuestHomeCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> getGuestHomeData()` — يجلب `get guest home data` من مصدر البيانات المناسب داخل `GuestHomeCubit`.

features/guest_home/presentation/cubit/guest_home_state.dart

- `class GuestHomeState` — الكلاس الذي يدير حالة وتفاعلات `guest home state`.
- `const factory GuestHomeState.initial()` — ينشئ كائنًا من `GuestHomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory GuestHomeState.loading()` — ينشئ كائنًا من `GuestHomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory GuestHomeState.success(GuestHomeResponseModel data)` — ينشئ كائنًا من `GuestHomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory GuestHomeState.failed(String error)` — ينشئ كائنًا من `GuestHomeState` ويهيّئ الاعتماديات والقيم المطلوبة.

features/guest_home/presentation/mappers/guest_home_mappers.dart

- `List<ProviderModel> mapGuestProviders(List<GuestNearestProvider> raw)` — يحوّل `map guest providers` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `List<PromoBanner> mapGuestPromoBanners(List<GuestSlider> raw)` — يحوّل `map guest promo banners` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `List<FeaturedServiceModel> mapGuestPackages(List<GuestPackage> raw)` — يحوّل `map guest packages` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `bool _hasAnyText(List<String?> values)` — خاص — يفحص شرط `has any text` ويعيد النتيجة داخل هذا الملف.
- `bool _hasText(String? value)` — خاص — يفحص شرط `has text` ويعيد النتيجة داخل هذا الملف.
- `String _textOrEmpty(String? value)` — خاص — ينفّذ منطق `text or empty` داخل هذا الملف.

features/guest_home/presentation/pages/guest_home_screen.dart

- `class GuestHomeScreen` — الكلاس الذي يمثل شاشة `guest home screen`.
- `const GuestHomeScreen({super.key})` — ينشئ كائنًا من `GuestHomeScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<GuestHomeScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _GuestHomeScreenState` — خاص — الكلاس الذي يمثل شاشة `guest home screen` `state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildShellScrollView({required Widget child, List<PromoBanner> banners = const []})`](../lib/features/guest_home/presentation/pages/guest_home_screen.dart#L95) — خاص — ينفّذ منطق `build shell scroll view` داخل `_GuestHomeScreenState`.
- `class _GuestHomeHeader` — خاص — الكلاس الذي يمثل شاشة `guest home header`.
- `const _GuestHomeHeader()` — خاص — ينشئ كائنًا من `GuestHomeHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _GuestHomeSuccessContent` — خاص — الكلاس الذي يمثل شاشة `guest home success` `content`.

- `const _GuestHomeSuccessContent({required this.providers, required this.packages})` — خاص — ينشئ كائنًا من `GuestHomeSuccessContent` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: help_center

features/help_center/presentation/pages/help_center_screen.dart

- `class HelpCenterScreen` — الكلاس الذي يمثل شاشة `help center screen`.
- `const HelpCenterScreen({super.key})` — ينشئ كائنًا من `HelpCenterScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ContactTile` — خاص — الكلاس الذي يمثل شاشة `contact tile`.
- `const _ContactTile({required this.icon, required this.iconColor, required this.bgColor, required this.label, required this.value, this.showDivider = true, required this.onTap})` — خاص — ينشئ كائنًا من `ContactTile` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: home

features/home/data/home_mock_data.dart

- `class QuickAction` — الكلاس الذي يمثل بيانات `quick action` وينقلها بين الطبقات.
- `const QuickAction({required this.labelKey, required this.assetPath})` — ينشئ كائنًا من `QuickAction` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class PromoBanner` — الكلاس الذي يمثل بيانات `promo banner` وينقلها بين الطبقات.
- `const PromoBanner({this.providerId, required this.imageUrl, required this.providerLogoUrl, required this.title, required this.subtitle})` — ينشئ كائنًا من `PromoBanner` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class HomeMockData` — الكلاس الذي يمثل بيانات `home mock data` وينقلها بين الطبقات.

features/home/data/models/home_response_model.dart

- `class HomeResponseModel` — الكلاس الذي يمثل بيانات `home response model` وينقلها بين الطبقات.
- `const HomeResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `HomeResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory HomeResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `HomeResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `List<String>? _stringList(dynamic value)` — خاص — ينفّذ منطق `string list` داخل هذا الملف.
- `String? _stringValue(dynamic value)` — خاص — ينفّذ منطق `string value` داخل هذا الملف.
- `int? _intValue(dynamic value)` — خاص — ينفّذ منطق `int value` داخل هذا الملف.
- `double? _doubleValue(dynamic value)` — خاص — ينفّذ منطق `double value` داخل هذا الملف.
- `class HomeDataModel` — الكلاس الذي يمثل بيانات `home data model` وينقلها بين الطبقات.

- `const HomeDataModel({this.sliders, this.pendingAppointments, this.nearestProviders, this.packages})` — ينشئ كائنًا من `HomeDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory HomeDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `HomeDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class HomeSlider` — الكلاس الذي يمثل بيانات `home slider` وينقلها بين الطبقات.
- `const HomeSlider({this.id, this.imageUrl, this.providerId, this.providerName})` — ينشئ كائنًا من `HomeSlider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory HomeSlider.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `HomeSlider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class HomePendingAppointment` — الكلاس الذي يمثل بيانات `home pending appointment` وينقلها بين الطبقات.
- `const HomePendingAppointment({this.invoiceId, this.providerId, this.providerBranchId, this.providerName, this.providerImageUrl, this.branchName, this.service, this.status, this.date})` — ينشئ كائنًا من `HomePendingAppointment` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory HomePendingAppointment.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `HomePendingAppointment` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class HomeNearestProvider` — الكلاس الذي يمثل بيانات `home nearest provider` وينقلها بين الطبقات.
- `const HomeNearestProvider({this.providerId, this.providerBranchId, this.providerName, this.providerImageUrl, this.branchName, this.fullAddress, this.mapUrl, this.distanceKm})` — ينشئ كائنًا من `HomeNearestProvider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory HomeNearestProvider.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `HomeNearestProvider` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class HomePackage` — الكلاس الذي يمثل بيانات `home package` وينقلها بين الطبقات.
- `const HomePackage({this.packageId, this.providerId, this.providerBranchId, this.providerName, this.name, this.imageUrl, this.description, this.priceBefore, this.priceAfter, this.discountPercentage, this.servicesCount})` — ينشئ كائنًا من `HomePackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory HomePackage.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `HomePackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/home/domain/models/appointment_model.dart`

- `class AppointmentModel` — الكلاس الذي يمثل بيانات `appointment model` وينقلها بين الطبقات.
- `const AppointmentModel({this.invoiceId, this.providerId, required this.providerLogoUrl, required this.providerName, required this.service, required this.branch, required this.status, required this.date})` — ينشئ كائنًا من `AppointmentModel` ويهيّئ الاعتماديات والقيم المطلوبة.

`features/home/domain/models/featured_service_model.dart`

- `class FeaturedServiceModel` — الكلاس الذي يمثل بيانات `featured service model` وينقلها بين الطبقات.

- `const FfeaturedServiceModel({this.packageId, this.providerId, this.providerBranchId, required this.imageUrl, required this.providerLogoUrl, required this.providerName, required this.serviceName, required this.location, required this.originalPrice, required this.discountedPrice, required this.rating, required this.reviewCount})` — ينشئ كائنًا من `FfeaturedServiceModel` ويهيّئ الاعتماديات والقيم المطلوبة.

features/home/domain/models/home_failure.dart

- `enum HomeFailureType` — يعرّف القيم الثابتة الخاصة بـ `home failure type`.
- `extension HomeFailureTypeCode` — يضيف وظائف مساعدة إلى نوع موجود دون تعديل تعريفه.
- `String get code` — ينفّذ منطق `code` داخل `HomeFailureTypeCode`.
- `HomeFailureType homeFailureFromCode(String code)` — ينفّذ منطق `home failure from code` داخل هذا الملف.
- `HomeFailureType homeFailureFromAppFailure(AppFailure failure)` — ينفّذ منطق `home failure from app failure` داخل هذا الملف.

features/home/domain/models/provider_model.dart

- `class ProviderModel` — الكلاس الذي يمثل بيانات `provider model` وينقلها بين الطبقات.
- `const ProviderModel({this.providerId, required this.imageUrl, required this.name, required this.rating, required this.reviewCount, required this.type, required this.location, this.badges = const [], this.mapUrl = '', this.distanceKm})`](../lib/features/home/domain/models/provider_model.dart#L13) — ينشئ كائنًا من `ProviderModel` ويهيّئ الاعتماديات والقيم المطلوبة.

features/home/domain/usecases/get_home_data_usecase.dart

- `class GetHomeDataUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get home data use case`.
- `GetHomeDataUseCase(this._repository)` — ينشئ كائنًا من `GetHomeDataUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<HomeResponseModel>> call({double? latitude, double? longitude})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/home/presentation/cubit/home_cubit.dart

- `class HomeCubit` — الكلاس الذي يدير حالة وتفاعلات `home cubit`.
- `HomeCubit(this._getHomeDataUseCase, this._locationRepository, this._locationService)` — ينشئ كائنًا من `HomeCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> getHomeData({bool refresh = false})` — يجلب `get home data` من مصدر البيانات المناسب داخل `HomeCubit`.
- `Future<void> _performHomeLoad({required bool refresh})` — خاص — ينفّذ منطق `perform home load` داخل `HomeCubit`.

features/home/presentation/cubit/home_state.dart

- `class HomeState` — الكلاس الذي يدير حالة وتفاعلات `home state`.
- `const factory HomeState.initial()` — ينشئ كائنًا من `HomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory HomeState.loading()` — ينشئ كائنًا من `HomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory HomeState.success(HomeResponseModel data)` — ينشئ كائنًا من `HomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory HomeState.failed(HomeFailureType failure, {@Default(false) bool isRetrying})` — ينشئ كائنًا من `HomeState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory HomeState.refreshing(HomeResponseModel data)` — ينشئ كائنًا من

`HomeState` ويهيّئ الاعتماديات والقيم المطلوبة.

- `const factory HomeState.refreshFailed(HomeResponseModel data, HomeFailureType failure)` — ينشئ كائنًا من `HomeState` ويهيّئ الاعتماديات والقيم المطلوبة.

features/home/presentation/cubit/location_cubit.dart

- `class LocationCubit` — الكلاس الذي يدير حالة وتفاعلات `location cubit`.
- `LocationCubit(this._locationRepository, this._locationService)` — ينشئ كائنًا من `LocationCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadLocation()` — يحمّل `load location` ويحدّث الحالة المرتبطة داخل `LocationCubit`.
- `Future<void> refreshLocation()` — يعيد تحميل أو محاولة `refresh location` داخل `LocationCubit`.
- `Future<void> retryLocation()` — يعيد تحميل أو محاولة `retry location` داخل `LocationCubit`.
- `Future<void> _resolveLocationName({required bool clearCache})` — خاص — ينفّذ منطق `resolve location name` داخل `LocationCubit`.
- `String? _locationNameFromPlacemarks(List<Placemark> placemarks)` — خاص — ينفّذ منطق `location name from placemarks` داخل `LocationCubit`.
- `String _fallbackLocationName(double latitude, double longitude)` — خاص — ينفّذ منطق `fallback location name` داخل `LocationCubit`.

features/home/presentation/cubit/location_state.dart

- `class LocationState` — الكلاس الذي يدير حالة وتفاعلات `location state`.
- `const LocationState()` — ينشئ كائنًا من `LocationState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `List<Object?> get props` — ينفّذ منطق `props` داخل `LocationState`.
- `class LocationInitial` — الكلاس الذي يدير حالة وتفاعلات `location initial`.
- `const LocationInitial()` — ينشئ كائنًا من `LocationInitial` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class LocationLoading` — الكلاس الذي يدير حالة وتفاعلات `location loading`.
- `const LocationLoading()` — ينشئ كائنًا من `LocationLoading` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class LocationLoaded` — الكلاس الذي يدير حالة وتفاعلات `location loaded`.
- `const LocationLoaded({required this.name, required this.latitude, required this.longitude})` — ينشئ كائنًا من `LocationLoaded` ويهيّئ الاعتماديات والقيم المطلوبة.
- `List<Object?> get props` — ينفّذ منطق `props` داخل `LocationLoaded`.
- `class LocationError` — الكلاس الذي يدير حالة وتفاعلات `location error`.
- `const LocationError(this.message)` — ينشئ كائنًا من `LocationError` ويهيّئ الاعتماديات والقيم المطلوبة.
- `List<Object?> get props` — ينفّذ منطق `props` داخل `LocationError`.

features/home/presentation/mappers/home_mappers.dart

- `List<ProviderModel> mapProviders(List<HomeNearestProvider> raw)` — يحوّل `map providers` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `List<AppointmentModel> mapAppointments(List<HomePendingAppointment> raw)` — يحوّل `map appointments` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `List<PromoBanner> mapPromoBanners(List<HomeSlider> raw)` — يحوّل `map promo banners` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `List<FeaturedServiceModel> mapPackages(List<HomePackage> raw)` — يحوّل `map packages` بين طبقات أو صيغ البيانات داخل هذا الملف.
- `bool _hasAnyText(List<String?> values)` — خاص — يفحص شرط `has any text` ويعيد

النتيجة داخل هذا الملف.

- `bool _hasText(String? value)` — خاص — يفحص شرط `has text` ويعيد النتيجة داخل هذا الملف.
- `String _textOrEmpty(String? value)` — خاص — ينفّذ منطق `text or empty` داخل هذا الملف.
- `String _appointmentStatusText(String? status)` — خاص — ينفّذ منطق `appointment status text` داخل هذا الملف.

features/home/presentation/pages/home_screen.dart

- `class HomeScreen` — الكلاس الذي يمثل شاشة `home screen`.
- `const HomeScreen({super.key, this.initialIndex = 0})` — ينشئ كائنًا من `HomeScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<HomeScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _HomeScreenState` — خاص — الكلاس الذي يمثل شاشة `home screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _HomeBody` — خاص — الكلاس الذي يمثل شاشة `home body`.
- `const _HomeBody({required this.onNavigateToTab})` — خاص — ينشئ كائنًا من `HomeBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `void _openAllProviders(BuildContext context)` — خاص — يعرض أو يفتح `open all providers` داخل `_HomeBody`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildLoadedView(BuildContext context, HomeResponseModel data, {bool isRefreshing = false})` — خاص — ينفّذ منطق `build loaded view` داخل `_HomeBody`.
- `Widget _buildShellScrollView({required List<Widget> slivers, List<PromoBanner> banners = const [], bool isRefreshing = false})`](../lib/features/home/presentation/pages/home_screen.dart#L289) — خاص — ينفّذ منطق `build shell scroll view` داخل `_HomeBody`.
- `class _ProfileBody` — خاص — الكلاس الذي يمثل شاشة `profile body`.
- `const _ProfileBody()` — خاص — ينشئ كائنًا من `ProfileBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_ProfileBody> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ProfileBodyState` — خاص — الكلاس الذي يمثل شاشة `profile body state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onProfileSummaryRefresh()` — خاص — يعالج حدث `on profile summary refresh` داخل `_ProfileBodyState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/activity/activity_empty_view.dart

- `class ActivityEmptyView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ActivityEmptyView({super.key, required this.title, required this.message, required this.icon})` — ينشئ كائنًا من `ActivityEmptyView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/activity/activity_error_view.dart

- `class ActivityErrorView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ActivityErrorView({super.key, required this.error, required this.onRetry})` — ينشئ كائنًا من `ActivityErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/activity/invoice_action_button.dart

- `class InvoiceActionButton` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const InvoiceActionButton({super.key, required this.item})` — ينشئ كائنًا من `InvoiceActionButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/activity/invoice_card.dart

- `class InvoiceCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const InvoiceCard({super.key, required this.item})` — ينشئ كائنًا من `InvoiceCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _providerInitial()` — خاص — ينفّذ منطق `provider initial` داخل `InvoiceCard`.
- `double? _amount(InvoiceItem item)` — خاص — ينفّذ منطق `amount` داخل هذا الملف.

features/home/presentation/widgets/activity/invoice_list.dart

- `class InvoiceList` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const InvoiceList({super.key, required this.items, required this.emptyTitleKey, required this.emptyMessageKey, required this.emptyIcon, required this.isLoadingMore, required this.hasMore})` — ينشئ كائنًا من `InvoiceList` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<InvoiceList> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class InvoiceListState` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `void initState()` — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void didUpdateWidget(covariant InvoiceList oldWidget)` — يتعامل مع تغيّر إعدادات الـ widget مع الاحتفاظ بالحالة.
- `void dispose()` — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onScroll()` — خاص — يعالج حدث `on scroll` داخل `InvoiceListState`.
- `void _maybePrefetchMore()` — خاص — ينفّذ منطق `maybe prefetch more` داخل `InvoiceListState`.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class TabScope` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const TabScope({super.key, required this.tab, required super.child})` — ينشئ كائنًا من `TabScope` ويهيّئ الاعتماديات والقيم المطلوبة.
- `static int of(BuildContext context)` — ينفّذ منطق `of` داخل `TabScope`.
- `bool updateShouldNotify(TabScope oldWidget)` — يحدّث `update should notify` داخل `TabScope`.

features/home/presentation/widgets/activity_screen.dart

- `class ActivityScreen` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.

- `const ActivityScreen({super.key})` — ينشئ كائنًا من `ActivityScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ActivityScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ActivityScreenState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onInvoicesRefresh()` — خاص — يعالج حدث `on invoices refresh` داخل `_ActivityScreenState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onTabChanged()` — خاص — يعالج حدث `on tab changed` داخل `_ActivityScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`features/home/presentation/widgets/ecard_preview_card.dart`

- `class ECardPreviewCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ECardPreviewCard({super.key})` — ينشئ كائنًا من `ECardPreviewCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ECardPreviewCard> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ECardPreviewCardState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _MediCardPreview` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _MediCardPreview({required this.card})` — خاص — ينشئ كائنًا من `MediCardPreview` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`features/home/presentation/widgets/featured_service_card.dart`

- `class FeaturedServiceCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const FeaturedServiceCard({super.key, required this.service})` — ينشئ كائنًا من `FeaturedServiceCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<FeaturedServiceCard> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _FeaturedServiceCardState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `FeaturedServiceModel get service` — خاص — ينفّذ منطق `service` داخل `_FeaturedServiceCardState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Future<void> _openPackageDetails(BuildContext cardContext)` — خاص — يعرض أو يفتح `open package details` داخل `_FeaturedServiceCardState`.
- `Future<void> _bookPackage(BuildContext context)` — خاص — ينفّذ منطق `book package` داخل `_FeaturedServiceCardState`.
- `class _PackageDetailsSheet` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _PackageDetailsSheet({required this.packageId, required this.fallback, required this.onBook})` — خاص — ينشئ كائنًا من `PackageDetailsSheet` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_PackageDetailsSheet> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ

.StatefulWidget

- `class _PackageDetailsSheetState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _PackageDetailsSheetContent` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _PackageDetailsSheetContent({required this.details, required this.fallback, required this.onBook})` — خاص — ينشئ كائنًا من `PackageDetailsSheetContent` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _imageUrl(String value)` — خاص — ينفّذ منطق `image url` داخل `_PackageDetailsSheetContent`.

features/home/presentation/widgets/featured_services_section.dart

- `class FeaturedServicesSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const FeaturedServicesSection({super.key, required this.services, this.onViewAll})` — ينشئ كائنًا من `FeaturedServicesSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/home_bottom_nav.dart

- `class HomeBottomNav` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const HomeBottomNav({super.key, required this.currentIndex, required this.onTap})` — ينشئ كائنًا من `HomeBottomNav` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _NavItem` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _NavItem({required this.icon, required this.labelKey})` — خاص — ينشئ كائنًا من `NavItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class _NavColumn` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _NavColumn({required this.item, required this.isSelected, required this.onTap})` — خاص — ينشئ كائنًا من `NavColumn` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/home_empty_view.dart

- `class HomeEmptyView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const HomeEmptyView({super.key})` — ينشئ كائنًا من `HomeEmptyView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/home_error_view.dart

- `class HomeErrorView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const HomeErrorView({super.key, this.failure = HomeFailureType.unknown, this.message, required this.onRetry, this.isRetrying = false})` — ينشئ كائنًا من `HomeErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات

الحالية.

features/home/presentation/widgets/home_header.dart

- `class HomeHeader` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const HomeHeader({super.key})` — ينشئ كائنًا من `HomeHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _HeaderTopRow` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _HeaderTopRow()` — خاص — ينشئ كائنًا من `HeaderTopRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _locationText(LocationState state, BuildContext context)` — خاص — ينفّذ منطق `location text` داخل `_HeaderTopRow`.
- `class _HeaderSearchBar` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _HeaderSearchBar()` — خاص — ينشئ كائنًا من `HeaderSearchBar` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/home_section_header.dart

- `class HomeSectionHeader` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const HomeSectionHeader({super.key, required this.title, this.onViewAll})` — ينشئ كائنًا من `HomeSectionHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/home_shimmer.dart

- `class HomeShimmer` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const HomeShimmer({super.key})` — ينشئ كائنًا من `HomeShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _SectionTitleShimmer` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/location_display.dart

- `class LocationDisplay` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const LocationDisplay({super.key, required this.text, required this.onTap})` — ينشئ كائنًا من `LocationDisplay` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/nearest_provider_card.dart

- `class NearestProviderCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const NearestProviderCard({super.key, required this.provider})` — ينشئ كائنًا من `NearestProviderCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/nearest_section.dart

- `class NearestSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const NearestSection({super.key, required this.providers, this.onViewAll})` — ينشئ كائنًا من `NearestSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/order_placed_dialog.dart

- `class OrderPlacedDialog` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `static Future<void> maybeShow(BuildContext context, {required String? orderNumber})` — ينفّذ منطق `maybe show` داخل `OrderPlacedDialog`.
- `class _OrderPlacedDialog` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _OrderPlacedDialog({required this.orderNumber})` — خاص — ينشئ كائنًا من `OrderPlacedDialog` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/pending_appointment_card.dart

- `class PendingAppointmentCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PendingAppointmentCard({super.key, required this.appointment})` — ينشئ كائنًا من `PendingAppointmentCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _StatusBadge` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _StatusBadge({required this.status})` — خاص — ينشئ كائنًا من `StatusBadge` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _localizedStatus(String status)` — خاص — ينفّذ منطق `localized status` داخل `_StatusBadge`.

features/home/presentation/widgets/pending_appointment_details_screen.dart

- `class PendingAppointmentDetailsScreen` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PendingAppointmentDetailsScreen({super.key, required this.appointment})` — ينشئ كائنًا من `PendingAppointmentDetailsScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildProviderAvatar()` — خاص — ينفّذ منطق `build provider avatar` داخل `PendingAppointmentDetailsScreen`.
- `class _InfoRow` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _InfoRow({required this.label, required this.value})` — خاص — ينشئ كائنًا من `InfoRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _StatusBadge` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _StatusBadge({required this.status})` — خاص — ينشئ كائنًا من `StatusBadge` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة

والمدخلات الحالية.

- `String _localizedStatus(String status)` — خاص — ينفّذ منطق `localized status` داخل `_StatusBadge`.

features/home/presentation/widgets/pending_appointments_section.dart

- `class PendingAppointmentsSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PendingAppointmentsSection({super.key, required this.appointments, this.onViewAll})` — ينشئ كائنًا من `PendingAppointmentsSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/promo_banner.dart

- `class PromoBannerWidget` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PromoBannerWidget({super.key, required this.banners})` — ينشئ كائنًا من `PromoBannerWidget` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<PromoBannerWidget> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _PromoBannerWidgetState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _startAutoScroll()` — خاص — ينفّذ منطق `start auto scroll` داخل `_PromoBannerWidgetState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/quick_actions_row.dart

- `class QuickActionsRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const QuickActionsRow({super.key, required this.actions})` — ينشئ كائنًا من `QuickActionsRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `void _onActionTap(BuildContext context, QuickAction action)` — خاص — يعالج حدث `on action tap` داخل `QuickActionsRow`.

features/home/presentation/widgets/search_bar_widget.dart

- `class SearchBarWidget` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const SearchBarWidget({super.key, this.latitude, this.longitude})` — ينشئ كائنًا من `SearchBarWidget` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/home/presentation/widgets/sort_filter.dart

- `enum SortOption` — يعرّف القيم الثابتة الخاصة بـ `sort option`.
- `class SortFilterSheet` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const SortFilterSheet({super.key})` — ينشئ كائنًا من `SortFilterSheet` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<SortFilterSheet> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _SortFilterSheetState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة

والمدخلات الحالية.

- `Widget _buildCheckItem(String label, SortOption option)` — خاص — ينفّذ منطق `build check item` داخل `_SortFilterSheetState`.
- `Widget _buildDropdown({required String hint, required String? value, required List<String> items, required ValueChanged<String?> onChanged})` — خاص — ينفّذ منطق `build dropdown` داخل `_SortFilterSheetState`.

features/home/presentation/widgets/summer_promo_dialog.dart

- `class SummerPromoDialog` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `static bool get _isPromoActive` — خاص — يفحص شرط `is promo active` ويعيد النتيجة داخل `SummerPromoDialog`.
- `static Future<void> maybeShow(BuildContext context)` — ينفّذ منطق `maybe show` داخل `SummerPromoDialog`.
- `class _SummerPromoDialog` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _SummerPromoDialog()` — خاص — ينشئ كائنًا من `SummerPromoDialog` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: notifications

features/notifications/data/domain/notification_repo.dart

- `class NotificationRepository` — الكلاس الذي يمثل بيانات `notification repository` وينقلها بين الطبقات.
- `NotificationRepository(this._service)` — ينشئ كائنًا من `NotificationRepository` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<String> get _bearerToken` — خاص — ينفّذ منطق `bearer token` داخل `NotificationRepository`.
- `Future<NotificationData> getNotifications(int pageNumber, int pageSize)` — يجلب `get notifications` من مصدر البيانات المناسب داخل `NotificationRepository`.
- `Future<void> markAsRead(int id)` — يغيّر علامة أو حالة `mark as read` داخل `NotificationRepository`.
- `Future<void> markAllAsRead()` — يغيّر علامة أو حالة `mark all as read` داخل `NotificationRepository`.
- `Future<bool> registerDeviceToken({required String fcmToken, required int platform})` — يسجّل `register device token` لدى النظام أو الخدمة داخل `NotificationRepository`.

features/notifications/data/domain/notification_service.dart

- `class NotificationService` — الكلاس الذي يمثل بيانات `notification service` وينقلها بين الطبقات.
- `NotificationService(this._client)` — ينشئ كائنًا من `NotificationService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<NotificationData> getNotifications(String token, int pageNumber, int pageSize)` — يجلب `get notifications` من مصدر البيانات المناسب داخل `NotificationService`.
- `Future<void> markAsRead(String token, int id)` — يغيّر علامة أو حالة `mark as read` داخل `NotificationService`.
- `Future<void> markAllAsRead(String token)` — يغيّر علامة أو حالة `mark all as read`

داخل NotificationService.

- `Future<void> registerDeviceToken(String token, {required String fcmToken, required int platform})` — يسجّل register device token لدى النظام أو الخدمة داخل NotificationService.
- `String _short(String token)` — خاص — ينفّذ منطق short داخل هذا الملف.

features/notifications/data/domain/use_cases/get_notifications_use_case.dart

- `class GetNotificationsUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: get notifications use case.
- `GetNotificationsUseCase(this._repo)` — ينشئ كائنًا من GetNotificationsUseCase ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<List<NotificationModel>>> call({required int pageNumber, required int pageSize})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/notifications/data/domain/use_cases/mark_all_notifications_read_use_case.dart

- `class MarkAllNotificationsReadUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: mark all notifications read use case.
- `MarkAllNotificationsReadUseCase(this._repo)` — ينشئ كائنًا من MarkAllNotificationsReadUseCase ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<void>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/notifications/data/domain/use_cases/mark_notification_read_use_case.dart

- `class MarkNotificationReadUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: mark notification read use case.
- `MarkNotificationReadUseCase(this._repo)` — ينشئ كائنًا من MarkNotificationReadUseCase ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<void>> call(int id)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/notifications/data/domain/use_cases/notification_error_message.dart

- `String notificationErrorMessage(Object error)` — ينفّذ منطق notification error message داخل هذا الملف.
- `String? _responseMessage(Object? data)` — خاص — ينفّذ منطق response message داخل هذا الملف.
- `String _failureMessage(AppFailure failure)` — خاص — ينفّذ منطق failure message داخل هذا الملف.

features/notifications/data/model/api_response.dart

- `class ApiResponse` — الكلاس الذي يمثل بيانات api response وينقلها بين الطبقات.
- `ApiResponse({required this.httpStatusCode, this.errors, required this.succeeded, required this.message, this.data})` — ينشئ كائنًا من ApiResponse ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ApiResponse.fromJson(Map<String, dynamic> json, T Function(Object? json) fromJsonT)` — ينشئ كائنًا من ApiResponse ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson(Object? Function(T value) toJsonT)` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/notifications/data/model/notification_model.dart

- `class NotificationModel` — الكلاس الذي يمثل بيانات notification model وينقلها بين الطبقات.
- `NotificationModel({required this.id, required this.title, required`

`this.body, required this.imageUrl, required this.isRead, required this.createdAt, this.data})` — ينشئ كائنًا من `NotificationModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory NotificationModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `NotificationModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class NotificationData` — الكلاس الذي يمثل بيانات `notification data` وينقلها بين الطبقات.
- `NotificationData({required this.items, required this.pageNumber, required this.pageSize, required this.totalCount, required this.totalPages})` — ينشئ كائنًا من `NotificationData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory NotificationData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `NotificationData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/notifications/data/notification_client.dart

- `class NotificationClient` — الكلاس الذي يمثل بيانات `notification client` وينقلها بين الطبقات.
- `factory NotificationClient(Dio dio, {String baseUrl})` — ينشئ كائنًا من `NotificationClient` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResponse<NotificationData>> getNotifications(@Header("Authorization") String token, @Query("pageNumber") int pageNumber, @Query("pageSize") int pageSize)` — يجلب `get notifications` من مصدر البيانات المناسب داخل `NotificationClient`.
- `Future<ApiResponse<dynamic>> sendNotification(@Header("Authorization") String token, @Body() Map<String, dynamic> body)` — يرسل `send notification` إلى الطبقة أو الخدمة المطلوبة داخل `NotificationClient`.
- `Future<ApiResponse<dynamic>> markAsRead(@Header("Authorization") String token, @Path("id") int id)` — يغيّر علامة أو حالة `mark as read` داخل `NotificationClient`.
- `Future<ApiResponse<dynamic>> markAllAsRead(@Header("Authorization") String token)` — يغيّر علامة أو حالة `mark all as read` داخل `NotificationClient`.
- `Future<ApiResponse<dynamic>> registerDeviceToken(@Header("Authorization") String token, @Body() Map<String, dynamic> body)` — يسجّل `register device token` لدى النظام أو الخدمة داخل `NotificationClient`.

features/notifications/presentation/cubit/notification_cubit.dart

- `class NotificationCubit` — الكلاس الذي يدير حالة وتفاعلات `notification cubit`.
- `NotificationCubit(this._getNotificationsUseCase, this._markNotificationReadUseCase, this._markAllNotificationsReadUseCase)` — ينشئ كائنًا من `NotificationCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> init()` — ينفّذ منطق `init` داخل `NotificationCubit`.
- `Future<void> getNotifications()` — يجلب `get notifications` من مصدر البيانات المناسب داخل `NotificationCubit`.
- `Future<void> loadMore()` — يحمّل `load more` ويحدّث الحالة المرتبطة داخل `NotificationCubit`.
- `Future<void> _fetch({required int page})` — خاص — يجلب `fetch` من مصدر البيانات المناسب داخل `NotificationCubit`.
- `Future<void> markAsRead(int id)` — يغيّر علامة أو حالة `mark as read` داخل `NotificationCubit`.

- `Future<void> markAllAsRead()` — يغيّر علامة أو حالة `mark all as read` داخل `NotificationCubit`.

features/notifications/presentation/cubit/notification_state.dart

- `class NotificationState` — الكلاس الذي يدير حالة وتفاعلات `notification state`.
- `const factory NotificationState({@Default([]) List<NotificationModel> items, @Default(false) bool isLoading, @Default(false) bool isLoadingMore, @Default(true) bool hasMore, @Default(1) int currentPage, String? error, String? markReadError})`](../lib/features/notifications/presentation/cubit/notification_state.dart#L8) — ينشئ كائنًا من `NotificationState` ويهيّئ الاعتماديات والقيم المطلوبة.

features/notifications/presentation/pages/notifications_screen.dart

- `class NotificationsScreen` — الكلاس الذي يمثل شاشة `notifications screen`.
- `const NotificationsScreen({super.key})` — ينشئ كائنًا من `NotificationsScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<NotificationsScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _NotificationsScreenState` — خاص — الكلاس الذي يمثل شاشة `notifications screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onScroll()` — خاص — يعالج حدث `on scroll` داخل `_NotificationsScreenState`.
- `Future<void> _openNotification(BuildContext context, NotificationModel notification)` — خاص — يعرض أو يفتح `open notification` داخل `_NotificationsScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _NotificationsTabsHeader` — خاص — الكلاس الذي يمثل شاشة `notifications tabs header`.
- `const _NotificationsTabsHeader({required this.selectedIndex, required this.onTabSelected, required this.onMarkAllRead})` — خاص — ينشئ كائنًا من `NotificationsTabsHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _NotificationTab` — خاص — الكلاس الذي يمثل شاشة `notification tab`.
- `const _NotificationTab({required this.title, required this.isSelected, required this.onTap})` — خاص — ينشئ كائنًا من `NotificationTab` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _NotificationsMessage` — خاص — الكلاس الذي يمثل شاشة `notifications message`.
- `const _NotificationsMessage({required this.icon, required this.title, required this.message, this.actionLabel, this.onAction})` — خاص — ينشئ كائنًا من `NotificationsMessage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/notifications/presentation/widgets/notification_card.dart

- `class NotificationCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const NotificationCard({super.key, required this.notification, this.onTap, this.onMarkAsRead})` — ينشئ كائنًا من `NotificationCard` ويهيّئ الاعتماديات

والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String _formatRelativeTime(DateTime dateTime)` — خاص — ينسّق `format relative time` للعرض أو الإرسال داخل `NotificationCard`.
- `class _NotificationImage` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _NotificationImage(this.imageUrl)` — خاص — ينشئ كائنًا من `NotificationImage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _NotificationFallbackIcon` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _NotificationFallbackIcon()` — خاص — ينشئ كائنًا من `NotificationFallbackIcon` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`features/notifications/push_notification_service.dart`

- `Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message)` — خاص — ينفّذ منطق `firebase messaging background handler` داخل هذا الملف.
- `class PushNotificationService` — الكلاس الذي يجمع مسؤوليات `push notification service`.
- `PushNotificationService._()` — ينشئ كائنًا من `PushNotificationService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Stream<NotificationAction> get onNotificationAction` — يعالج حدث `on notification action` داخل `PushNotificationService`.
- `Stream<String> get onTokenRefresh` — يعالج حدث `on token refresh` داخل `PushNotificationService`.
- `Stream<RemoteMessage> get onMessageReceived` — يعالج حدث `on message received` داخل `PushNotificationService`.
- `String? get currentToken` — ينفّذ منطق `current token` داخل `PushNotificationService`.
- `Future<bool> initialize()` — ينفّذ منطق `initialize` داخل `PushNotificationService`.
- `Future<bool> _requestPermission()` — خاص — ينفّذ منطق `request permission` داخل `PushNotificationService`.
- `Future<void> setupFlutterNotifications()` — يحدّث `setup flutter notifications` داخل `PushNotificationService`.
- `Future<void> showNotification(RemoteMessage message)` — يعرض أو يفتح `show notification` داخل `PushNotificationService`.
- `int _notificationId(RemoteMessage message, {required String? title, required String? body})` — خاص — ينفّذ منطق `notification id` داخل `PushNotificationService`.
- `Future<Uint8List> _downloadBytes(String url)` — خاص — ينفّذ منطق `download bytes` داخل `PushNotificationService`.
- `Color _parseColor(String hex)` — خاص — يحلّل `parse color` ويحوّله لصيغة قابلة للاستخدام داخل `PushNotificationService`.
- `Future<void> _setupMessageHandlers()` — خاص — يحدّث `setup message handlers` داخل `PushNotificationService`.
- `Future<void> _setupTokenHandling()` — خاص — يحدّث `setup token handling` داخل

.PushNotificationService

- `Future<String?> _waitForApnsToken()` — خاص — ينفّذ منطق `wait for apns token` داخل `PushNotificationService`.
- `Future<bool> registerDeviceToken()` — يسجّل `register device token` لدى النظام أو الخدمة داخل `PushNotificationService`.
- `Future<String?> _fetchTokenForRegistration()` — خاص — يجلب `fetch token for registration` من مصدر البيانات المناسب داخل `PushNotificationService`.
- `String _short(String token)` — خاص — ينفّذ منطق `short` داخل `PushNotificationService`.
- `static void _handleNotificationTap(NotificationResponse response)` — خاص — يعالج حدث `handle notification tap` داخل `PushNotificationService`.
- `void _handleMessageOpen(RemoteMessage message)` — خاص — يعالج حدث `handle message open` داخل `PushNotificationService`.
- `void _navigateFromPayload(Map<String, dynamic> data)` — خاص — ينفّذ الانتقال أو الفتح الخاص بـ `navigate from payload` داخل `PushNotificationService`.
- `static Future<bool> openNotificationPackage(int packageId)` — يعرض أو يفتح `open notification package` داخل `PushNotificationService`.
- `Future<void> _openPackageFromNotification(int packageId)` — خاص — يعرض أو يفتح `open package from notification` داخل `PushNotificationService`.
- `static int? _intOrNull(Object? raw)` — خاص — ينفّذ منطق `int or null` داخل `PushNotificationService`.
- `Future<bool> subscribeToTopic(String topic)` — ينفّذ منطق `subscribe to topic` داخل `PushNotificationService`.
- `Future<bool> unsubscribeFromTopic(String topic)` — ينفّذ منطق `unsubscribe from topic` داخل `PushNotificationService`.
- `Future<bool> deleteToken()` — يحذف أو يمسح `delete token` داخل `PushNotificationService`.
- `Future<void> cancelAllNotifications()` — يفحص شرط `cancel all notifications` ويعيد النتيجة داخل `PushNotificationService`.
- `Future<void> cancelNotification(int id)` — يفحص شرط `cancel notification` ويعيد النتيجة داخل `PushNotificationService`.
- `Future<List<PendingNotificationRequest>> getPendingNotifications()` — يجلب `get pending notifications` من مصدر البيانات المناسب داخل `PushNotificationService`.
- `Future<bool> areNotificationsEnabled()` — ينفّذ منطق `are notifications enabled` داخل `PushNotificationService`.
- `void dispose()` — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `class NotificationAction` — الكلاس الذي يجمع مسؤوليات `notification action`.
- `NotificationAction({required this.type, required this.data, this.actionId, this.messageId})` — ينشئ كائنًا من `NotificationAction` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String toString()` — يعيد تمثيلًا نصيًا للكائن.
- `enum NotificationActionType` — يعرّف القيم الثابتة الخاصة بـ `notification action type`.

Feature: packages

`features/packages/presentation/pages/all_packages_screen.dart`

- `class AllPackagesScreen` — الكلاس الذي يمثل شاشة `all packages screen`.

- `const AllPackagesScreen({super.key})` — ينشئ كائنًا من `AllPackagesScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<AllPackagesScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AllPackagesScreenState` — خاص — الكلاس الذي يمثل شاشة `all packages screen` `state`.
- `String? get _query` — خاص — ينفّذ منطق `query` داخل `_AllPackagesScreenState`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onSearchChanged(String _)` — خاص — يعالج حدث `on search changed` داخل `_AllPackagesScreenState`.
- `void _openPackage(BuildContext context, AllPackage item)` — خاص — يعرض أو يفتح `open package` داخل `_AllPackagesScreenState`.
- `void _onScroll()` — خاص — يعالج حدث `on scroll` داخل `_AllPackagesScreenState`.
- `Future<void> _load({bool reset = false})` — خاص — يحمّل `load` ويحدّث الحالة المرتبطة داخل `_AllPackagesScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildBody(bool isArabic)` — خاص — ينفّذ منطق `build body` داخل `_AllPackagesScreenState`.

Feature: profile

`features/profile/data/models/change_language_request_model.dart`

- `class ChangeLanguageRequestModel` — الكلاس الذي يمثل بيانات `change language request` `model` وينقلها بين الطبقات.
- `const ChangeLanguageRequestModel({required this.language})` — ينشئ كائنًا من `ChangeLanguageRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ChangeLanguageRequestModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ChangeLanguageRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/profile/data/models/change_language_response_model.dart`

- `class ChangeLanguageResponseModel` — الكلاس الذي يمثل بيانات `change language` `response model` وينقلها بين الطبقات.
- `const ChangeLanguageResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ChangeLanguageResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ChangeLanguageResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ChangeLanguageResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/profile/data/models/change_password_request_model.dart`

- `class ChangePasswordRequestModel` — الكلاس الذي يمثل بيانات `change password request` `model` وينقلها بين الطبقات.
- `const ChangePasswordRequestModel({required this.currentPassword, required this.newPassword, required this.confirmNewPassword})` — ينشئ كائنًا من `ChangePasswordRequestModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/profile/data/models/change_password_response_model.dart

- `class ChangePasswordResponseModel` — الكلاس الذي يمثل بيانات `change password response model` وينقلها بين الطبقات.
- `const ChangePasswordResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.success, this.message, this.token, this.expiresAt, this.data})` — ينشئ كائنًا من `ChangePasswordResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ChangePasswordResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ChangePasswordResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/profile/data/models/member_card_activation_response_model.dart

- `class MemberCardActivationResponseModel` — الكلاس الذي يمثل بيانات `member card activation response model` وينقلها بين الطبقات.
- `const MemberCardActivationResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `MemberCardActivationResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory MemberCardActivationResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `MemberCardActivationResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/profile/data/models/profile_response_model.dart

- `class ProfileResponseModel` — الكلاس الذي يمثل بيانات `profile response model` وينقلها بين الطبقات.
- `const ProfileResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProfileResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProfileResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProfileResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProfileData` — الكلاس الذي يمثل بيانات `profile data` وينقلها بين الطبقات.
- `const ProfileData({this.firstName, this.lastName, this.phoneNumber, this.nationalId, this.cardNumber, this.passportNumber, this.isMale, this.birthDate, this.imageUrl})` — ينشئ كائنًا من `ProfileData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProfileData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProfileData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `ProfileData copyWith({String? firstName, String? lastName, String? phoneNumber, String? nationalId, String? cardNumber, String? passportNumber, bool? isMale, String? birthDate, String? imageUrl})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

features/profile/data/models/profile_summary_response_model.dart

- `class ProfileSummaryResponseModel` — الكلاس الذي يمثل بيانات `profile summary response model` وينقلها بين الطبقات.
- `ProfileSummaryResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProfileSummaryResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProfileSummaryResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProfileSummaryResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProfileSummaryData` — الكلاس الذي يمثل بيانات `profile summary data` وينقلها بين الطبقات.
- `const ProfileSummaryData({this.familyMembersCount, this.rewardPoints})` — ينشئ كائنًا من `ProfileSummaryData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProfileSummaryData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProfileSummaryData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/profile/domain/usecases/activate_member_card_usecase.dart

- `class ActivateMemberCardUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `activate member card use case`.
- `ActivateMemberCardUseCase(this._repository)` — ينشئ كائنًا من `ActivateMemberCardUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<MemberCardActivationResponseModel>> call({required String cardNumber})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/domain/usecases/change_language_usecase.dart

- `class ChangeLanguageUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `change language use case`.
- `ChangeLanguageUseCase(this.repository)` — ينشئ كائنًا من `ChangeLanguageUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ChangeLanguageResponseModel>> call({required String language})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/domain/usecases/change_password_usecase.dart

- `class ChangePasswordUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `change password use case`.
- `ChangePasswordUseCase(this._repository)` — ينشئ كائنًا من `ChangePasswordUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ChangePasswordResponseModel>> call({required String currentPassword, required String newPassword, required String confirmNewPassword})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/domain/usecases/delete_profile_image_usecase.dart

- `class DeleteProfileImageUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `delete profile image use case`.
- `DeleteProfileImageUseCase(this._repository)` — ينشئ كائنًا من `DeleteProfileImageUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ProfileResponseModel>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/domain/usecases/get_profile_summary_usecase.dart

- `class GetProfileSummaryUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get profile summary use case`.
- `GetProfileSummaryUseCase(this._repository)` — ينشئ كائنًا من `GetProfileSummaryUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ProfileSummaryResponseModel>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/domain/usecases/get_profile_usecase.dart

- `class GetProfileUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get profile use case`.

- `GetProfileUseCase(this._repository)` — ينشئ كائنًا من `GetProfileUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ProfileResponseModel>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/domain/usecases/update_profile_v2_usecase.dart

- `class UpdateProfileV2UseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `update profile v2 use case`.
- `UpdateProfileV2UseCase(this._repository)` — ينشئ كائنًا من `UpdateProfileV2UseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<ProfileResponseModel>> call({required String firstName, required String lastName, required String phoneNumber, required String nationalId, String? passportNumber, required bool isMale, required String birthDate, String? imagePath, bool removeImage = false, String? currentImageUrl})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/profile/presentation/cubit/change_password_cubit.dart

- `class ChangePasswordCubit` — الكلاس الذي يدير حالة وتفاعلات `change password cubit`.
- `ChangePasswordCubit(this._changePasswordUseCase)` — ينشئ كائنًا من `ChangePasswordCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> changePassword({required String currentPassword, required String newPassword, required String confirmNewPassword})` — يحدّث `change password` داخل `ChangePasswordCubit`.

features/profile/presentation/cubit/change_password_state.dart

- `class ChangePasswordState` — الكلاس الذي يدير حالة وتفاعلات `change password state`.
- `const factory ChangePasswordState.initial()` — ينشئ كائنًا من `ChangePasswordState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ChangePasswordState.loading()` — ينشئ كائنًا من `ChangePasswordState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ChangePasswordState.success()` — ينشئ كائنًا من `ChangePasswordState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ChangePasswordState.failure(String message)` — ينشئ كائنًا من `ChangePasswordState` ويهيّئ الاعتماديات والقيم المطلوبة.

features/profile/presentation/cubit/profile_cubit.dart

- `class ProfileCubit` — الكلاس الذي يدير حالة وتفاعلات `profile cubit`.
- `ProfileCubit(this._getProfileUseCase, this._getProfileSummaryUseCase, this._updateProfileUseCase, this._deleteProfileImageUseCase, this._changeLanguageUseCase, this._activateMemberCardUseCase, this._logoutUseCase)` — ينشئ كائنًا من `ProfileCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadProfile({bool forceRefresh = false})` — يحمّل `load profile` ويحدّث الحالة المرتبطة داخل `ProfileCubit`.
- `Future<void> _performProfileLoad()` — خاص — ينفّذ منطق `perform profile load` داخل `ProfileCubit`.
- `Future<bool> updateProfile({required String firstName, required String lastName, required String phoneNumber, required String nationalId, String? passportNumber, required bool isMale, required String birthDate, String? imagePath, bool removeImage = false, required ProfileData currentData})` — يحدّث `update profile` داخل `ProfileCubit`.
- `Future<void> changeLanguage({required String language})` — يحدّث `change language` داخل `ProfileCubit`.

- `Future<bool> deleteProfileImage(ProfileData currentData)` — يحذف أو يمسح `delete profile image` داخل `ProfileCubit`.
- `Future<bool> activateMemberCard({required String cardNumber, required ProfileData currentData})` — ينفّذ منطق `activate member card` داخل `ProfileCubit`.
- `Future<void> refreshSummary()` — يعيد تحميل أو محاولة `refresh summary` داخل `ProfileCubit`.
- `Future<void> _loadSummary(ProfileData profile)` — خاص — يحمّل `load summary` ويحدّث الحالة المرتبطة داخل `ProfileCubit`.
- `Future<void> logout()` — ينهي الجلسة الحالية وينظّف بيانات المصادقة داخل `ProfileCubit`.

features/profile/presentation/cubit/profile_state.dart

- `class ProfileState` — الكلاس الذي يدير حالة وتفاعلات `profile state`.
- `const factory ProfileState.initial()` — ينشئ كائنًا من `ProfileState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ProfileState.loading()` — ينشئ كائنًا من `ProfileState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ProfileState.success(ProfileData data, [ProfileSummaryData? summary])`(../lib/features/profile/presentation/cubit/profile_state.dart#L11) — ينشئ كائنًا من `ProfileState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ProfileState.updating(ProfileData data)` — ينشئ كائنًا من `ProfileState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ProfileState.updated(ProfileData data)` — ينشئ كائنًا من `ProfileState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory ProfileState.failed(String error)` — ينشئ كائنًا من `ProfileState` ويهيّئ الاعتماديات والقيم المطلوبة.

features/profile/presentation/pages/change_password_screen.dart

- `class ChangePasswordScreen` — الكلاس الذي يمثل شاشة `change password screen`.
- `const ChangePasswordScreen({super.key})` — ينشئ كائنًا من `ChangePasswordScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ChangePasswordScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ChangePasswordScreenState` — خاص — الكلاس الذي يمثل شاشة `change password screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onNewPasswordFocusChanged()` — خاص — يعالج حدث `on new password focus changed` داخل `_ChangePasswordScreenState`.
- `void _onConfirmPasswordFocusChanged()` — خاص — يعالج حدث `on confirm password focus changed` داخل `_ChangePasswordScreenState`.
- `void _onNewPasswordChanged()` — خاص — يعالج حدث `on new password changed` داخل `_ChangePasswordScreenState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _submit()` — خاص — يرسل `submit` إلى الطبقة أو الخدمة المطلوبة داخل `_ChangePasswordScreenState`.
- `String? _validateNewPassword(String? value)` — خاص — يتحقق من صحة `validate new password` داخل `_ChangePasswordScreenState`.
- `bool _isPasswordValid(String password)` — خاص — يفحص شرط `is password valid` ويعيد النتيجة داخل `_ChangePasswordScreenState`.
- `bool _hasMinimumLength(String password)` — خاص — يفحص شرط `has minimum`

length ويعيد النتيجة داخل _ChangePasswordScreenState.

- bool _hasLetter(String password) — خاص — يفحص شرط has letter ويعيد النتيجة داخل _ChangePasswordScreenState.
- bool _hasNumber(String password) — خاص — يفحص شرط has number ويعيد النتيجة داخل _ChangePasswordScreenState.
- bool _hasSpecialCharacter(String password) — خاص — يفحص شرط has special character ويعيد النتيجة داخل _ChangePasswordScreenState.
- Widget build(BuildContext context) — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- Widget _buildPasswordField({Key? fieldKey, required TextEditingController controller, required String hintText, required bool showPassword, required VoidCallback onToggle, required String? Function(String?) validator, FocusNode? focusNode, AutovalidateMode autovalidateMode = AutovalidateMode.onUserInteraction}) — خاص — ينفّذ منطق build password field داخل _ChangePasswordScreenState.
- Widget _buildPasswordRequirements() — خاص — ينفّذ منطق build password requirements داخل _ChangePasswordScreenState.
- Widget _buildPasswordRequirement({required String label, required bool isMet, required bool showInvalid}) — خاص — ينفّذ منطق build password requirement داخل _ChangePasswordScreenState.
- class _FieldLabel — خاص — الكلاس الذي يمثل شاشة field label.
- const _FieldLabel(this.label) — خاص — ينشئ كائنًا من FieldLabel ويهيّئ الاعتماديات والقيم المطلوبة.
- Widget build(BuildContext context) — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/pages/edit_profile_screen.dart

- class EditProfileScreen — الكلاس الذي يمثل شاشة edit profile screen.
- const EditProfileScreen({super.key, required this.profileData}) — ينشئ كائنًا من EditProfileScreen ويهيّئ الاعتماديات والقيم المطلوبة.
- State<EditProfileScreen> createState() — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- class _EditProfileScreenState — خاص — الكلاس الذي يمثل شاشة edit profile screen state.
- Future<void> _pickImage() — خاص — ينفّذ منطق pick image داخل _EditProfileScreenState.
- bool get _hasCurrentPhoto — خاص — يفحص شرط has current photo ويعيد النتيجة داخل _EditProfileScreenState.
- Future<void> _showImageOptions() — خاص — يعرض أو يفتح show image options داخل _EditProfileScreenState.
- Future<void> _removeCurrentImage() — خاص — يحذف أو يمسح remove current image داخل _EditProfileScreenState.
- Future<void> _selectBirthDate() — خاص — يغيّر الاختيار الخاص بـ select birth date داخل _EditProfileScreenState.
- void initState() — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- void dispose() — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- String? _validateName(String? value) — خاص — يتحقق من صحة validate name داخل _EditProfileScreenState.
- String? _validatePhoneNumber(String? value) — خاص — يتحقق من صحة validate

phone number داخل _EditProfileScreenState.

- String? _validateNationalId(String? value) — خاص — يتحقق من صحة validate national id داخل _EditProfileScreenState.
- Widget build(BuildContext context) — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- Future<void> _saveProfile() — خاص — ينفّذ منطق save profile داخل _EditProfileScreenState.
- TextInputFormatter _digitsOnly({required int maxDigits}) — خاص — ينفّذ منطق digits only داخل _EditProfileScreenState.
- TextInputFormatter _lettersOnly() — خاص — ينفّذ منطق letters only داخل _EditProfileScreenState.

features/profile/presentation/pages/profile_screen.dart

- class ProfileScreen — الكلاس الذي يمثل شاشة profile screen.
- const ProfileScreen({super.key}) — ينشئ كائنًا من ProfileScreen ويهيّئ الاعتماديات والقيم المطلوبة.
- State<ProfileScreen> createState() — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- class _ProfileScreenState — خاص — الكلاس الذي يمثل شاشة profile screen state.
- void initState() — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- void _onProfileSummaryRefresh() — خاص — يعالج حدث on profile summary refresh داخل _ProfileScreenState.
- void dispose() — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- Widget build(BuildContext context) — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/activate_card_banner.dart

- class ActivateCardBanner — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- const ActivateCardBanner({super.key, required this.data}) — ينشئ كائنًا من ActivateCardBanner ويهيّئ الاعتماديات والقيم المطلوبة.
- Widget build(BuildContext context) — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/card_number_input_sheet.dart

- Future<void> showActivateCardDialog(BuildContext context, ProfileData data) — يعرض أو يفتح show activate card dialog داخل هذا الملف.
- class CardNumberInputSheet — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- const CardNumberInputSheet({super.key}) — ينشئ كائنًا من CardNumberInputSheet ويهيّئ الاعتماديات والقيم المطلوبة.
- State<CardNumberInputSheet> createState() — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- class CardNumberInputSheetState — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- void dispose() — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- void _submit() — خاص — يرسل submit إلى الطبقة أو الخدمة المطلوبة داخل CardNumberInputSheetState.
- String? _validateCardNumber(String? value) — خاص — يتحقق من صحة validate card number داخل CardNumberInputSheetState.
- Widget build(BuildContext context) — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/edit_profile/edit_identity_section.dart

- `class EditIdentitySection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditIdentitySection({super.key, required this.cardNumber, required this.cardNumberController, required this.nationalIdController, required this.nationalIdFocusNode, required this.nationalIdAutovalidate, required this.digitsFormatters, required this.nationalIdValidator, required this.passportNumberController, required this.passportNumberFocusNode, required this.passportNumberAutovalidate, required this.phoneNumberController, required this.phoneNumberFocusNode, required this.phoneNumberAutovalidate, required this.phoneNumberValidator, required this.birthDateController, required this.birthDateIsEmpty, required this.onSelectBirthDate, required this.isMale, required this.onGenderChanged})` — ينشئ كائنًا من `EditIdentitySection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class EditBirthDateField` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditBirthDateField({super.key, required this.controller, required this.isEmpty, required this.onTap})` — ينشئ كائنًا من `EditBirthDateField` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/edit_profile/edit_name_row.dart

- `class EditNameRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditNameRow({super.key, required this.firstNameController, required this.lastNameController, required this.firstNameFocusNode, required this.lastNameFocusNode, required this.firstNameAutovalidate, required this.lastNameAutovalidate, required this.nameValidator, required this.lettersFormatters})` — ينشئ كائنًا من `EditNameRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/edit_profile/edit_profile_avatar.dart

- `class EditProfileAvatar` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditProfileAvatar({super.key, required this.imageFile, required this.isPickingImage, required this.removeImage, required this.imageUrl, required this.onPreviewTap, required this.onEditTap})` — ينشئ كائنًا من `EditProfileAvatar` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _RemoteImage` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _RemoteImage({required this.imageUrl})` — خاص — ينشئ كائنًا من `RemoteImage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `bool hasValidProfileImageUrl(String? url)` — يفحص شرط `has valid profile image url` ويعيد النتيجة داخل هذا الملف.
- `void showProfileImagePreview(BuildContext context, {required File? imageFile, required String? imageUrl, required bool removeImage})` — يعرض أو يفتح `show profile image preview` داخل هذا الملف.

- `class _PreviewPhoto` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _PreviewPhoto({required this.imageFile, required this.imageUrl})` — خاص — ينشئ كائنًا من `PreviewPhoto` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/edit_profile/edit_profile_gender_dropdown.dart

- `class EditProfileGenderDropdown` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditProfileGenderDropdown({super.key, required this.isMale, required this.onChanged})` — ينشئ كائنًا من `EditProfileGenderDropdown` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/edit_profile/edit_profile_labels.dart

- `class EditProfileRequiredHint` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditProfileRequiredHint(this.text, {super.key})` — ينشئ كائنًا من `EditProfileRequiredHint` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class EditProfileFieldLabel` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditProfileFieldLabel(this.label, {super.key})` — ينشئ كائنًا من `EditProfileFieldLabel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/edit_profile/edit_profile_text_field.dart

- `class EditProfileTextField` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const EditProfileTextField({super.key, required this.controller, required this.hintText, this.keyboardType, this.required = true, this.readOnly = false, this.maxLength, this.inputFormatters, this.validator, this.focusNode, this.autovalidateMode = AutovalidateMode.disabled})` — ينشئ كائنًا من `EditProfileTextField` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/profile_avatar.dart

- `class ProfileAvatar` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileAvatar({super.key, required this.imageUrl})` — ينشئ كائنًا من `ProfileAvatar` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _fallbackAvatar()` — خاص — ينفّذ منطق `fallback avatar` داخل `ProfileAvatar`.

features/profile/presentation/widgets/profile_card.dart

- `class ProfileCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileCard({super.key, required this.children})` — ينشئ كائنًا من `ProfileCard` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/profile_content.dart

- `class ProfileContent` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileContent({super.key, required this.data, this.summary})` — ينشئ كائنًا من `ProfileContent` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Future<void> showLogoutConfirmation(BuildContext context)` — يعرض أو يفتح `show logout confirmation` داخل هذا الملف.

features/profile/presentation/widgets/profile_error_view.dart

- `class ProfileErrorView` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileErrorView({super.key, required this.error})` — ينشئ كائنًا من `ProfileErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/profile_item.dart

- `class ProfileItem` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileItem({super.key, this.icon, required this.label, this.trailing, this.route})` — ينشئ كائنًا من `ProfileItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/profile_language_item.dart

- `class ProfileLanguageItem` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileLanguageItem({super.key, required this.data})` — ينشئ كائنًا من `ProfileLanguageItem` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Future<void> _showLanguageSheet(BuildContext context, String current)` — خاص — يعرض أو يفتح `show language sheet` داخل `ProfileLanguageItem`.
- `Widget option(String code, String name)` — ينفّذ منطق `option` داخل `ProfileLanguageItem`.

features/profile/presentation/widgets/profile_section_title.dart

- `class ProfileSectionTitle` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileSectionTitle({super.key, required this.title})` — ينشئ كائنًا من `ProfileSectionTitle` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/profile_shimmer.dart

- `class ProfileShimmer` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileShimmer({super.key})` — ينشئ كائنًا من `ProfileShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _SectionTitleShimmer` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.

- `const _SectionTitleShimmer()` — خاص — ينشئ كائنًا من `SectionTitleShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/profile/presentation/widgets/profile_user_info_card.dart

- `class ProfileUserInfoCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProfileUserInfoCard({super.key, required this.data})` — ينشئ كائنًا من `ProfileUserInfoCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Feature: search

features/search/data/data_sources/search_remote_data_source.dart

- `class SearchRemoteDataSource` — الكلاس الذي يعرّف الاتصال بمصدر البيانات أو REST API.
- `factory SearchRemoteDataSource(Dio dio, {String baseUrl})` — ينشئ كائنًا من `SearchRemoteDataSource` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<SearchResponseModel> searchProviders({@Query('Keyword') String? keyword, @Query('CategoryId') int? categoryId, @Query('GovernorateId') int? governorateId, @Query('CityId') int? cityId, @Query('ProviderId') int? providerId, @Query('BranchId') int? branchId, @Query('SortBy') int? sortBy, @Query('UserLatitude') double? userLatitude, @Query('UserLongitude') double? userLongitude, @Query('PageNumber') int? pageNumber, @Query('PageSize') int? pageSize})` — ينفّذ البحث الخاص بـ `search providers` داخل `SearchRemoteDataSource`.
- `Future<CategoryResponseModel> getCategories()` — يجلب `get categories` من مصدر البيانات المناسب داخل `SearchRemoteDataSource`.
- `Future<CategoryResponseModel> getServiceCategories()` — يجلب `get service categories` من مصدر البيانات المناسب داخل `SearchRemoteDataSource`.
- `Future<LookupResponseModel> getGovernorates()` — يجلب `get governorates` من مصدر البيانات المناسب داخل `SearchRemoteDataSource`.
- `Future<LookupResponseModel> getCities(@Query('governorateId') int governorateId)` — يجلب `get cities` من مصدر البيانات المناسب داخل `SearchRemoteDataSource`.
- `Future<CategoryServicesResponseModel> getCategoryServices(@Path('categoryId') int categoryId, @Query('PageNumber') int? pageNumber, @Query('PageSize') int? pageSize, @Extras() Map<String, dynamic> extras)` — يجلب `get category services` من مصدر البيانات المناسب داخل `SearchRemoteDataSource`.
- `Future<GuestCategoryServicesResponseModel> getGuestCategoryServices(@Query('categoryId') int categoryId, @Query('Search') String? search, @Query('PageNumber') int? pageNumber, @Query('PageSize') int? pageSize)` — يجلب `get guest category services` من مصدر البيانات المناسب داخل `SearchRemoteDataSource`.

features/search/data/models/category_model.dart

- `class CategoryModel` — الكلاس الذي يمثل بيانات `category model` وينقلها بين الطبقات.
- `const CategoryModel({this.id, this.name, this.imageUrl})` — ينشئ كائنًا من `CategoryModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/search/data/models/category_response_model.dart

- `class CategoryResponseModel` — الكلاس الذي يمثل بيانات `category response model` وينقلها بين الطبقات.
- `const CategoryResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `CategoryResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/search/data/models/category_services_response_model.dart

- `class CategoryServicesResponseModel` — الكلاس الذي يمثل بيانات `category services response model` وينقلها بين الطبقات.
- `const CategoryServicesResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `CategoryServicesResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryServicesResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryServicesResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class CategoryServicesDataModel` — الكلاس الذي يمثل بيانات `category services data model` وينقلها بين الطبقات.
- `const CategoryServicesDataModel({this.categoryId, this.categoryName, this.services, this.totalSelectedCount})` — ينشئ كائنًا من `CategoryServicesDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory CategoryServicesDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `CategoryServicesDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class PagedServicesModel` — الكلاس الذي يمثل بيانات `paged services model` وينقلها بين الطبقات.
- `const PagedServicesModel({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `PagedServicesModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory PagedServicesModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `PagedServicesModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ServiceItemModel` — الكلاس الذي يمثل بيانات `service item model` وينقلها بين الطبقات.
- `const ServiceItemModel({this.serviceId, this.serviceNameEn, this.serviceNameAr, this.isSelected})` — ينشئ كائنًا من `ServiceItemModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String? get serviceName` — ينفّذ منطق `service name` داخل `ServiceItemModel`.
- `factory ServiceItemModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ServiceItemModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/search/data/models/guest_category_services_response_model.dart

- `class GuestCategoryServicesResponseModel` — الكلاس الذي يمثل بيانات `guest category services response model` وينقلها بين الطبقات.
- `const GuestCategoryServicesResponseModel({this.httpStatusCode,`

`this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `GuestCategoryServicesResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory GuestCategoryServicesResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestCategoryServicesResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class GuestPagedServicesDataModel` — الكلاس الذي يمثل بيانات `guest paged services data model` وينقلها بين الطبقات.
- `const GuestPagedServicesDataModel({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `GuestPagedServicesDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestPagedServicesDataModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestPagedServicesDataModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `CategoryServicesDataModel toCategoryServicesDataModel(int categoryId)` — يحوّل `to category services data model` بين طبقات أو صيغ البيانات داخل `GuestPagedServicesDataModel`.
- `class GuestServiceItemModel` — الكلاس الذي يمثل بيانات `guest service item model` وينقلها بين الطبقات.
- `const GuestServiceItemModel({this.id, this.serviceName, this.serviceNameAr, this.serviceNameEn})` — ينشئ كائنًا من `GuestServiceItemModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory GuestServiceItemModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `GuestServiceItemModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/search/data/models/lookup_model.dart`

- `class LookupModel` — الكلاس الذي يمثل بيانات `lookup model` وينقلها بين الطبقات.
- `const LookupModel({this.id, this.name})` — ينشئ كائنًا من `LookupModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory LookupModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `LookupModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/search/data/models/lookup_response_model.dart`

- `class LookupResponseModel` — الكلاس الذي يمثل بيانات `lookup response model` وينقلها بين الطبقات.
- `const LookupResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `LookupResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory LookupResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `LookupResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`features/search/data/models/search_provider_model.dart`

- `class SearchProviderModel` — الكلاس الذي يمثل بيانات `search provider model` وينقلها بين الطبقات.
- `const SearchProviderModel({this.providerId, this.providerName, this.categoryName, this.imageUrl, this.hotline, this.maxDiscountPercentage, this.averageRating, this.totalReviews, this.distanceKm})` — ينشئ كائنًا من `SearchProviderModel` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory SearchProviderModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `SearchProviderModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `int? _intValue(dynamic value)` — خاص — ينفّذ منطق `int value` داخل هذا الملف.
- `double? _doubleValue(dynamic value)` — خاص — ينفّذ منطق `double value` داخل هذا الملف.
- `String? _stringValue(dynamic value)` — خاص — ينفّذ منطق `string value` داخل هذا الملف.

features/search/data/models/search_response_model.dart

- `class SearchResponseModel` — الكلاس الذي يمثل بيانات `search response model` وينقلها بين الطبقات.
- `const SearchResponseModel({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `SearchResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory SearchResponseModel.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `SearchResponseModel` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class SearchResponseData` — الكلاس الذي يمثل بيانات `search response data` وينقلها بين الطبقات.
- `const SearchResponseData({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `SearchResponseData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory SearchResponseData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `SearchResponseData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

features/search/data/repositories/search_repository_impl.dart

- `class SearchRepositoryImpl` — الكلاس الذي يمثل بيانات `search repository impl` وينقلها بين الطبقات.
- `SearchRepositoryImpl(this._dataSource)` — ينشئ كائنًا من `SearchRepositoryImpl` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<SearchResponseData>> searchProviders({String? keyword, int? categoryId, int? governorateId, int? cityId, int? providerId, int? branchId, int? sortBy, double? userLatitude, double? userLongitude, int? pageNumber, int? pageSize})` — ينفّذ البحث الخاص بـ `search providers` داخل `SearchRepositoryImpl`.
- `Future<ApiResult<List<CategoryModel>>> getCategories()` — يجلب `get categories` من مصدر البيانات المناسب داخل `SearchRepositoryImpl`.
- `Future<ApiResult<List<CategoryModel>>> getServiceCategories()` — يجلب `get service categories` من مصدر البيانات المناسب داخل `SearchRepositoryImpl`.
- `Future<ApiResult<List<LookupModel>>> getGovernorates()` — يجلب `get governorates` من مصدر البيانات المناسب داخل `SearchRepositoryImpl`.
- `Future<ApiResult<List<LookupModel>>> getCities(int governorateId)` — يجلب `get cities` من مصدر البيانات المناسب داخل `SearchRepositoryImpl`.
- `Future<ApiResult<CategoryServicesDataModel>> getCategoryServices(int categoryId, {int? pageNumber, int? pageSize, bool skipAuthInterceptor = false})` — يجلب `get category services` من مصدر البيانات المناسب داخل `SearchRepositoryImpl`.
- `Future<ApiResult<CategoryServicesDataModel>> getGuestCategoryServices(int categoryId, {String? search, int? pageNumber, int? pageSize})` — يجلب `get guest category services` من مصدر البيانات

المناسب داخل `SearchRepositoryImpl`.

- `String _responseMessage({required dynamic errors, required String? message, required String fallback})` — خاص — ينفّذ منطق `response message` داخل هذا الملف.
- `String _extractMessages(dynamic value)` — خاص — ينفّذ منطق `extract messages` داخل هذا الملف.

features/search/domain/repositories/search_repository.dart

- `class SearchRepository` — الكلاس الذي يجمع مسؤوليات `search repository`.
- `Future<ApiResult<SearchResponseData>> searchProviders({String? keyword, int? categoryId, int? governorateId, int? cityId, int? providerId, int? branchId, int? sortBy, double? userLatitude, double? userLongitude, int? pageNumber, int? pageSize})` — ينفّذ البحث الخاص بـ `search providers` داخل `SearchRepository`.
- `Future<ApiResult<List<CategoryModel>>> getCategories()` — يجلب `get categories` من مصدر البيانات المناسب داخل `SearchRepository`.
- `Future<ApiResult<List<CategoryModel>>> getServiceCategories()` — يجلب `get service categories` من مصدر البيانات المناسب داخل `SearchRepository`.
- `Future<ApiResult<List<LookupModel>>> getGovernorates()` — يجلب `get governorates` من مصدر البيانات المناسب داخل `SearchRepository`.
- `Future<ApiResult<List<LookupModel>>> getCities(int governorateId)` — يجلب `get cities` من مصدر البيانات المناسب داخل `SearchRepository`.
- `Future<ApiResult<CategoryServicesDataModel>> getCategoryServices(int categoryId, {int? pageNumber, int? pageSize, bool skipAuthInterceptor = false})` — يجلب `get category services` من مصدر البيانات المناسب داخل `SearchRepository`.
- `Future<ApiResult<CategoryServicesDataModel>> getGuestCategoryServices(int categoryId, {String? search, int? pageNumber, int? pageSize})` — يجلب `get guest category services` من مصدر البيانات المناسب داخل `SearchRepository`.

features/search/domain/use_cases/get_categories_use_case.dart

- `class GetCategoriesUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get categories use case`.
- `GetCategoriesUseCase(this._repository)` — ينشئ كائنًا من `GetCategoriesUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<List<CategoryModel>>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/search/domain/use_cases/get_category_services_use_case.dart

- `class GetCategoryServicesUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get category services use case`.
- `GetCategoryServicesUseCase(this._repository)` — ينشئ كائنًا من `GetCategoryServicesUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<CategoryServicesDataModel>> call(int categoryId, {int? pageNumber, int? pageSize, bool skipAuthInterceptor = false})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.
- `Future<ApiResult<CategoryServicesDataModel>> callGuestServices(int categoryId, {String? search, int? pageNumber, int? pageSize})` — ينفّذ منطق `call guest services` داخل `GetCategoryServicesUseCase`.

features/search/domain/use_cases/get_cities_use_case.dart

- `class GetCitiesUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get cities use case`.
- `GetCitiesUseCase(this._repository)` — ينشئ كائنًا من `GetCitiesUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<List<LookupModel>>> call(int governorateId)` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/search/domain/use_cases/get_governorates_use_case.dart

- `class GetGovernoratesUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get governorates use case`.
- `GetGovernoratesUseCase(this._repository)` — ينشئ كائنًا من `GetGovernoratesUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<List<LookupModel>>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/search/domain/use_cases/get_service_categories_use_case.dart

- `class GetServiceCategoriesUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get service categories use case`.
- `GetServiceCategoriesUseCase(this._repository)` — ينشئ كائنًا من `GetServiceCategoriesUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<List<CategoryModel>>> call()` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/search/domain/use_cases/search_providers_use_case.dart

- `class SearchProvidersUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `search providers use case`.
- `SearchProvidersUseCase(this._repository)` — ينشئ كائنًا من `SearchProvidersUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<SearchResponseData>> call({String? keyword, int? categoryId, int? governorateId, int? cityId, int? providerId, int? branchId, int? sortBy, double? userLatitude, double? userLongitude, int? pageNumber, int? pageSize})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

features/search/presentation/cubit/filter_cubit.dart

- `class FilterCubit` — الكلاس الذي يدير حالة وتفاعلات `filter cubit`.
- `FilterCubit(this._getGovernoratesUseCase, this._getCitiesUseCase)` — ينشئ كائنًا من `FilterCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> loadGovernorates()` — يحمّل `load governorates` ويحدّث الحالة المرتبطة داخل `FilterCubit`.
- `Future<void> selectGovernorate(int? governorateId)` — يغيّر الاختيار الخاص بـ `select governorate` داخل `FilterCubit`.
- `void selectCity(int? cityId)` — يغيّر الاختيار الخاص بـ `select city` داخل `FilterCubit`.
- `void selectSortBy(int? sortBy)` — يغيّر الاختيار الخاص بـ `select sort by` داخل `FilterCubit`.
- `void resetFilters()` — ينفّذ منطق `reset filters` داخل `FilterCubit`.

features/search/presentation/cubit/filter_state.dart

- `class FilterState` — الكلاس الذي يدير حالة وتفاعلات `filter state`.
- `const FilterState({this.governorates = const [], this.cities = const [], this.isLoadingGovernorates = false, this.isLoadingCities = false, this.selectedGovernorateId, this.selectedCityId, this.selectedSortBy, this.error})`](../lib/features/search/presentation/cubit/filter_state.dart#L13) — ينشئ كائنًا من

`FilterState` ويهيّئ الاعتماديات والقيم المطلوبة.

- `FilterState copyWith({List<LookupModel>? governorates, List<LookupModel>? cities, bool? isLoadingGovernorates, bool? isLoadingCities, int? selectedGovernorateId, bool clearGovernorate = false, int? selectedCityId, bool clearCity = false, int? selectedSortBy, bool clearSortBy = false, String? error, bool clearError = false})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

features/search/presentation/cubit/search_cubit.dart

- `class SearchCubit` — الكلاس الذي يدير حالة وتفاعلات `search cubit`.
- `SearchCubit(this._searchProvidersUseCase, this._getCategoriesUseCase, this._getCategoryServicesUseCase, this._getServiceCategoriesUseCase, this._cartRepository, this._cartSyncService, {LocationService? locationService, LocationRepository? locationRepository})` — ينشئ كائنًا من `SearchCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `bool get enableCartFeatures` — ينفّذ منطق `enable cart features` داخل `SearchCubit`.
- `Future<void> init({double? lat, double? lng, int? categoryId, bool enableCartFeatures = true})` — ينفّذ منطق `init` داخل `SearchCubit`.
- `Future<void> _loadCategories()` — خاص — يحمّل `load categories` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `Future<void> _loadServiceCategories()` — خاص — يحمّل `load service categories` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `Future<void> _loadCartCounters()` — خاص — يحمّل `load cart counters` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `Future<bool> loadCategoryServices(int categoryId, {String? search})` — يحمّل `load category services` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `Future<int?> loadFirstServiceCategoryWithData()` — يحمّل `load first service category with data` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `bool hasMoreCategoryServices(int categoryId)` — يفحص شرط `has more category services` ويعيد النتيجة داخل `SearchCubit`.
- `Future<bool> loadMoreCategoryServices(int categoryId)` — يحمّل `load more category services` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `Future<void> addServiceToCart({required int serviceId, required int categoryId})` — ينشئ أو يضيف `add service to cart` داخل `SearchCubit`.
- `Future<void> removeServiceFromCart({required int serviceId, required int categoryId})` — يحذف أو يمسح `remove service from cart` داخل `SearchCubit`.
- `Future<void> search({String keyword = ''})` — ينفّذ البحث الخاص بـ `search` داخل `SearchCubit`.
- `Future<void> loadMore()` — يحمّل `load more` ويحدّث الحالة المرتبطة داخل `SearchCubit`.
- `bool _hasMoreProviders({required int page, required int newItemsCount, required int? totalPages, required int? totalCount, required int pageSize})` — خاص — يفحص شرط `has more providers` ويعيد النتيجة داخل `SearchCubit`.
- `Future<void> selectCategory(int? id)` — يغيّر الاختيار الخاص بـ `select category` داخل `SearchCubit`.
- `Future<void> applyFilters({int? governorateId, int? cityId, int? sortBy})` — ينفّذ منطق `apply filters` داخل `SearchCubit`.
- `Future<LocationResult> _resolveUserLocation()` — خاص — ينفّذ منطق `resolve user location` داخل `SearchCubit`.
- `String _locationUnavailableMessage(LocationAccessStatus status)` — خاص —

ينفّذ منطق `location unavailable message` داخل `SearchCubit`.

- `Future<void> _fetch({required int page})` — خاص — يجلب `fetch` من مصدر البيانات المناسب داخل `SearchCubit`.
- `void _emitLocalServiceSelection({required int serviceId, required int categoryId, required bool isSelected, required Set<int> pendingServiceIds})` — خاص — ينفّذ منطق `emit local service selection` داخل `SearchCubit`.
- `CategoryServicesDataModel _copyCategoryServiceSelection(CategoryServicesDataModel data, {required int serviceId, required bool isSelected})` — خاص — ينفّذ منطق `copy category service selection` داخل `SearchCubit`.
- `int _nextSelectedCount(int current, {required bool increment})` — خاص — ينفّذ منطق `next selected count` داخل `SearchCubit`.
- `bool? _serviceSelection(CategoryServicesDataModel? data, int serviceId)` — خاص — ينفّذ منطق `service selection` داخل `SearchCubit`.
- `void _onCartSyncEvent(CartSyncEvent event)` — خاص — يعالج حدث `on cart sync event` داخل `SearchCubit`.
- `void _emitSelectionSnapshot(Set<int> selectedServiceIds, {Map<int, int>? categorySelectedCounts})` — خاص — ينفّذ منطق `emit selection snapshot` داخل `SearchCubit`.
- `void _emitServiceSelectionInLoadedCategories({required int serviceId, required bool isSelected})` — خاص — ينفّذ منطق `emit service selection in loaded categories` داخل `SearchCubit`.
- `void _emitAllServicesUnselected()` — خاص — ينفّذ منطق `emit all services unselected` داخل `SearchCubit`.
- `CategoryServicesDataModel _copyAllCategoryServicesSelection(CategoryServicesDataModel data, {required bool isSelected})` — خاص — ينفّذ منطق `copy all category services selection` داخل `SearchCubit`.
- `Future<void> close()` — يغلق الـ Cubit أو الـ streams والموارد التابعة له.

features/search/presentation/cubit/search_state.dart

- `class SearchState` — الكلاس الذي يدير حالة وتفاعلات `search state`.
- `const SearchState({this.items = const [], this.categories = const [], this.serviceCategories = const [], this.isLoadingServiceCategories = false, this.categorySelectedCounts = const {}, this.isLoading = false, this.isLoadingMore = false, this.error, this.hasMore = true, this.currentPage = 1, this.selectedCategoryId, this.keyword = '', this.userLatitude, this.userLongitude, this.isLoadingServices = false, this.servicesError, this.servicesData, this.services = const [], this.servicesByCategoryId = const {}, this.loadingServiceCategoryIds = const {}, this.loadingMoreServiceCategoryIds = const {}, this.pendingServiceIds = const {}, this.cartActionError, this.categoriesError, this.serviceCategoriesError, this.filterError})](../lib/features/search/presentation/cubit/search_state.dart#L54) — ينشئ كائنًا من `SearchState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `SearchState copyWith({List<SearchProviderModel>? items, List<CategoryModel>? categories, List<CategoryModel>? serviceCategories, bool? isLoadingServiceCategories, Map<int, int>? categorySelectedCounts, bool? isLoading, bool? isLoadingMore, String? error, bool clearError = false, bool? hasMore, int? currentPage, int? selectedCategoryId, bool clearSelectedCategory = false, String? keyword, double? userLatitude,`

```
double? userLongitude, bool? isLoadingServices, String? servicesError,
        bool clearServicesError = false, CategoryServicesDataModel?
           servicesData, List<ServiceItemModel>? services, Map<int,
        CategoryServicesDataModel>? servicesByCategoryId, Set<int>?
  loadingServiceCategoryIds, Set<int>? loadingMoreServiceCategoryIds,
         Set<int>? pendingServiceIds, String? cartActionError, bool
        clearCartActionError = false, String? categoriesError, bool
   clearCategoriesError = false, String? serviceCategoriesError, bool
       clearServiceCategoriesError = false, String? filterError, bool
```
`clearFilterError = false})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

features/search/presentation/screens/choose_provider_screen.dart

- `class ChooseProviderScreen` — الكلاس الذي يمثل شاشة `choose provider screen`.
- `const ChooseProviderScreen({super.key})` — ينشئ كائنًا من `ChooseProviderScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildBody(BuildContext context, CartState state)` — خاص — ينفّذ منطق `build body` داخل `ChooseProviderScreen`.
- `class _CoveragePill` — خاص — الكلاس الذي يمثل شاشة `coverage pill`.
- `const _CoveragePill({required this.provider})` — خاص — ينشئ كائنًا من `CoveragePill` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ProviderCard` — خاص — الكلاس الذي يمثل شاشة `provider card`.
- `const _ProviderCard({required this.provider, required this.isSelected, required this.onTap})` — خاص — ينشئ كائنًا من `ProviderCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ChooseProviderBottomBar` — خاص — الكلاس الذي يمثل شاشة `choose provider bottom bar`.
- `const _ChooseProviderBottomBar({required this.provider})` — خاص — ينشئ كائنًا من `ChooseProviderBottomBar` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _Initials` — خاص — الكلاس الذي يمثل شاشة `initials`.
- `const _Initials({required this.name})` — خاص — ينشئ كائنًا من `Initials` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _EmptyView` — خاص — الكلاس الذي يمثل شاشة `empty view`.
- `const _EmptyView()` — خاص — ينشئ كائنًا من `EmptyView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ErrorView` — خاص — الكلاس الذي يمثل شاشة `error view`.
- `const _ErrorView({required this.message, required this.onRetry})` — خاص — ينشئ كائنًا من `ErrorView` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

- `String _money(double value)` — خاص — ينفّذ منطق `money` داخل هذا الملف.

features/search/presentation/screens/search_screen.dart

- `class SearchScreen` — الكلاس الذي يمثل شاشة `search screen`.
- `const SearchScreen({super.key, this.userLatitude, this.userLongitude, this.categoryId, this.initialTabIndex = 0, this.enableCartFeatures = true})` — ينشئ كائنًا من `SearchScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<SearchScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _SearchScreenState` — خاص — الكلاس الذي يمثل شاشة `search screen state`.
- `bool get _showServicesTab` — خاص — يعرض أو يفتح `show services tab` داخل `_SearchScreenState`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onScroll()` — خاص — يعالج حدث `on scroll` داخل `_SearchScreenState`.
- `void _onSearchChanged(String value)` — خاص — يعالج حدث `on search changed` داخل `_SearchScreenState`.
- `void _openFilterSheet()` — خاص — يعرض أو يفتح `open filter sheet` داخل `_SearchScreenState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/search/presentation/widgets/category_chip.dart

- `class CategoryChip` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CategoryChip({super.key, required this.category, required this.isSelected, required this.onTap, this.badgeCount})` — ينشئ كائنًا من `CategoryChip` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _chipImage({required bool isSelected})` — خاص — ينفّذ منطق `chip image` داخل `CategoryChip`.

features/search/presentation/widgets/filter_bottom_sheet.dart

- `class FilterBottomSheet` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const FilterBottomSheet({super.key, this.hideProviderFilters = false})` — ينشئ كائنًا من `FilterBottomSheet` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _SectionTitle` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _SectionTitle({required this.title})` — خاص — ينشئ كائنًا من `SectionTitle` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _SortOptionTile` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _SortOptionTile({required this.label, required this.value, required this.groupValue, required this.onTap})` — خاص — ينشئ كائنًا من `SortOptionTile` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _LookupDropdown` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.

- `const _LookupDropdown({required this.hint, required this.isLoading, required this.items, required this.value, required this.onChanged, this.isDisabled = false})` — خاص — ينشئ كائنًا من `LookupDropdown` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _SortOption` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _SortOption({required this.value, required this.labelKey})` — خاص — ينشئ كائنًا من `SortOption` ويهيّئ الاعتماديات والقيم المطلوبة.

features/search/presentation/widgets/provider_card.dart

- `class ProviderCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderCard({super.key, required this.provider})` — ينشئ كائنًا من `ProviderCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildImageSection()` — خاص — ينفّذ منطق `build image section` داخل `ProviderCard`.
- `bool _hasText(String? value)` — خاص — يفحص شرط `has text` ويعيد النتيجة داخل `ProviderCard`.
- `class _DiscountChipsRow` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _DiscountChipsRow({this.categoryName, this.maxDiscount})` — خاص — ينشئ كائنًا من `DiscountChipsRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildChip(String label, Color bg, Color fg, FontWeight weight)` — خاص — ينفّذ منطق `build chip` داخل `_DiscountChipsRow`.
- `class _RatingAndLocationRow` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _RatingAndLocationRow({required this.provider})` — خاص — ينشئ كائنًا من `RatingAndLocationRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/search/presentation/widgets/search_app_bar.dart

- `class SearchAppBar` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const SearchAppBar({super.key, required this.controller, required this.onChanged, this.onFilterTap, this.hasActiveFilters = false})` — ينشئ كائنًا من `SearchAppBar` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<SearchAppBar> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _SearchAppBarState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `TextEditingController get controller` — خاص — ينفّذ منطق `controller` داخل `_SearchAppBarState`.
- `bool get _showOverlay` — خاص — يعرض أو يفتح `show overlay` داخل `_SearchAppBarState`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _onFocusChanged()` — خاص — يعالج حدث `on focus changed` داخل `_SearchAppBarState`.
- `void _onTextChanged()` — خاص — يعالج حدث `on text changed` داخل

_SearchAppBarState.

- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/search/presentation/widgets/search_providers_tab.dart

- `class SearchProvidersTab` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const SearchProvidersTab({super.key, required this.scrollController})` — ينشئ كائنًا من SearchProvidersTab ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ResultsBody` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ResultsBody({required this.scrollController})` — خاص — ينشئ كائنًا من ResultsBody ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _EmptyView` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _EmptyView()` — خاص — ينشئ كائنًا من EmptyView ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ErrorView` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ErrorView({required this.message, required this.onRetry})` — خاص — ينشئ كائنًا من ErrorView ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

features/search/presentation/widgets/search_services_tab.dart

- `class SearchServicesTab` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const SearchServicesTab({super.key, required this.enableCartFeatures})` — ينشئ كائنًا من SearchServicesTab ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<SearchServicesTab> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ServicesTabState` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `String? _currentServiceSearch(BuildContext context)` — خاص — ينفّذ منطق current service search داخل _ServicesTabState.
- `void _autoSelectFirstCategory(BuildContext context, SearchState state)` — خاص — ينفّذ منطق auto select first category داخل _ServicesTabState.
- `Widget _buildServicesBody(BuildContext context, SearchState state, int totalSelected)` — خاص — ينفّذ منطق build services body داخل _ServicesTabState.
- `class _ServiceCardConnector` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ServiceCardConnector({required this.serviceId, required this.categoryId, required this.serviceName, required this.isSelected})` — خاص — ينشئ كائنًا من ServiceCardConnector ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ServiceCard` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ServiceCard({required this.serviceName, required this.isSelected,`

- `const ServiceCard({required this.isPending, required this.onToggle})` — خاص — ينشئ كائنًا من `ServiceCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ChooseProviderBar` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ChooseProviderBar({required this.count})` — خاص — ينشئ كائنًا من `ChooseProviderBar` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`features/search/presentation/widgets/search_shimmer.dart`

- `class SearchShimmer` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const SearchShimmer({super.key})` — ينشئ كائنًا من `SearchShimmer` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ShimmerCard` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ShimmerCard()` — خاص — ينشئ كائنًا من `ShimmerCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Module: firebase_options.dart

`firebase_options.dart`

- `class DefaultFirebaseOptions` — الكلاس الذي يجمع مسؤوليات `default firebase options`.
- `static FirebaseOptions get currentPlatform` — ينفّذ منطق `current platform` داخل `DefaultFirebaseOptions`.

Module: main.dart

`main.dart`

- `void main()` — نقطة بدء التطبيق؛ تهيّئ الخدمات ثم تشغّل واجهة Flutter.
- `Future<void> _initializeNonBlockingServices()` — خاص — ينفّذ منطق `initialize non blocking services` داخل هذا الملف.
- `bool get _shouldInitializeFirebase` — خاص — يفحص شرط `should initialize firebase` ويعيد النتيجة داخل هذا الملف.
- `class MyApp` — الكلاس الذي يجمع مسؤوليات `my app`.
- `const MyApp({super.key, required this.initialLocation})` — ينشئ كائنًا من `MyApp` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<MyApp> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _MyAppState` — خاص — الكلاس الذي يجمع مسؤوليات `my app state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class AppVideoSplash` — الكلاس الذي يجمع مسؤوليات `app video splash`.
- `const AppVideoSplash({super.key, required this.onFinished})` — ينشئ كائنًا من

`AppVideoSplash` ويهيّئ الاعتماديات والقيم المطلوبة.

- `State<AppVideoSplash> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AppVideoSplashState` — خاص — الكلاس الذي يجمع مسؤوليات `app video splash` `state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void _handleVideoProgress()` — خاص — يعالج حدث `handle video progress` داخل `_AppVideoSplashState`.
- `void _finish()` — خاص — ينفّذ منطق `finish` داخل `_AppVideoSplashState`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Module: medicard_network

`medicard_network/domain/usecases/get_package_details_usecase.dart`

- `class GetPackageDetailsUseCase` — الكلاس الذي يعزل عملية أعمال واحدة: `get package` `details use case`.
- `GetPackageDetailsUseCase(this._repository)` — ينشئ كائنًا من `GetPackageDetailsUseCase` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ApiResult<PackageDetailsResponse>> call({required int packageId})` — ينفّذ الـ use case ويمرّر النتيجة إلى الطبقة المستدعية.

`medicard_network/logic/provider_page_cubit.dart`

- `class ProviderPageCubit` — الكلاس الذي يجمع مسؤوليات `provider page cubit`.
- `ProviderPageCubit(this._medicardRepository, this._invoiceRepository, this._cartRepository, this._cartSyncService, this.providerId)` — ينشئ كائنًا من `ProviderPageCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> init()` — ينفّذ منطق `init` داخل `ProviderPageCubit`.
- `Future<void> loadProviderPage()` — يحمّل `load provider page` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadPackages({bool reset = false})` — يحمّل `load packages` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadMorePackages()` — يحمّل `load more packages` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadReviews({bool reset = false})` — يحمّل `load reviews` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadMoreReviews()` — يحمّل `load more reviews` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadProviderServices({bool reset = false})` — يحمّل `load provider services` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadServices({bool reset = false})` — يحمّل `load services` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadMoreServices()` — يحمّل `load more services` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> loadCategories()` — يحمّل `load categories` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `void _updateTab(int categoryId, CategoryTab Function(CategoryTab)`

`update()` — خاص — يحدّث `update tab` داخل `ProviderPageCubit`.

- `bool _isArabic()` — خاص — يفحص شرط `is arabic` ويعيد النتيجة داخل `ProviderPageCubit`.
- `void _mergeCategoryServices(List<ProviderService> services)` — خاص — ينفّذ منطق `merge category services` داخل `ProviderPageCubit`.
- `Future<void> loadCategoryServices(int categoryId, {bool reset = false})` — يحمّل `load category services` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<void> selectCategoryTab(int index)` — يغيّر الاختيار الخاص بـ `select category tab` داخل `ProviderPageCubit`.
- `Future<void> retryTabLoad(int index)` — يعيد تحميل أو محاولة `retry tab load` داخل `ProviderPageCubit`.
- `Future<void> loadMoreCategoryServices()` — يحمّل `load more category services` ويحدّث الحالة المرتبطة داخل `ProviderPageCubit`.
- `Future<bool> requireAuth()` — ينفّذ منطق `require auth` داخل `ProviderPageCubit`.
- `Future<bool> addToCart(int serviceId, {int? categoryId})` — ينشئ أو يضيف `add to cart` داخل `ProviderPageCubit`.
- `Future<void> removeFromCart(int serviceId)` — يحذف أو يمسح `remove from cart` داخل `ProviderPageCubit`.
- `Future<void> togglePackageDetails(int packageId)` — يغيّر الاختيار الخاص بـ `toggle package details` داخل `ProviderPageCubit`.
- `Future<ApiResult<CreatedInvoice>> createInvoiceFromPackage({required int packageId, required int providerBranchId, bool isCash = true, String? cardNumber})` — ينشئ أو يضيف `create invoice from package` داخل `ProviderPageCubit`.
- `void updateSearchQuery(String query)` — يحدّث `update search query` داخل `ProviderPageCubit`.
- `Future<void> switchView(ProviderViewType view)` — ينفّذ منطق `switch view` داخل `ProviderPageCubit`.
- `List<ProviderService> _filterServices(List<ProviderService> services, String? query)` — خاص — ينفّذ منطق `filter services` داخل `ProviderPageCubit`.
- `void _setServiceSelected({required int serviceId, required bool isSelected, required Set<int> pendingCartActionIds})` — خاص — يحدّث `set service selected` داخل `ProviderPageCubit`.
- `void _onCartSyncEvent(CartSyncEvent event)` — خاص — يعالج حدث `on cart sync event` داخل `ProviderPageCubit`.
- `List<ProviderPackage> _filterPackages(List<ProviderPackage> packages, String? query)` — خاص — ينفّذ منطق `filter packages` داخل `ProviderPageCubit`.
- `List<ProviderReview> _filterReviews(List<ProviderReview> reviews, String? query)` — خاص — ينفّذ منطق `filter reviews` داخل `ProviderPageCubit`.
- `Future<void> close()` — يغلق الـ Cubit أو الـ streams والموارد التابعة له.

`medicard_network/logic/provider_page_state.dart`

- `enum ProviderViewType` — يعرّف القيم الثابتة الخاصة بـ `provider view type`.
- `class CategoryTab` — الكلاس الذي يجمع مسؤوليات `category tab`.
- `const CategoryTab({required this.categoryId, required this.categoryName, this.imageUrl, this.services, this.isLoading = false, this.hasMore = true, this.error})` — ينشئ كائنًا من `CategoryTab` ويهيّئ الاعتماديات والقيم المطلوبة.
- `int get selectedCount` — يغيّر الاختيار الخاص بـ `selected count` داخل `CategoryTab`.
- `CategoryTab copyWith({dynamic services, bool? isLoading, bool? hasMore, String? error, bool clearError = false})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

- `class ProviderPageState` — الكلاس الذي يجمع مسؤوليات `provider page state`.
- `const ProviderPageState({this.providerData, this.isProviderLoading = false, this.providerError, this.services = const [], this.fullServices = const [], this.isServicesLoading = false, this.isServicesLoadingMore = false, this.servicesHasMore = false, this.servicesError, this.categoryTabs = const [], this.selectedCategoryIndex = 0, this.packages = const [], this.fullPackages = const [], this.isPackagesLoading = false, this.isPackagesLoadingMore = false, this.packagesHasMore = true, this.packagesError, this.packageDetailsById = const {}, this.loadingPackageDetailsIds = const {}, this.packageDetailsErrors = const {}, this.reviews = const [], this.fullReviews = const [], this.isReviewsLoading = false, this.isReviewsLoadingMore = false, this.reviewsHasMore = true, this.reviewsError, this.currentView = ProviderViewType.services, this.catalogView = ProviderViewType.services, this.searchQuery, this.pendingCartActionIds = const {}, this.pendingPackageInvoiceIds = const {}})](../lib/medicard_network/logic/provider_page_state.dart#L91) — ينشئ كائنًا من `ProviderPageState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `CategoryTab? get selectedTab` — يغيّر الاختيار الخاص بـ `selected tab` داخل `ProviderPageState`.
- `int get displayServicesCount` — ينفّذ منطق `display services count` داخل `ProviderPageState`.
- `bool isServiceInCart(int serviceId)` — يفحص شرط `is service in cart` ويعيد النتيجة داخل `ProviderPageState`.
- `int getCartItemCount()` — يجلب `get cart item count` من مصدر البيانات المناسب داخل `ProviderPageState`.
- `ProviderPageState copyWith({ProviderPageData? providerData, bool resetProviderData = false, bool? isProviderLoading, String? providerError, bool resetProviderError = false, List<ProviderService>? services, List<ProviderService>? fullServices, bool? isServicesLoading, bool? isServicesLoadingMore, bool? servicesHasMore, String? servicesError, bool resetServicesError = false, List<CategoryTab>? categoryTabs, int? selectedCategoryIndex, List<ProviderPackage>? packages, List<ProviderPackage>? fullPackages, bool? isPackagesLoading, bool? isPackagesLoadingMore, bool? packagesHasMore, String? packagesError, bool resetPackagesError = false, Map<int, PackageDetailsData>? packageDetailsById, Set<int>? loadingPackageDetailsIds, Map<int, String>? packageDetailsErrors, List<ProviderReview>? reviews, List<ProviderReview>? fullReviews, bool? isReviewsLoading, bool? isReviewsLoadingMore, bool? reviewsHasMore, String? reviewsError, bool resetReviewsError = false, ProviderViewType? currentView, ProviderViewType? catalogView, String? searchQuery, bool resetSearchQuery = false, Set<int>? pendingCartActionIds, Set<int>? pendingPackageInvoiceIds})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.

medicard_network/presentation/pages/branches_screen.dart

- `class BranchesScreen` — الكلاس الذي يمثل شاشة `branches screen`.
- `const BranchesScreen({super.key, required this.branches, this.providerName})` — ينشئ كائنًا من `BranchesScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<BranchesScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _BranchesScreenState` — خاص — الكلاس الذي يمثل شاشة `branches screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.

- `List<ProviderBranch> get _filteredBranches` — خاص — ينفّذ منطق `filtered branches` داخل `_BranchesScreenState`.
- `String _branchesCountText(int count, BuildContext context)` — خاص — ينفّذ منطق `branches count text` داخل `_BranchesScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildHeader(BuildContext context)` — خاص — ينفّذ منطق `build header` داخل `_BranchesScreenState`.
- `class _BranchCard` — خاص — الكلاس الذي يمثل شاشة `branch card`.
- `const _BranchCard({required this.branch, required this.onTap})` — خاص — ينشئ كائنًا من `BranchCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> _openMap()` — خاص — يعرض أو يفتح `open map` داخل `_BranchCard`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/pages/provider_page_screen.dart`

- `class ProviderPageScreen` — الكلاس الذي يمثل شاشة `provider page screen`.
- `const ProviderPageScreen({super.key, required this.providerId, this.autoOpenPackageId})` — ينشئ كائنًا من `ProviderPageScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ProviderPageScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _ProviderPageScreenState` — خاص — الكلاس الذي يمثل شاشة `provider page screen state`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `bool _handleScrollNotification(ScrollNotification scrollInfo, ProviderPageState state, ProviderPageCubit cubit)` — خاص — يعالج حدث `handle scroll notification` داخل `_ProviderPageScreenState`.
- `void _openBranchSelector(List<ProviderBranch>? branches, String? providerName)` — خاص — يعرض أو يفتح `open branch selector` داخل `_ProviderPageScreenState`.
- `class _AutoOpenPackage` — خاص — الكلاس الذي يمثل شاشة `auto open package`.
- `const _AutoOpenPackage({required this.packageId, this.selectedBranch})` — خاص — ينشئ كائنًا من `AutoOpenPackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<_AutoOpenPackage> createState()` — خاص — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _AutoOpenPackageState` — خاص — الكلاس الذي يمثل شاشة `auto open package state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `Future<void> _open()` — خاص — يعرض أو يفتح `open` داخل `_AutoOpenPackageState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/pages/reviews_screen.dart`

- `class ReviewsScreen` — الكلاس الذي يمثل شاشة `reviews screen`.
- `const ReviewsScreen({super.key, required this.providerId, this.providerName})` — ينشئ كائنًا من `ReviewsScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<ReviewsScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.

- `class _ReviewsScreenState` — خاص — الكلاس الذي يمثل شاشة `reviews screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onScroll()` — خاص — يعالج حدث `on scroll` داخل `_ReviewsScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/category_service_card.dart`

- `class CategoryServiceCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CategoryServiceCard({super.key, required this.service, this.categoryId})` — ينشئ كائنًا من `CategoryServiceCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ServicePrice` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _ServicePrice({required this.service})` — خاص — ينشئ كائنًا من `ServicePrice` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/package_details_sheet.dart`

- `void openPackageDetailsSheet(BuildContext context, ProviderPageCubit cubit, ProviderPackage packageData, ProviderBranch? selectedBranch)` — يعرض أو يفتح `open package details sheet` داخل هذا الملف.
- `Future<void> createPackageInvoice(BuildContext context, ProviderPageCubit cubit, int packageId, ProviderBranch? selectedBranch)` — ينشئ أو يضيف `create package invoice` داخل هذا الملف.
- `Future<void> openPackageCheckout(BuildContext context, ProviderPageCubit cubit, ProviderPackage packageData, ProviderBranch? selectedBranch, {PackageDetailsData? details})` — يعرض أو يفتح `open package checkout` داخل هذا الملف.
- `class _PackageDetailsSheetBody` — خاص — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const _PackageDetailsSheetBody({required this.packageData, required this.selectedBranch})` — خاص — ينشئ كائنًا من `PackageDetailsSheetBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/provider_content_section.dart`

- `class ProviderContentSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderContentSection({super.key, required this.selectedBranch})` — ينشئ كائنًا من `ProviderContentSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class CategoryTabsSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CategoryTabsSection({super.key, required this.state})` — ينشئ كائنًا من `CategoryTabsSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ProviderServicesBody` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.

- `const ProviderServicesBody({super.key, required this.state})` — ينشئ كائنًا من `ProviderServicesBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class TabServicesBody` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const TabServicesBody({super.key, required this.state})` — ينشئ كائنًا من `TabServicesBody` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class PackagesSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PackagesSection({super.key, required this.state, required this.selectedBranch})` — ينشئ كائنًا من `PackagesSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ReviewsSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ReviewsSection({super.key, required this.state})` — ينشئ كائنًا من `ReviewsSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/provider_package_card.dart`

- `class ProviderPackageCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderPackageCard({super.key, required this.packageData, this.onDetailsTap, this.onQuickAdd})` — ينشئ كائنًا من `ProviderPackageCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/provider_rebuild_scopes.dart`

- `class ProviderRebuildScopes` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `static bool isHeaderSliceChanged(ProviderPageState previous, ProviderPageState current)` — يفحص شرط `is header slice changed` ويعيد النتيجة داخل `ProviderRebuildScopes`.
- `static bool isSearchRowSliceChanged(ProviderPageState previous, ProviderPageState current)` — يفحص شرط `is search row slice changed` ويعيد النتيجة داخل `ProviderRebuildScopes`.
- `static bool isCartBarSliceChanged(ProviderPageState previous, ProviderPageState current)` — يفحص شرط `is cart bar slice changed` ويعيد النتيجة داخل `ProviderRebuildScopes`.
- `static bool isContentSliceChanged(ProviderPageState previous, ProviderPageState current)` — يفحص شرط `is content slice changed` ويعيد النتيجة داخل `ProviderRebuildScopes`.
- `static bool isPackageSheetSliceChanged(int packageId, ProviderPageState previous, ProviderPageState current)` — يفحص شرط `is package sheet slice changed` ويعيد النتيجة داخل `ProviderRebuildScopes`.

`medicard_network/presentation/widgets/provider_review_card.dart`

- `class ProviderReviewCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderReviewCard({super.key, required this.review})` — ينشئ كائنًا من `ProviderReviewCard` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/provider_search_row.dart`

- `class ProviderSearchRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderSearchRow({super.key})` — ينشئ كائنًا من `ProviderSearchRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/provider_service_card.dart`

- `class ProviderServiceCard` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderServiceCard({super.key, required this.service})` — ينشئ كائنًا من `ProviderServiceCard` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`medicard_network/presentation/widgets/provider_shared_widgets.dart`

- `String formatPrice(double? price)` — ينسّق `format price` للعرض أو الإرسال داخل هذا الملف.
- `String packageImageUrl(String value)` — ينفّذ منطق `package image url` داخل هذا الملف.
- `class ProviderInfoRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderInfoRow({super.key, required this.data, required this.servicesCount, required this.selectedBranch, required this.onBranchTap, required this.onRatingTap})` — ينشئ كائنًا من `ProviderInfoRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class PackageServiceNames` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PackageServiceNames({super.key, required this.serviceNames})` — ينشئ كائنًا من `PackageServiceNames` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class PackagePlainServiceNames` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const PackagePlainServiceNames({super.key, required this.serviceNames})` — ينشئ كائنًا من `PackagePlainServiceNames` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class ServiceBadge` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ServiceBadge({super.key, required this.label, required this.color, required this.backgroundColor, this.icon})` — ينشئ كائنًا من `ServiceBadge` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class CategoryTabChip` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const CategoryTabChip({super.key, required this.tab, required this.isSelected, required this.onTap})` — ينشئ كائنًا من `CategoryTabChip` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

medicard_network/presentation/widgets/provider_top_header.dart

- `class ProviderTopHeader` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const ProviderTopHeader({super.key, required this.selectedBranch, required this.onBranchTap})` — ينشئ كائنًا من `ProviderTopHeader` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

medicard_network/repository/medicard_network_repository.dart

- `class MedicardNetworkRepository` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات `medicard network repository`.
- `MedicardNetworkRepository(this._apiService)` — ينشئ كائنًا من `MedicardNetworkRepository` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String _cleanError(Object error)` — خاص — ينفّذ منطق `clean error` داخل `MedicardNetworkRepository`.
- `void _logDebugError(String operation, Object error)` — خاص — ينفّذ منطق `log debug error` داخل `MedicardNetworkRepository`.
- `Future<ApiResult<ProviderPageResponse>> getProviderPage({required int providerId})` — يجلب `get provider page` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<List<ProviderBranch>>> getProviderBranchLocations({required int providerId})` — يجلب `get provider branch locations` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<List<ProviderCategory>>> getProviderCategories({required int providerId})` — يجلب `get provider categories` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<AllPackagesResponse>> getAllPackages({int pageNumber = 1, int pageSize = 15, String? search})` — يجلب `get all packages` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<ServicesByCategoryResponse>> getServicesByCategory({required int providerId, required int categoryId, int pageNumber = 1, int pageSize = 20})` — يجلب `get services by category` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<ProviderServicesResponse>> getProviderServices({required int providerId, String? search, int pageNumber = 1, int pageSize = 20})` — يجلب `get provider services` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<ProviderReviewsResponse>> getProviderReviews({required int providerId, int pageNumber = 1, int pageSize = 20})` — يجلب `get provider reviews` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<ProviderPackagesResponse>> getProviderPackages({required int providerId, int pageNumber = 1, int pageSize = 20})` — يجلب `get provider packages` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.
- `Future<ApiResult<PackageDetailsResponse>> getPackageDetails({required int packageId})` — يجلب `get package details` من مصدر البيانات المناسب داخل `MedicardNetworkRepository`.

medicard_network/service/medicard_network_api_service.dart

- `class MedicardNetworkApiService` — الكلاس الذي يقدّم خدمة `medicard network api` لباقي التطبيق.
- `factory MedicardNetworkApiService(Dio dio, {String baseUrl})` — ينشئ كائنًا من `MedicardNetworkApiService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<ProviderPageResponse> getProviderPage(@Path("providerId") int providerId)` — يجلب `get provider page` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<ProviderBranchesResponse> getProviderBranchLocations(@Path("providerId") int providerId)` — يجلب `get provider branch locations` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<ProviderServicesResponse> getProviderServices(@Query("providerId") int providerId, {@Query("search") String? search, @Query("pageNumber") int? pageNumber, @Query("pageSize") int? pageSize})` — يجلب `get provider services` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<ProviderReviewsResponse> getProviderReviews(@Path("providerId") int providerId, {@Query("pageNumber") int? pageNumber, @Query("pageSize") int? pageSize})` — يجلب `get provider reviews` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<ProviderPackagesResponse> getProviderPackages(@Path("providerId") int providerId, {@Query("pageNumber") int? pageNumber, @Query("pageSize") int? pageSize})` — يجلب `get provider packages` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<PackageDetailsResponse> getPackageDetails(@Path("packageId") int packageId)` — يجلب `get package details` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<AllPackagesResponse> getAllPackages({@Query("pageNumber") int? pageNumber, @Query("pageSize") int? pageSize, @Query("Search") String? search})` — يجلب `get all packages` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<ProviderCategoriesResponse> getProviderCategories(@Query("providerId") int providerId)` — يجلب `get provider categories` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.
- `Future<ServicesByCategoryResponse> getServicesByCategory(@Query("categoryId") int categoryId, @Query("providerId") int providerId, {@Query("pageNumber") int? pageNumber, @Query("pageSize") int? pageSize})` — يجلب `get services by category` من مصدر البيانات المناسب داخل `MedicardNetworkApiService`.

Module: network

`network/data/all_packages_models.dart`

- `class AllPackagesResponse` — الكلاس الذي يمثل بيانات `all packages response` وينقلها بين الطبقات.
- `const AllPackagesResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `AllPackagesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AllPackagesResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AllPackagesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class AllPackagesData` — الكلاس الذي يمثل بيانات `all packages data` وينقلها بين الطبقات.

- `const AllPackagesData({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `AllPackagesData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AllPackagesData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AllPackagesData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class AllPackage` — الكلاس الذي يمثل بيانات `all package` وينقلها بين الطبقات.
- `const AllPackage({this.id, this.name, this.providerId, this.providerNameAr, this.providerNameEn, this.description, this.image, this.priceBefore, this.priceAfter, this.discountPercentage, this.serviceNames})` — ينشئ كائنًا من `AllPackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory AllPackage.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `AllPackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `double? asDouble(Object? value)` — ينفّذ منطق `as double` داخل `AllPackage`.
- `String providerName(bool isArabic)` — ينفّذ منطق `provider name` داخل `AllPackage`.

network/data/provider_categories_models.dart

- `class ProviderCategoriesResponse` — الكلاس الذي يمثل بيانات `provider categories` وينقلها بين الطبقات.
- `const ProviderCategoriesResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProviderCategoriesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderCategoriesResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderCategoriesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class ProviderCategory` — الكلاس الذي يمثل بيانات `provider category` وينقلها بين الطبقات.
- `const ProviderCategory({this.imageUrl, this.id, this.name})` — ينشئ كائنًا من `ProviderCategory` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderCategory.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderCategory` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class ServicesByCategoryResponse` — الكلاس الذي يمثل بيانات `services by category` وينقلها بين الطبقات.
- `const ServicesByCategoryResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ServicesByCategoryResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ServicesByCategoryResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ServicesByCategoryResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class ServicesByCategoryData` — الكلاس الذي يمثل بيانات `services by category data` وينقلها بين الطبقات.
- `const ServicesByCategoryData({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `ServicesByCategoryData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ServicesByCategoryData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ServicesByCategoryData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `class ProviderCategoryService` — الكلاس الذي يمثل بيانات `provider category service` وينقلها بين الطبقات.
- `const ProviderCategoryService({this.id, this.serviceName, this.serviceNameAr, this.serviceNameEn, this.instruction, this.priceBefore, this.discountPercentage, this.priceAfter, this.isSpecialOffer})` — ينشئ كائنًا من `ProviderCategoryService` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory ProviderCategoryService.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderCategoryService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `double? asDouble(Object? value)` — ينفّذ منطق `as double` داخل `ProviderCategoryService`.
- `String displayName(bool isArabic)` — ينفّذ منطق `display name` داخل `ProviderCategoryService`.
- `ProviderService toProviderService({required int categoryId, required bool isArabic, bool? isSelected})` — يحوّل `to provider service` بين طبقات أو صيغ البيانات داخل `ProviderCategoryService`.

`network/data/provider_page_response_model.dart`

- `class ProviderPageResponse` — الكلاس الذي يمثل بيانات `provider page response` وينقلها بين الطبقات.
- `ProviderPageResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProviderPageResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderPageResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderPageResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderBranchesResponse` — الكلاس الذي يمثل بيانات `provider branches` `response` وينقلها بين الطبقات.
- `ProviderBranchesResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProviderBranchesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderBranchesResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderBranchesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderPageData` — الكلاس الذي يمثل بيانات `provider page data` وينقلها بين الطبقات.
- `ProviderPageData({required this.id, required this.name, required this.categoryName, this.categoryId, required this.imageUrl, this.hotLine, this.phoneNumber1, this.phoneNumber2, this.totalServices, this.averageRating, this.totalReviews, this.branches, this.services, this.pageNumber, this.pageSize, this.totalPages})` — ينشئ كائنًا من `ProviderPageData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderPageData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderPageData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `String get providerName` — ينفّذ منطق `provider name` داخل `ProviderPageData`.
- `String get providerLogo` — ينفّذ منطق `provider logo` داخل `ProviderPageData`.
- `int get providerId` — ينفّذ منطق `provider id` داخل `ProviderPageData`.
- `double? get rating` — ينفّذ منطق `rating` داخل `ProviderPageData`.
- `int? get reviewsCount` — ينفّذ منطق `reviews count` داخل `ProviderPageData`.
- `List<String>? get phones` — ينفّذ منطق `phones` داخل `ProviderPageData`.
- `int? get servicesCount` — ينفّذ منطق `services count` داخل `ProviderPageData`.
- `class ProviderBranch` — الكلاس الذي يمثل بيانات `provider branch` وينقلها بين الطبقات.
- `ProviderBranch({required this.branchId, required this.branchName, this.phoneNumber1, this.phoneNumber2, this.fullAddress, this.mapUrl, this.latitude, this.longitude, this.distanceKm})` — ينشئ كائنًا من

`ProviderBranch` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory ProviderBranch.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderBranch` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `String? get phone` — ينفّذ منطق `phone` داخل `ProviderBranch`.
- `String? get mapsUrl` — يحوّل `maps url` بين طبقات أو صيغ البيانات داخل `ProviderBranch`.
- `class ProviderReviewsResponse` — الكلاس الذي يمثل بيانات `provider reviews response` وينقلها بين الطبقات.
- `ProviderReviewsResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProviderReviewsResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderReviewsResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderReviewsResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderReviewsData` — الكلاس الذي يمثل بيانات `provider reviews data` وينقلها بين الطبقات.
- `ProviderReviewsData({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `ProviderReviewsData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderReviewsData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderReviewsData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderReview` — الكلاس الذي يمثل بيانات `provider review` وينقلها بين الطبقات.
- `ProviderReview({this.userName, this.userImageUrl, this.rating, this.ratingText, this.createdAt})` — ينشئ كائنًا من `ProviderReview` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderReview.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderReview` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderPackagesResponse` — الكلاس الذي يمثل بيانات `provider packages response` وينقلها بين الطبقات.
- `ProviderPackagesResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProviderPackagesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderPackagesResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderPackagesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderPackagesData` — الكلاس الذي يمثل بيانات `provider packages data` وينقلها بين الطبقات.
- `ProviderPackagesData({this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `ProviderPackagesData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderPackagesData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderPackagesData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderPackage` — الكلاس الذي يمثل بيانات `provider package` وينقلها بين الطبقات.
- `ProviderPackage({this.id, this.name, this.imageUrl, this.description,`

`this.priceBefore, this.priceAfter, this.discountPercentage, this.servicesCount})` — ينشئ كائنًا من `ProviderPackage` ويهيّئ الاعتماديات والقيم المطلوبة.

- `factory ProviderPackage.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderPackage` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class PackageDetailsResponse` — الكلاس الذي يمثل بيانات `package details response` وينقلها بين الطبقات.
- `PackageDetailsResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `PackageDetailsResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory PackageDetailsResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `PackageDetailsResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class PackageDetailsData` — الكلاس الذي يمثل بيانات `package details data` وينقلها بين الطبقات.
- `PackageDetailsData({this.id, this.name, this.description, this.image, this.priceBefore, this.priceAfter, this.discountPercentage, this.serviceNames, this.providerId})` — ينشئ كائنًا من `PackageDetailsData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory PackageDetailsData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `PackageDetailsData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.

`network/data/provider_services_response_model.dart`

- `class ProviderServicesResponse` — الكلاس الذي يمثل بيانات `provider services response` وينقلها بين الطبقات.
- `ProviderServicesResponse({this.httpStatusCode, this.errors, this.succeeded, this.message, this.data})` — ينشئ كائنًا من `ProviderServicesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderServicesResponse.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderServicesResponse` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderServicesData` — الكلاس الذي يمثل بيانات `provider services data` وينقلها بين الطبقات.
- `ProviderServicesData({this.groups, this.items, this.pageNumber, this.pageSize, this.totalCount, this.totalPages})` — ينشئ كائنًا من `ProviderServicesData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderServicesData.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderServicesData` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `List<ProviderService>? get services` — ينفّذ منطق `services` داخل `ProviderServicesData`.
- `Pagination? get pagination` — ينفّذ منطق `pagination` داخل `ProviderServicesData`.
- `class ProviderServicesGroup` — الكلاس الذي يمثل بيانات `provider services group` وينقلها بين الطبقات.
- `ProviderServicesGroup({this.categoryId, this.categoryName, this.services})` — ينشئ كائنًا من `ProviderServicesGroup` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderServicesGroup.fromJson(Map<String, dynamic> json)` — ينشئ

كائنًا من `ProviderServicesGroup` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `class ProviderService` — الكلاس الذي يمثل بيانات `provider service` وينقلها بين الطبقات.
- `ProviderService({this.id, this.serviceName, this.instruction, this.priceBefore, this.priceAfter, this.discountPercentage, this.isSpecialOffer, this.isSelected, this.categoryId})` — ينشئ كائنًا من `ProviderService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `factory ProviderService.fromJson(Map<String, dynamic> json)` — ينشئ كائنًا من `ProviderService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Map<String, dynamic> toJson()` — يحوّل الكائن إلى JSON صالح للإرسال أو التخزين.
- `bool? get hasSpecialOffer` — يفحص شرط `has special offer` ويعيد النتيجة داخل `ProviderService`.
- `ProviderService withCategoryId(int? nextCategoryId)` — ينفّذ منطق `with category id` داخل `ProviderService`.
- `ProviderService copyWith({bool? isSelected})` — ينشئ نسخة جديدة مع تغيير القيم المطلوبة فقط.
- `class Pagination` — الكلاس الذي يمثل بيانات `pagination` وينقلها بين الطبقات.
- `Pagination({required this.currentPage, required this.pageSize, required this.totalCount, required this.totalPages, required this.hasNextPage, required this.hasPreviousPage})` — ينشئ كائنًا من `Pagination` ويهيّئ الاعتماديات والقيم المطلوبة.

Module: presentation

`presentation/logic/medicard_login_cubit.dart`

- `class MedicardLoginCubit` — الكلاس الذي يجمع مسؤوليات `medicard login cubit`.
- `MedicardLoginCubit(this._loginCardUseCase)` — ينشئ كائنًا من `MedicardLoginCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> login({String? cardNumber, String? phoneNumber, required String password})` — ينفّذ تسجيل الدخول ويتعامل مع نتيجة المصادقة داخل `MedicardLoginCubit`.

`presentation/logic/medicard_login_state.dart`

- `class MedicardLoginState` — الكلاس الذي يجمع مسؤوليات `medicard login state`.
- `const factory MedicardLoginState.initial()` — ينشئ كائنًا من `MedicardLoginState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory MedicardLoginState.loading()` — ينشئ كائنًا من `MedicardLoginState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory MedicardLoginState.success(AuthResponseModel response)` — ينشئ كائنًا من `MedicardLoginState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory MedicardLoginState.failed({required String error})` — ينشئ كائنًا من `MedicardLoginState` ويهيّئ الاعتماديات والقيم المطلوبة.

`presentation/logic/register_cubit.dart`

- `class RegisterCubit` — الكلاس الذي يجمع مسؤوليات `register cubit`.
- `RegisterCubit(this._registerUseCase)` — ينشئ كائنًا من `RegisterCubit` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<void> register({required String firstName, required String lastName, required String phoneNumber, required bool isMale, required String birthDate, required String password, required String`

`confirmPassword, String? nationalId, String? passportNumber})` — يسجّل `register` لدى النظام أو الخدمة داخل `RegisterCubit`.

`presentation/logic/register_state.dart`

- `class RegisterState` — الكلاس الذي يجمع مسؤوليات `register state`.
- `const factory RegisterState.initial()` — ينشئ كائنًا من `RegisterState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory RegisterState.loading()` — ينشئ كائنًا من `RegisterState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory RegisterState.success(AuthResponseModel response)` — ينشئ كائنًا من `RegisterState` ويهيّئ الاعتماديات والقيم المطلوبة.
- `const factory RegisterState.failed({required String error})` — ينشئ كائنًا من `RegisterState` ويهيّئ الاعتماديات والقيم المطلوبة.

`presentation/medicard_login_screen.dart`

- `class MediCardLoginScreen` — الكلاس الذي يجمع مسؤوليات `medi card login screen`.
- `const MediCardLoginScreen({super.key})` — ينشئ كائنًا من `MediCardLoginScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<MediCardLoginScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _MediCardLoginScreenState` — خاص — الكلاس الذي يجمع مسؤوليات `medi card login screen state`.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onLoginPressed(BuildContext context)` — خاص — يعالج حدث `on login pressed` داخل `_MediCardLoginScreenState`.
- `String? _validateLoginIdentifier(String? value)` — خاص — يتحقق من صحة `validate login identifier` داخل `_MediCardLoginScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `Widget _buildPhoneField()` — خاص — ينفّذ منطق `build phone field` داخل `_MediCardLoginScreenState`.
- `Widget _buildPasswordField()` — خاص — ينفّذ منطق `build password field` داخل `_MediCardLoginScreenState`.
- `class _LoginIdentifier` — خاص — الكلاس الذي يجمع مسؤوليات `login identifier`.
- `const _LoginIdentifier({this.cardNumber, this.phoneNumber})` — خاص — ينشئ كائنًا من `LoginIdentifier` ويهيّئ الاعتماديات والقيم المطلوبة.
- `_LoginIdentifier _parseLoginIdentifier(String raw)` — خاص — يحلّل `parse login identifier` ويحوّله لصيغة قابلة للاستخدام داخل هذا الملف.
- `bool _isEgyptianMobile(String value)` — خاص — يفحص شرط `is egyptian mobile` ويعيد النتيجة داخل هذا الملف.
- `bool _looksLikePhoneAttempt(String value)` — خاص — ينفّذ منطق `looks like phone attempt` داخل هذا الملف.
- `bool _isPotentialCardNumber(String value)` — خاص — يفحص شرط `is potential card number` ويعيد النتيجة داخل هذا الملف.
- `class _OrDivider` — خاص — الكلاس الذي يجمع مسؤوليات `or divider`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ActivateCardButton` — خاص — الكلاس الذي يجمع مسؤوليات `activate card button`.
- `const _ActivateCardButton({required this.onPressed})` — خاص — ينشئ كائنًا من

`ActivateCardButton` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class _ContinueAsGuestButton` — خاص — الكلاس الذي يجمع مسؤوليات `continue as guest button`.
- `const _ContinueAsGuestButton({required this.onPressed})` — خاص — ينشئ كائنًا من `ContinueAsGuestButton` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`presentation/register_screen.dart`

- `class RegisterScreen` — الكلاس الذي يجمع مسؤوليات `register screen`.
- `const RegisterScreen({super.key})` — ينشئ كائنًا من `RegisterScreen` ويهيّئ الاعتماديات والقيم المطلوبة.
- `State<RegisterScreen> createState()` — ينشئ كائن الحالة المرتبط بالـ StatefulWidget.
- `class _RegisterScreenState` — خاص — الكلاس الذي يجمع مسؤوليات `register screen state`.
- `void initState()` — خاص — يهيّئ حالة الـ widget والاشتراكات عند إنشائها.
- `void dispose()` — خاص — يلغي الاشتراكات ويحرّر الموارد عند انتهاء عمر الكائن.
- `void _onPasswordChanged()` — خاص — يعالج حدث `on password changed` داخل `_RegisterScreenState`.
- `String? _validatePassword(String? value)` — خاص — يتحقق من صحة `validate password` داخل `_RegisterScreenState`.
- `String? _validateConfirmPassword(String? value)` — خاص — يتحقق من صحة `validate confirm password` داخل `_RegisterScreenState`.
- `bool _hasUsableAuthSession(AuthResponseModel response)` — خاص — يفحص شرط `has usable auth session` ويعيد النتيجة داخل `_RegisterScreenState`.
- `bool _isPasswordValid(String password)` — خاص — يفحص شرط `is password valid` ويعيد النتيجة داخل `_RegisterScreenState`.
- `bool _hasMinimumLength(String password)` — خاص — يفحص شرط `has minimum length` ويعيد النتيجة داخل `_RegisterScreenState`.
- `bool _hasLetter(String password)` — خاص — يفحص شرط `has letter` ويعيد النتيجة داخل `_RegisterScreenState`.
- `bool _hasNumber(String password)` — خاص — يفحص شرط `has number` ويعيد النتيجة داخل `_RegisterScreenState`.
- `bool _hasSpecialCharacter(String password)` — خاص — يفحص شرط `has special character` ويعيد النتيجة داخل `_RegisterScreenState`.
- `String? _validateName(String? value, {required String requiredKey})` — خاص — يتحقق من صحة `validate name` داخل `_RegisterScreenState`.
- `String? _validatePhoneNumber(String? value)` — خاص — يتحقق من صحة `validate phone number` داخل `_RegisterScreenState`.
- `String? _validateNationalId(String? value)` — خاص — يتحقق من صحة `validate national id` داخل `_RegisterScreenState`.
- `String? _validatePassportNumber(String? value)` — خاص — يتحقق من صحة `validate passport number` داخل `_RegisterScreenState`.
- `void _clearInputFocus()` — خاص — يحذف أو يمسح `clear input focus` داخل `_RegisterScreenState`.
- `Future<void> _selectBirthDate()` — خاص — يغيّر الاختيار الخاص بـ `select birth date`

داخل `_RegisterScreenState`.

- `void _onRegisterPressed(BuildContext context)` — خاص — يعالج حدث `on register pressed` داخل `_RegisterScreenState`.
- `Widget build(BuildContext context)` — خاص — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`presentation/widgets/register/register_account_section.dart`

- `class RegisterAccountSection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterAccountSection({super.key, required this.phoneNumberController, required this.phoneNumberFocusNode, required this.phoneNumberAutovalidate, required this.phoneNumberValidator, required this.isMale, required this.onMaleTap, required this.onFemaleTap, required this.birthDate, required this.onSelectBirthDate, required this.passwordController, required this.obscurePassword, required this.onPasswordToggle, required this.passwordFocusNode, required this.passwordAutovalidate, required this.passwordValidator, required this.passwordMinLength, required this.confirmPasswordFieldKey, required this.confirmPasswordController, required this.obscureConfirmPassword, required this.onConfirmPasswordToggle, required this.confirmPasswordFocusNode, required this.confirmPasswordAutovalidate, required this.confirmPasswordValidator})` — ينشئ كائنًا من `RegisterAccountSection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`presentation/widgets/register/register_birth_date_field.dart`

- `class RegisterBirthDateField` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterBirthDateField({super.key, required this.birthDate, required this.onTap})` — ينشئ كائنًا من `RegisterBirthDateField` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`presentation/widgets/register/register_gender_selector.dart`

- `class RegisterGenderSelector` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterGenderSelector({super.key, required this.isMale, required this.onMaleTap, required this.onFemaleTap})` — ينشئ كائنًا من `RegisterGenderSelector` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

`presentation/widgets/register/register_identity_section.dart`

- `class RegisterIdentitySection` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterIdentitySection({super.key, required this.nationalIdFieldKey, required this.nationalIdController, required this.nationalIdFocusNode, required this.nationalIdAutovalidate, required this.nationalIdValidator, required this.passportNumberController, required this.passportNumberFocusNode, required this.passportNumberAutovalidate, required this.passportNumberValidator, required this.onPassportChanged})` — ينشئ كائنًا من `RegisterIdentitySection` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات

الحالية.

presentation/widgets/register/register_name_row.dart

- `class RegisterNameRow` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterNameRow({super.key, required this.firstNameController, required this.lastNameController, required this.firstNameFocusNode, required this.lastNameFocusNode, required this.firstNameAutovalidate, required this.lastNameAutovalidate, required this.firstNameValidator, required this.lastNameValidator})` — ينشئ كائنًا من `RegisterNameRow` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

presentation/widgets/register/register_password_field.dart

- `class RegisterPasswordField` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterPasswordField({super.key, this.fieldKey, required this.controller, required this.hintText, required this.obscure, required this.onToggle, this.validator, this.focusNode, this.autovalidateMode = AutovalidateMode.disabled})` — ينشئ كائنًا من `RegisterPasswordField` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

presentation/widgets/register/register_password_requirements.dart

- `class RegisterPasswordRequirements` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterPasswordRequirements({super.key, required this.password, required this.minLength})` — ينشئ كائنًا من `RegisterPasswordRequirements` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.
- `class RegisterPasswordRequirement` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterPasswordRequirement({super.key, required this.label, required this.isMet, required this.showInvalid})` — ينشئ كائنًا من `RegisterPasswordRequirement` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

presentation/widgets/register/register_required_hint.dart

- `class RegisterRequiredHint` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterRequiredHint(this.text, {super.key})` — ينشئ كائنًا من `RegisterRequiredHint` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

presentation/widgets/register/register_text_field.dart

- `class RegisterTextField` — الكلاس الذي يبني مكوّن واجهة قابلًا لإعادة الاستخدام.
- `const RegisterTextField({super.key, this.fieldKey, required this.controller, required this.hintText, this.keyboardType, this.required = true, this.validator, this.onChanged, this.inputFormatters, this.focusNode, this.autovalidateMode = AutovalidateMode.disabled})` — ينشئ كائنًا من `RegisterTextField` ويهيّئ الاعتماديات والقيم المطلوبة.

- `Widget build(BuildContext context)` — يبني جزء واجهة Flutter اعتمادًا على الحالة والمدخلات الحالية.

Module: repository

`repository/medicard_repository_impl.dart`

- `class MedicardRepositoryImpl` — الكلاس المسؤول عن عقد أو تنفيذ الوصول إلى بيانات `medicard repository impl`.
- `MedicardRepositoryImpl(this._apiService)` — ينشئ كائنًا من `MedicardRepositoryImpl` ويهيّئ الاعتماديات والقيم المطلوبة.
- `String _formatApiMessages(dynamic value)` — خاص — ينسّق `format api messages` للعرض أو الإرسال داخل `MedicardRepositoryImpl`.
- `String _friendlyChangePasswordMessage(String message)` — خاص — ينفّذ منطق `friendly change password message` داخل `MedicardRepositoryImpl`.
- `bool _isRegisterSuccess(AuthResponseModel response)` — خاص — يفحص شرط `is register success` ويعيد النتيجة داخل `MedicardRepositoryImpl`.
- `bool _isLoginSuccess(AuthResponseModel response)` — خاص — يفحص شرط `is login success` ويعيد النتيجة داخل `MedicardRepositoryImpl`.
- `String _authFailureMessage(AuthResponseModel response)` — خاص — ينفّذ منطق `auth failure message` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<AuthResponseModel>> register(AuthRegisterRequestModel request)` — يسجّل `register` لدى النظام أو الخدمة داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<AuthResponseModel>> login(AuthLoginRequestModel request)` — ينفّذ تسجيل الدخول ويتعامل مع نتيجة المصادقة داخل `MedicardRepositoryImpl`.
- `String _otpFailureMessage(OtpResponseModel response)` — خاص — ينفّذ منطق `otp failure message` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<OtpResponseModel>> sendOtp(String phoneNumber)` — يرسل `send otp` إلى الطبقة أو الخدمة المطلوبة داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<OtpResponseModel>> verifyOtp({required String phoneNumber, required String otp})` — ينفّذ منطق `verify otp` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<OtpResponseModel>> resetPassword({required String phoneNumber, required String password, required String confirmPassword})` — ينفّذ منطق `reset password` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<CardHomeInfoResponseModel>> getHomeInfo(String cardNo, String lang)` — يجلب `get home info` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<CardPersonalInfoResponseModel>> getPersonalInfo(String cardNo, String lang)` — يجلب `get personal info` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<CardSupportContactResponseModel>> getSupportContacts(String lang)` — يجلب `get support contacts` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<CardUpdateProfileResponseModel>> updateProfile(CardUpdateProfileRequestModel request, String lang)` — يحدّث `update profile` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<HomeResponseModel>> getHomeData({double? latitude, double? longitude})` — يجلب `get home data` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.

- `Future<ApiResult<ProfileResponseModel>> getProfile()` — يجلب `get profile` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<ProfileSummaryResponseModel>> getProfileSummary()` — يجلب `get profile summary` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<ProfileResponseModel>> deleteProfileImage()` — يحذف أو يمسح `delete profile image` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<MemberCardActivationResponseModel>> activateMemberCard({required String cardNumber})` — ينفّذ منطق `activate member card` داخل `MedicardRepositoryImpl`.
- `String _memberCardActivationBackendMessage(MemberCardActivationResponseModel response)` — خاص — ينفّذ منطق `member card activation backend message` داخل `MedicardRepositoryImpl`.
- `String _backendMessageFromDio(Object error)` — خاص — ينفّذ منطق `backend message from dio` داخل `MedicardRepositoryImpl`.
- `String _extractBackendMessages(dynamic value)` — خاص — ينفّذ منطق `extract backend messages` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<CardDataResponseModel>> getCardData()` — يجلب `get card data` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<ProfileResponseModel>> updateProfileV2({required String firstName, required String lastName, required String phoneNumber, required String nationalId, String? passportNumber, required bool isMale, required String birthDate, String? imagePath, bool removeImage = false, String? currentImageUrl})` — يحدّث `update profile v2` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<ChangePasswordResponseModel>> changePassword({required String currentPassword, required String newPassword, required String confirmNewPassword})` — يحدّث `change password` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<ChangeLanguageResponseModel>> changeLanguage({required String language})` — يحدّث `change language` داخل `MedicardRepositoryImpl`.
- `Future<ApiResult<GuestHomeResponseModel>> getGuestHomeData({double? latitude, double? longitude})` — يجلب `get guest home data` من مصدر البيانات المناسب داخل `MedicardRepositoryImpl`.

Module: service

service/medicard_api_service.dart

- `class MedicardApiService` — الكلاس الذي يقدّم خدمة `medicard api service` لباقي التطبيق.
- `factory MedicardApiService(Dio dio, {String baseUrl})` — ينشئ كائنًا من `MedicardApiService` ويهيّئ الاعتماديات والقيم المطلوبة.
- `Future<AuthResponseModel> register(@Body() AuthRegisterRequestModel request)` — يسجّل `register` لدى النظام أو الخدمة داخل `MedicardApiService`.
- `Future<AuthResponseModel> login(@Body() AuthLoginRequestModel request)` — ينفّذ تسجيل الدخول ويتعامل مع نتيجة المصادقة داخل `MedicardApiService`.
- `Future<OtpResponseModel> sendOtp(@Body() SendOtpRequestModel request)` — يرسل `send otp` إلى الطبقة أو الخدمة المطلوبة داخل `MedicardApiService`.
- `Future<OtpResponseModel> verifyOtp(@Body() VerifyOtpRequestModel request)` — ينفّذ منطق `verify otp` داخل `MedicardApiService`.
- `Future<OtpResponseModel> resetPassword(@Body() ResetPasswordRequestModel`

request) — ينفّذ منطق reset password داخل MedicardApiService.

- Future<CardHomeInfoResponseModel> getHomeInfo(@Path("lang") String lang, @Query("CardNo") String cardNo) — يجلب get home info من مصدر البيانات المناسب داخل MedicardApiService.
- Future<CardPersonalInfoResponseModel> getPersonalInfo(@Path("lang") String lang, @Query("CardNo") String cardNo) — يجلب get personal info من مصدر البيانات المناسب داخل MedicardApiService.
- Future<CardSupportContactResponseModel> getSupportContacts(@Path("lang") String lang) — يجلب get support contacts من مصدر البيانات المناسب داخل MedicardApiService.
- Future<HomeResponseModel> getHomeData(@Query("latitude") double? latitude, @Query("longitude") double? longitude) — يجلب get home data من مصدر البيانات المناسب داخل MedicardApiService.
- Future<CardUpdateProfileResponseModel> updateProfile(@Path("lang") String lang, @Part(name: "CardNo") String cardNo, @Part(name: "FirstName") String? firstName, @Part(name: "LastName") String? lastName, @Part(name: "PhoneNumber") String? phoneNumber, @Part(name: "Email") String? email, @Part(name: "NationalId") String? nationalId, @Part(name: "PassportNumber") String? passportNumber, @Part(name: "Birthdate") String? birthdate, @Part(name: "IsMale") bool? isMale, @Part(name: "ProfileImage") MultipartFile? profileImage) — يحدّث update profile داخل MedicardApiService.
- Future<ProfileResponseModel> getProfile() — يجلب get profile من مصدر البيانات المناسب داخل MedicardApiService.
- Future<ProfileSummaryResponseModel> getProfileSummary() — يجلب get profile summary من مصدر البيانات المناسب داخل MedicardApiService.
- Future<CardDataResponseModel> getCardData() — يجلب get card data من مصدر البيانات المناسب داخل MedicardApiService.
- Future<ProfileResponseModel> deleteProfileImage() — يحذف أو يمسح delete profile image داخل MedicardApiService.
- Future<MemberCardActivationResponseModel> activateMemberCard(@Body() Map<String, dynamic> body) — ينفّذ منطق activate member card داخل MedicardApiService.
- Future<ProfileResponseModel> updateProfileV2(@Part(name: 'FirstName') String firstName, @Part(name: 'LastName') String lastName, @Part(name: 'PhoneNumber') String phoneNumber, @Part(name: 'NationalId') String nationalId, @Part(name: 'PassportNumber') String? passportNumber, @Part(name: 'IsMale') bool isMale, @Part(name: 'BirthDate') String birthDate, @Part(name: 'Image') MultipartFile? image) — يحدّث update profile v2 داخل MedicardApiService.
- Future<ChangePasswordResponseModel> changePassword(@Header('Authorization') String token, @Body() Map<String, dynamic> body) — يحدّث change password داخل MedicardApiService.
- Future<ChangeLanguageResponseModel> changeLanguage(@Header('Authorization') String token, @Body() ChangeLanguageRequestModel body) — يحدّث change language داخل MedicardApiService.
- Future<ContentResponseModel> getContent(@Path('slug') String slug) — يجلب get content من مصدر البيانات المناسب داخل MedicardApiService.
- Future<GetFamilyMembersResponseModel> getFamilyMembers() — يجلب get family members من مصدر البيانات المناسب داخل MedicardApiService.

- `Future<GetFamilyMemberResponseModel> getFamilyMember(@Path('id') int id)` — يجلب `get family member` من مصدر البيانات المناسب داخل `MedicardApiService`.
- `Future<FamilyMemberOperationResponseModel> createFamilyMember(@Body() CreateFamilyMemberRequestModel request)` — ينشئ أو يضيف `create family member` داخل `MedicardApiService`.
- `Future<FamilyMemberOperationResponseModel> updateFamilyMember(@Path('id') int id, @Body() CreateFamilyMemberRequestModel request)` — يحدّث `update family member` داخل `MedicardApiService`.
- `Future<FamilyMemberOperationResponseModel> deleteFamilyMember(@Path('id') int id)` — يحذف أو يمسح `delete family member` داخل `MedicardApiService`.
- `Future<GuestHomeResponseModel> getGuestHomeData(@Query('latitude') double? latitude, @Query('longitude') double? longitude)` — يجلب `get guest home data` من مصدر البيانات المناسب داخل `MedicardApiService`.